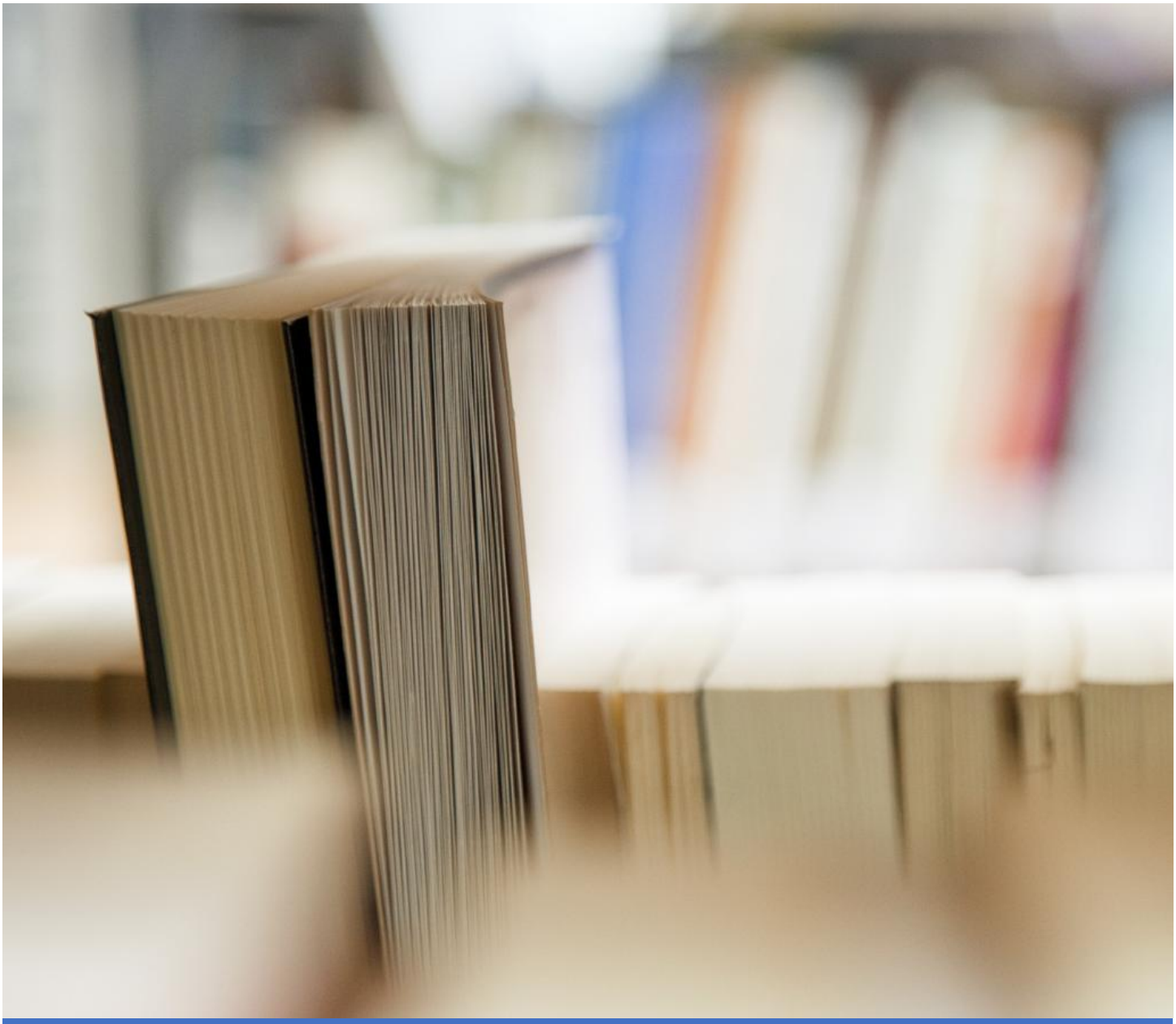
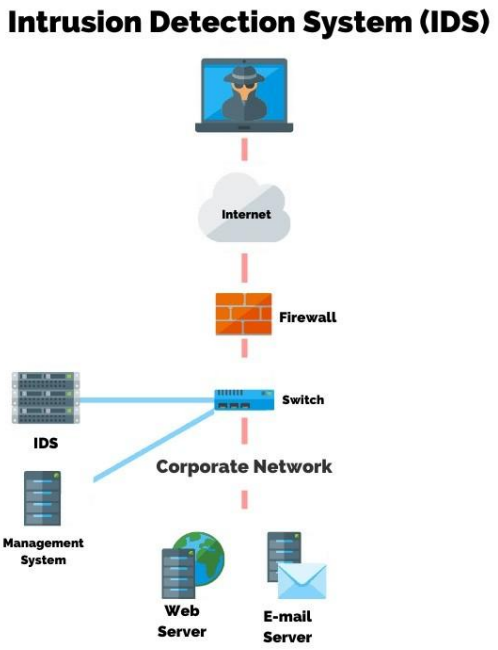


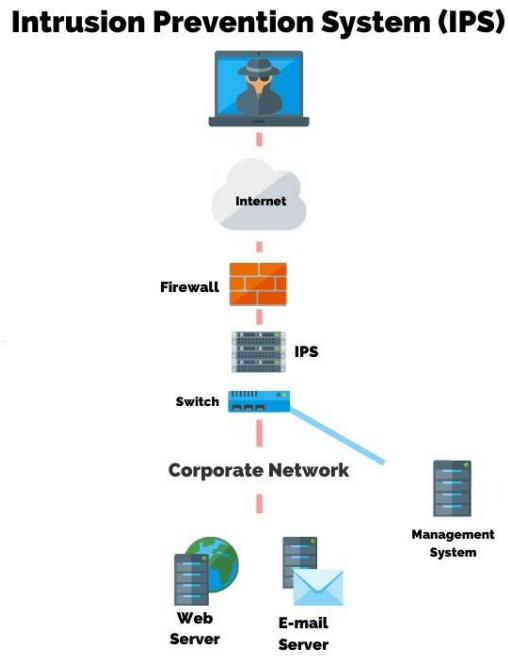
Project Report: Configure an Intrusion Detection System (IDS) and Intrusion Prevention System (IPS)

Magaseng John Manchidi

10/6/25

Computer Security(SCSE082)





Executive summary

In order to safeguard a vulnerable Metasploitable virtual machine this project installs Snort on an Ubuntu virtual machine and sets up an intrusion detection and prevention system (IDS/IPS). While the Snort sensor operates in IDS mode for detection and IPS/inline mode to actively drop malicious packets the attacker virtual machine (Kali) initiates reconnaissance and exploitation tests (nmap scans brute force and basic web attacks). The report details how to set up the environment in VirtualBox (host-only/internal networking) how to install and configure Snort on Ubuntu how to create and adjust custom rules how to use Kali and Metasploitable to simulate attacks the results (alerts dropped packets false positives) performance observations and suggestions. Key findings: When rules were available Snort reliably recognized scripted scans and popular exploit signatures; some exploitation attempts were stopped by switching to inline (drop/block) mode; and it is crucial to adjust rules and suppressions to keep false positives under control. (Sources: Metasploitable documentation VirtualBox host-only/bridged VM networking guidelines official Snort documents and useful Snort recommendations.)

Contents

1.Introduction 3
Network Configuration for IDS/IPS Lab: Figure Analysis 3
Summary of the Lab Network 8
Network Connectivity Verification for IDS/IPS Lab: Figure Analysis 9
2.Snort IDS/IPS Rule Configuration: Figure Analysis 14
3.Snort Installation and Environment Setup: Figure Analysis 18
4. Attacking Metasploitable using kali and Ubuntu for IDS and IPS 21
Rule 1: ICMP Ping Detection & Prevention 21
RULE 2: Telnet Traffic Detection & Prevention 24
RULE 3 : FTP Traffic Detection & Prevention 27
Rule 4: HTTP Admin Access Detection & Prevention 30
Rule 5 - SSH Traffic Detection & Prevention Test Results 33
Rules 6-8 - r-services Detection & Prevention Test Results 36
Rule 9 - UnrealIRCd Backdoor Detection & Prevention Test Results 39
Rule 3 & Rule 10 - FTP Traffic & Backdoor Exploit Testing 42
Rules 12-13 - Port Scan Detection & Prevention Test Results 45
Snort Installation and Configuration Summary 48
Kali Configuration Summary 53
VIRTUALBOX MACHINES SETUP NETWORK SUMMARY 63

1.Introduction

In order to identify attacks and when necessary immediately stop harmful traffic modern networks need constant monitoring. In this project Kali (attacker) Ubuntu (sensor) and Metasploitable2 (target) are used to illustrate configuration attack simulation and evaluation in a reproducible VirtualBox lab. Snort is used as the detection/prevention engine. Metasploitable is frequently used for both offensive and defensive testing and it is purposefully vulnerable.
Docs for Rapid7

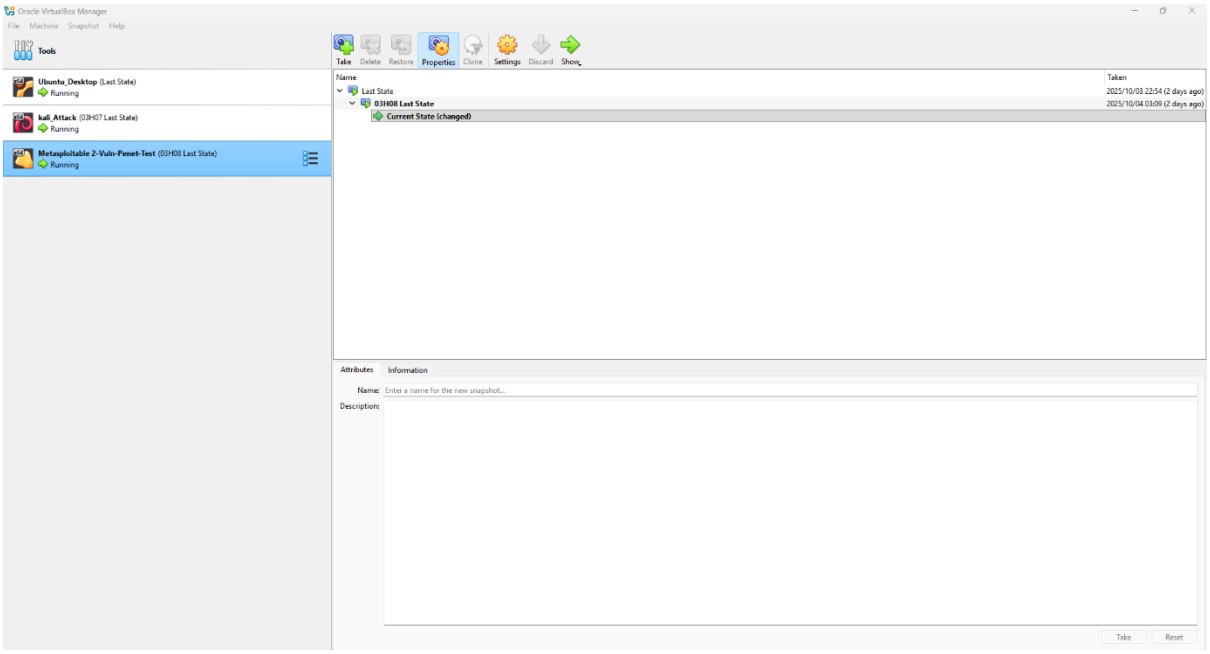


Figure 1.0 IDS_IPS_Setup

In figure 1.0 there are three virtual machines—Ubuntu Desktop Kali Attack and Metasploitable 2 Vul-Penet-Test—are running in the virtual machine environment depicted in the picture which shows the Oracle VirtualBox Manager interface. Snapshots of each virtual machine were obtained at 23:02 on October 3 2025 and the selected Metasploitable 2 Vul-Penet-Test VM's current state was marked as "Current State (changed)". The capacity to save restore or alter the virtual machine states is indicated by the interface's snapshot management choices (Take Delete Restore Properties Clone Settings Discard Show). A standard Windows desktop environment hosting the virtual machines is suggested by the taskbar at the bottom which displays the current date and time as October 6 2025 14:59 a weather widget that reads "Sunny" at 31°C and a number of application icons.

Network Configuration for IDS/IPS Lab: Figure Analysis

In order to provide a controlled environment for testing the Snort IDS/IPS the following figures show how to set up a virtual lab network and assign static IP addresses to three virtual computers.

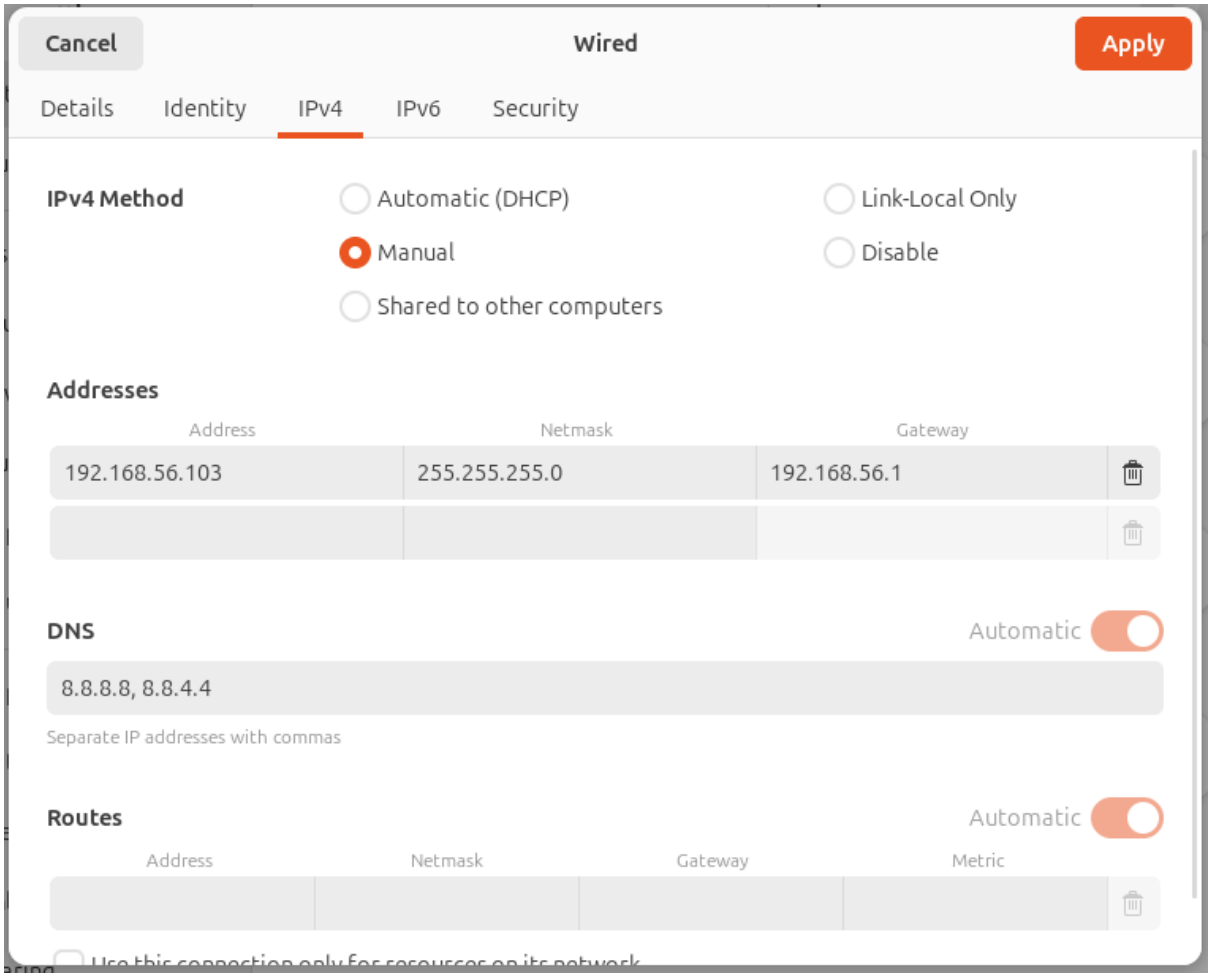


Figure 1.1: Static IP Configuration on Ubuntu Sensor VM

Figure 1.1: Ubuntu Sensor VM Static IP Configuration
The Ubuntu computer that will serve as the IDS/IPS Sensor's network settings window is depicted in this illustration.

IPv4 Method: Manually configured. In contrast to Automatic (DHCP) where the IP address may fluctuate this guarantees the sensor has a constant predictable IP address which is essential for a lab setting.

Address Assigned Manually:

Provided by: 192.168.56.103

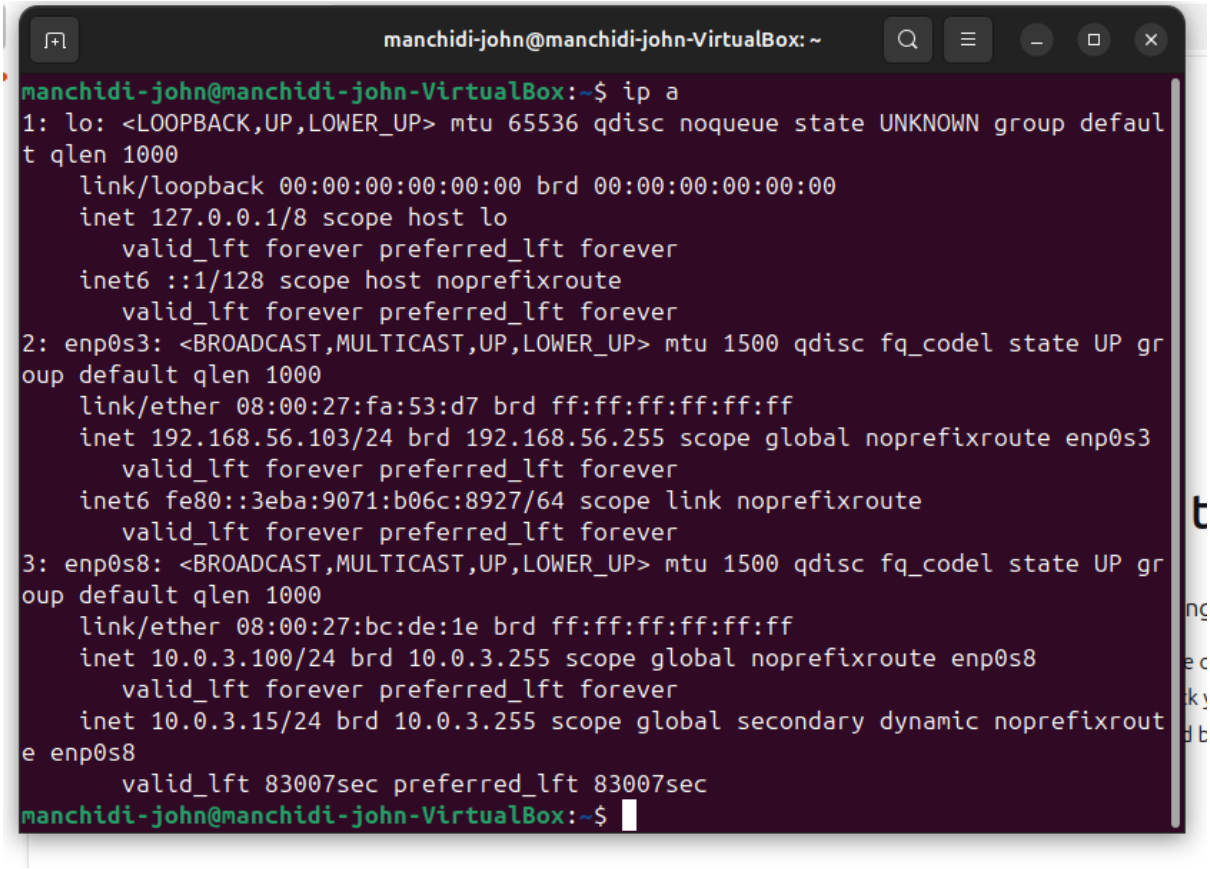
Netmask: 255.255.255.0 a subnet of /24

Port of entry: 192.168.56.1

Google's public DNS servers which provide dependable domain name resolution are 8.8.8.8 8.8.4.4.

The goal of creating a static IP on the 192.168.56.0/24 network is to have this Ubuntu computer ready for Snort installation. All traffic on this network segment will be observed by it.

Figure 1.2: Network Interface Verification on Ubuntu Sensor VM

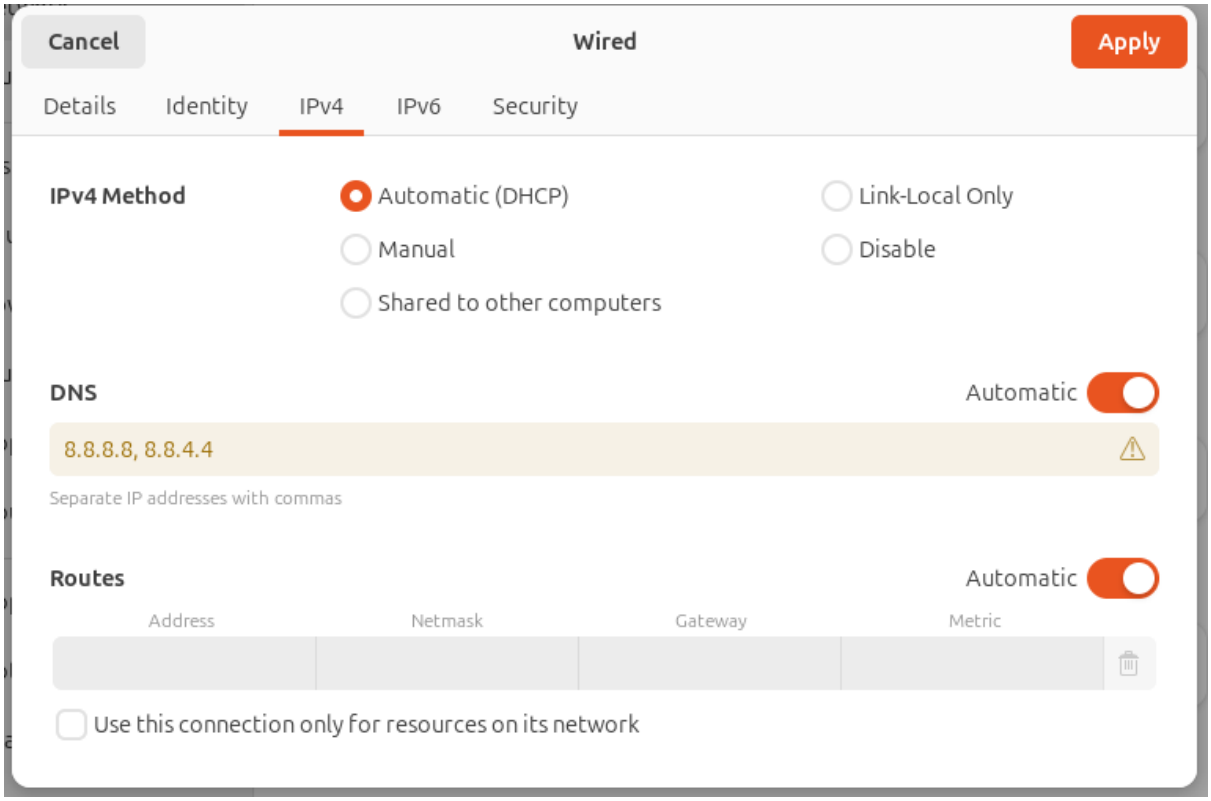


This terminal output from the Ubuntu Sensor virtual machine (VM) provides further information about the network setup and validates the successful configuration from Figure 1.1.

Interface enp0s3:

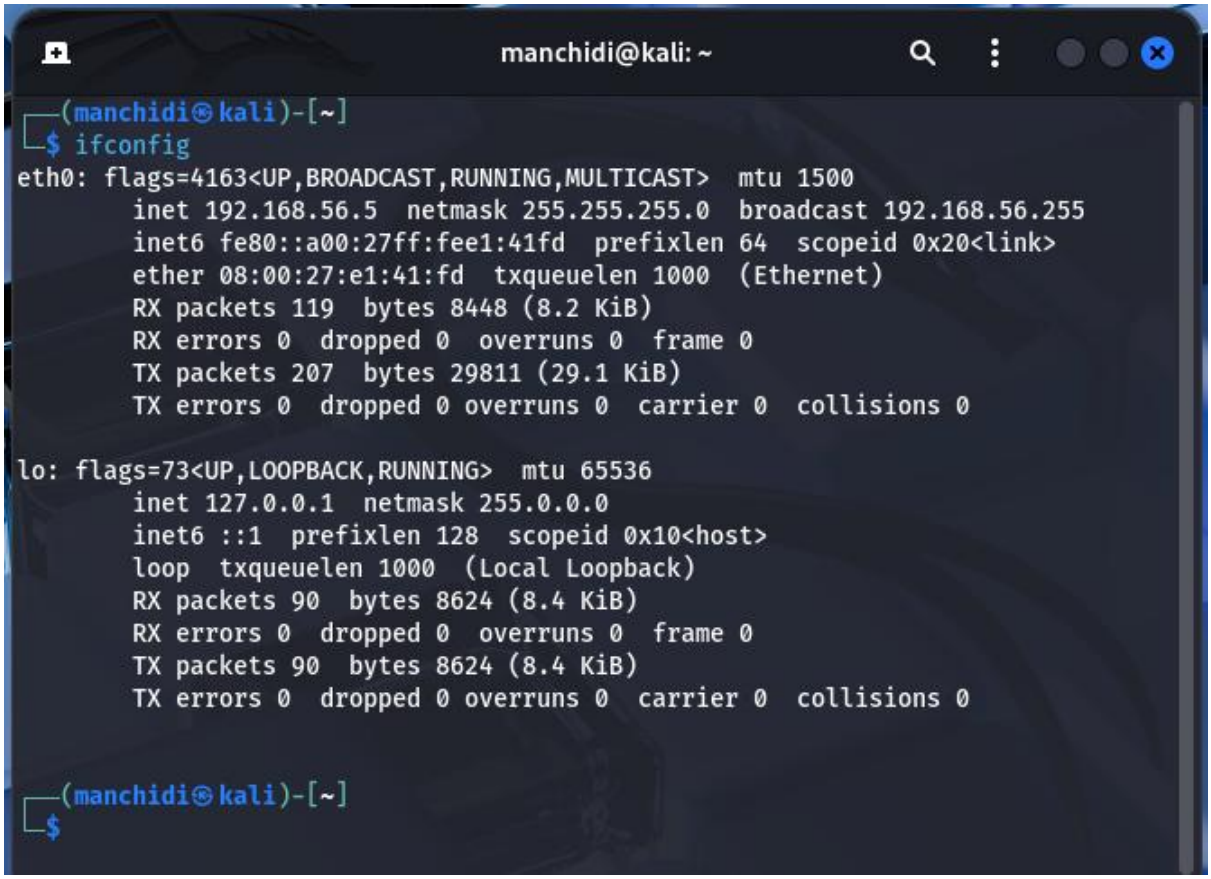
The sensor is connected to the main lab network using the static IP address 192.168.56.103/24 which is set up in Figure 1.1.

Interface enp0s8:



The IP addresses 10.0.3.100/24 and 10.0.3.15/24 show that this virtual machine has a second network adapter linked to a separate network (10.0.3.0/24). In this typical configuration one interface (enp0s3) is used for management and monitoring while the other interface (enp0s8) may be used for a different network segment or Network Address Translation (NAT) which enables the virtual machine (VM) to connect to the internet and download programs such as Snort.

Figure 1.3: IP Assignment on Kali Linux Attacker VM



The Kali Linux virtual machine is using the ifconfig command to check its network settings and this is a terminal output.

Eth0 interface:

The IP address shown in Figure 1.3 is confirmed using inet: 192.168.56.5.

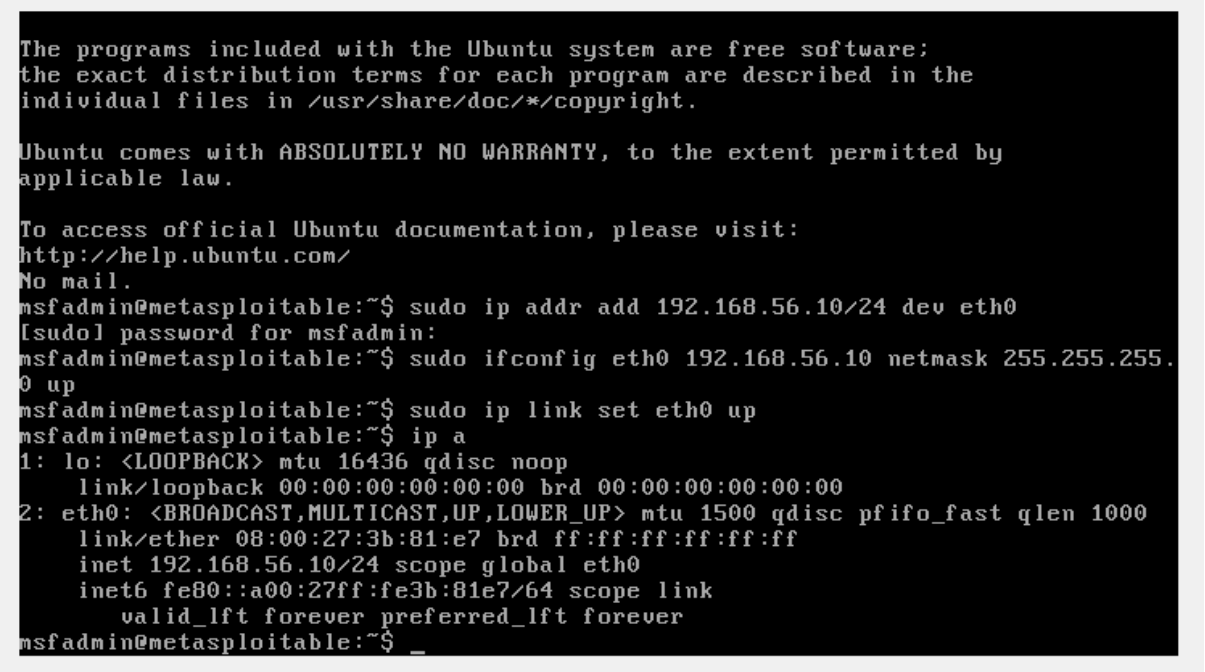
IP address: 255.255.255.0

Internet Protocol: 192.168.56.255

Interface lo: The loopback interface (127.0.0.1) which is the industry standard for internal machine communication.

Status: The interface is operational and prepared for network communication as indicated by the UP BROADCAST RUNNING and MULTICAST flags.

Figure 1.4: Static IP Configuration on Metasploitable Target VM



The Metasploitable virtual machine which will serve as the victim/target computer is where this terminal session is located. The commands needed to manually assign a static IP address are displayed.

Orders Fulfilled:

To add the IP address 192.168.56.10 to the eth0 interface run sudo ip addr add 192.168.56.10/24 dev eth0.

To set the IP and bring up the interface use the alternative command sudo ifconfig eth0 192.168.56.10 netmask 255.255.255.0 up.

To make sure the network interface is operational use sudo ip link set eth0 up.

Using IP A for verification:

The result verifies that the IP address 192.168.56.10/24 is now assigned to eth0.

The goal is to make the susceptible target computer available for the simulated attacks by placing it on the same 192.168.56.0/24 network as the attacker and the sensor.

Summary of the Lab Network

These numbers show that the virtual lab network was successfully set up using the static IP addresses shown below:

Role	VM Name	Operating System	IP Address	Figure Reference

IDS/IPS Sensor	Ubuntu	Ubuntu Desktop	`192.168.56.103`	Figure 1.1 & 1.2
Attacker	Kali Linux	Kali Linux	`192.168.56.5`	Figure 1.3 &
Victim/Target	Metasploitable	Linux (Metasploitable)	`192.168.56.10`	Figure 1.4

Section of the Network: 192.168.56.0/24 Mask for the subnet: 255.255.255.0

For your IDS/IPS project this setup is ideal. Installing the Snort software on the Ubuntu sensor (192.168.56.103) will allow it to track any communication between the victim (192.168.56.10) and the attacker (192.168.56.5).

Network Connectivity Verification for IDS/IPS Lab: Figure Analysis

A series of ping tests between the three virtual machines are shown in the following figures. In order to deploy and test the Snort IDS/IPS it is essential to confirm that all systems are on the same network segment and have the ability to communicate with one another.

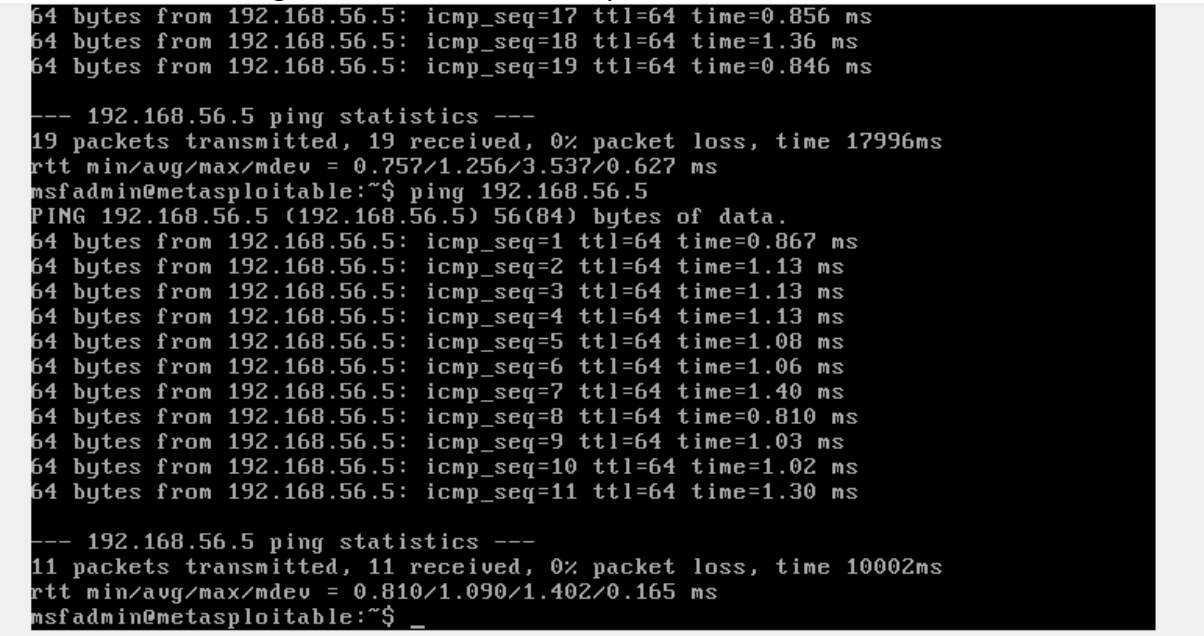


Figure 1.5: Metasploitable → Kali Linux (Two-way Communication)

The Metasploitable (Victim) VM's terminal is seen in this figure pinging the Kali Linux (Attacker) VM.

Ping 192.168.56.5 (the IP address of Kali Linux)

Outcome:

19 packets were sent 19 received and there was no packet loss.

The attacker effectively responds to the victim.

Significance: This demonstrates that communication between the attacker and target machines is possible. Both the attacker's exploits and the victim's reactions (such as a reverse shell) which the IDS must detect depend on this bidirectional reachability.

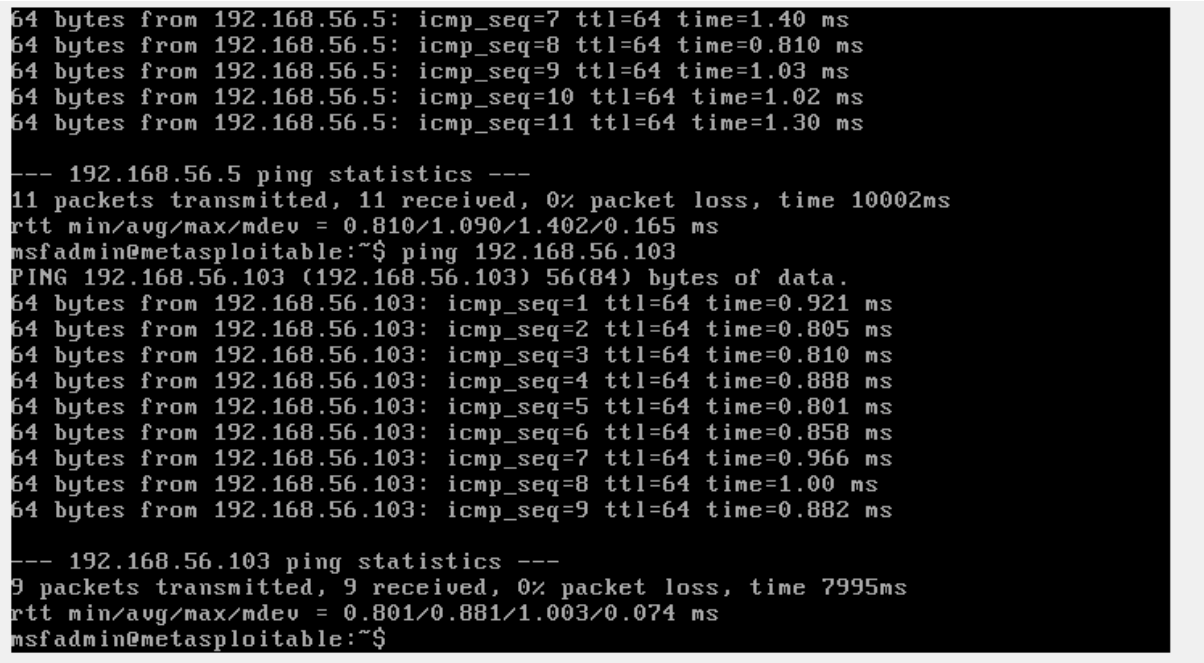


Figure 1.6: Metasploitable → Ubuntu Sensor

This is an extension of the Metasploitable terminal that tests the Ubuntu (Sensor) virtual machine's connectivity.

Ping 192.168.56.103 (the IP address of the Ubuntu Sensor)

Outcome:

9 packets were sent 9 were received and there was no packet loss.

Responses from the sensor are successfully sent to the victim.

Significance: This confirms that the network segment containing the Snort sensor can reach the target computer. To properly monitor it the sensor needs to be able to see the traffic going to and coming from the victim.

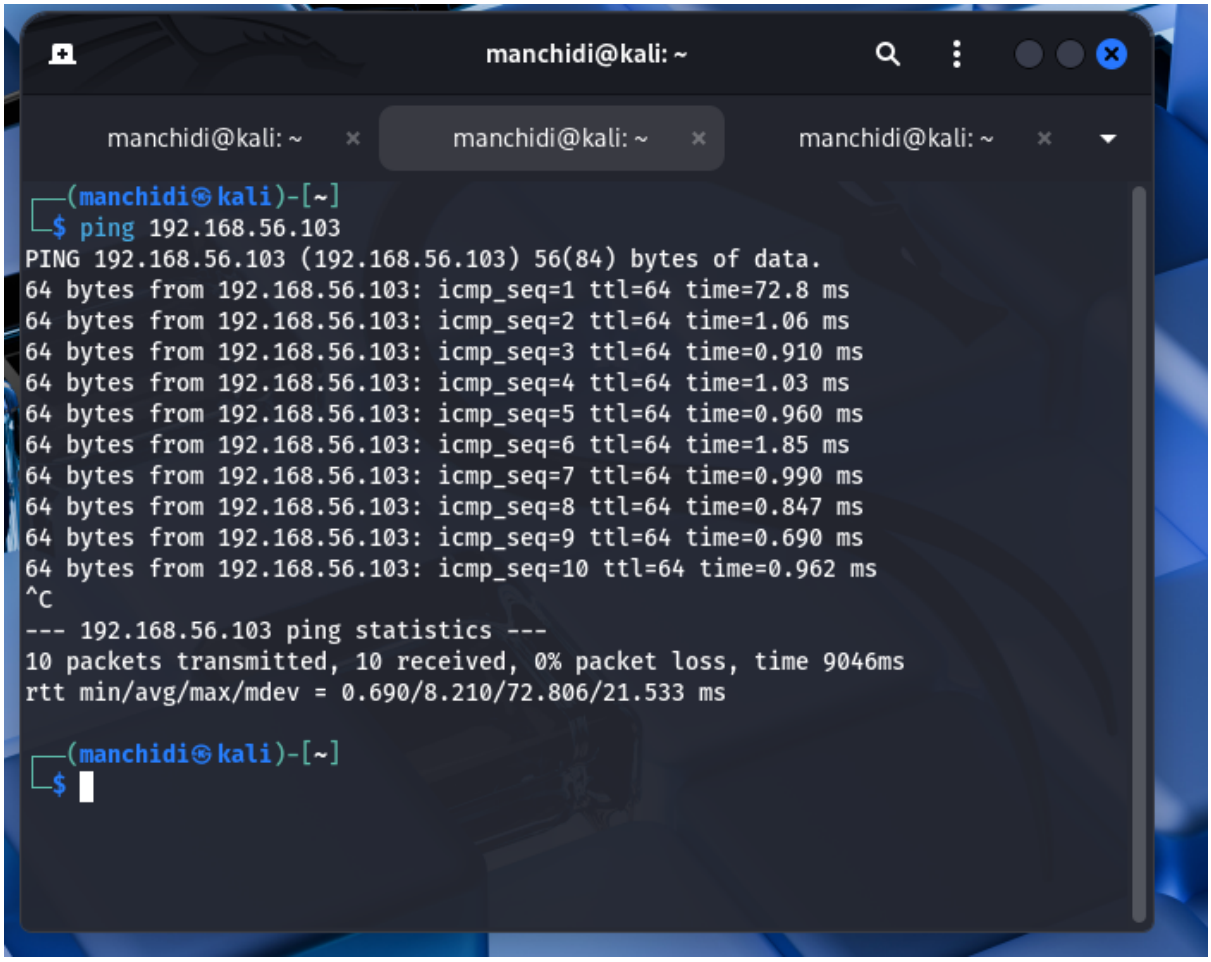


Figure 1.7: Kali Linux → Ubuntu Sensor

A terminal on the Kali Linux (Attacker) virtual machine is seen in this figure pinging the Ubuntu (Sensor) virtual machine.

Ping 192.168.56.103 is the command.

Outcome:

Ten packets sent ten received and no packet loss

As the network stack initializes the first packet has a larger delay (72.8 ms) while later packets have low steady latency (~1 ms).

Significance: Verifies the attacker's ability to see the sensor. This connectivity ensures they are on the same broadcast domain and is required for management (e.g. SSH access) even if the sensor will mostly be in passive monitoring mode.

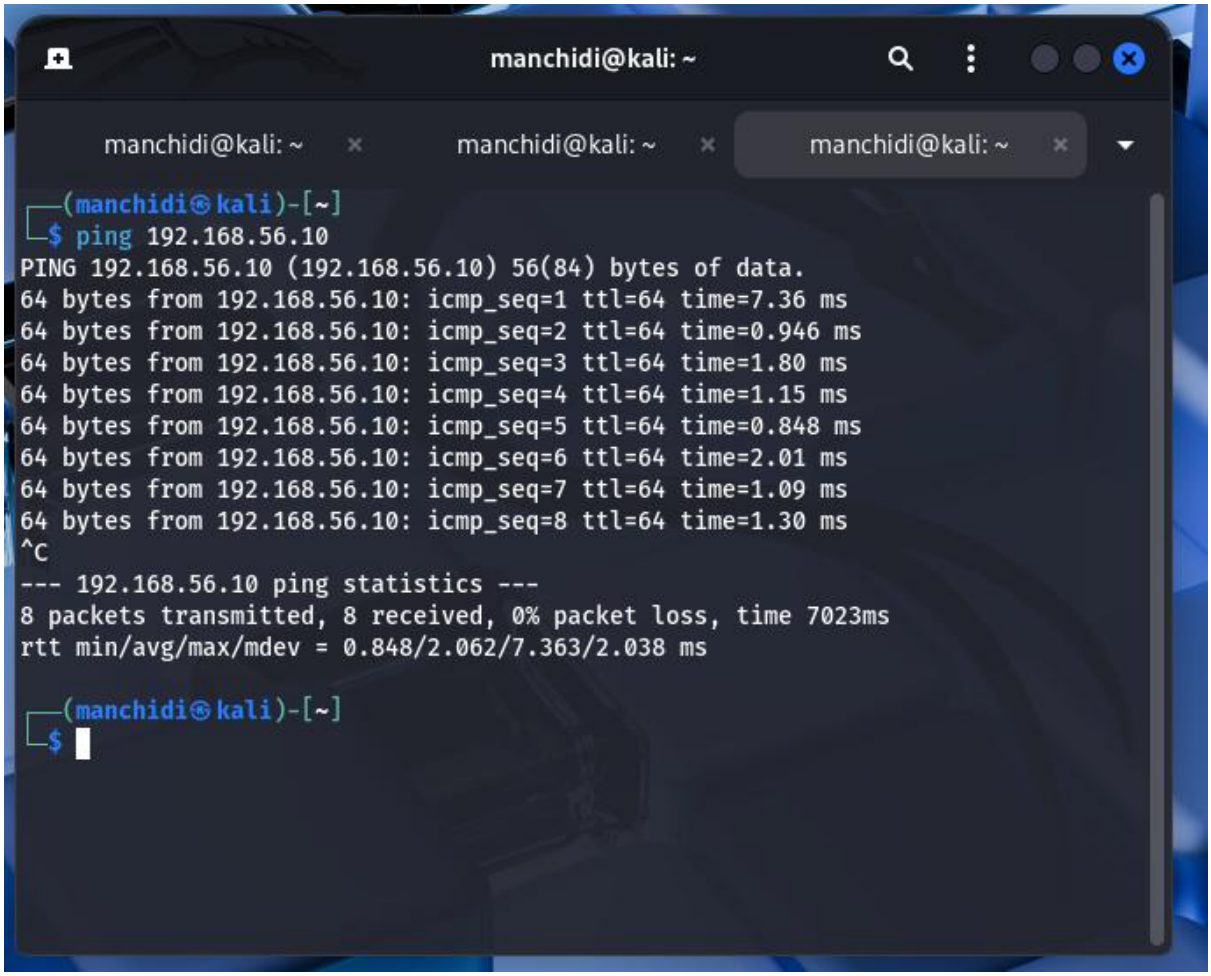


Figure 1.8: Kali Linux → Metasploitable

The Metasploitable (Victim) VM is pinging the Kali Linux (Attacker) VM in this figure.

Ping 192.168.56.10 is the command.

Outcome:

Eight packets were sent eight received and there was no packet loss.

Low latency and successful communication.

Significance: Your project's most important link is this one. It verifies if there is an active direct line of communication between the victim and the attacker. This path will be used by all attack traffic (scans exploits etc.) and the Snort sensor must keep an eye on it.

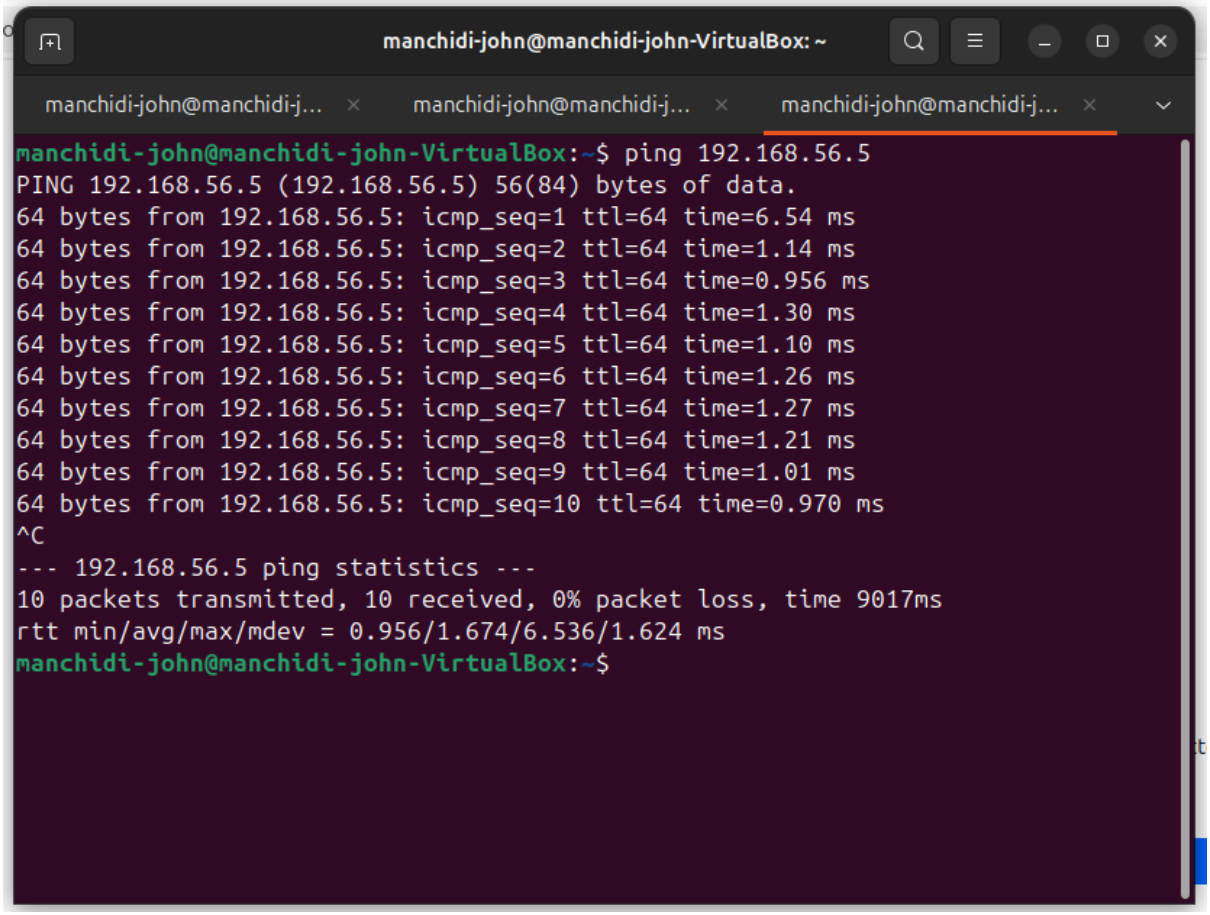


Figure 1.9: Ubuntu Sensor → Kali Linux

A terminal on the Ubuntu (Sensor) virtual machine is seen in this figure pinging the Kali Linux (Attacker) virtual machine.

Ping 192.168.56.5 is the command.

Outcome:

Ten packets sent ten received and no packet loss

low-latency reliable communication.

Significance: Confirms that the attacker is visible to the sensor. This is necessary for the sensor to record the first packets of each assault that the Kali machine launches.

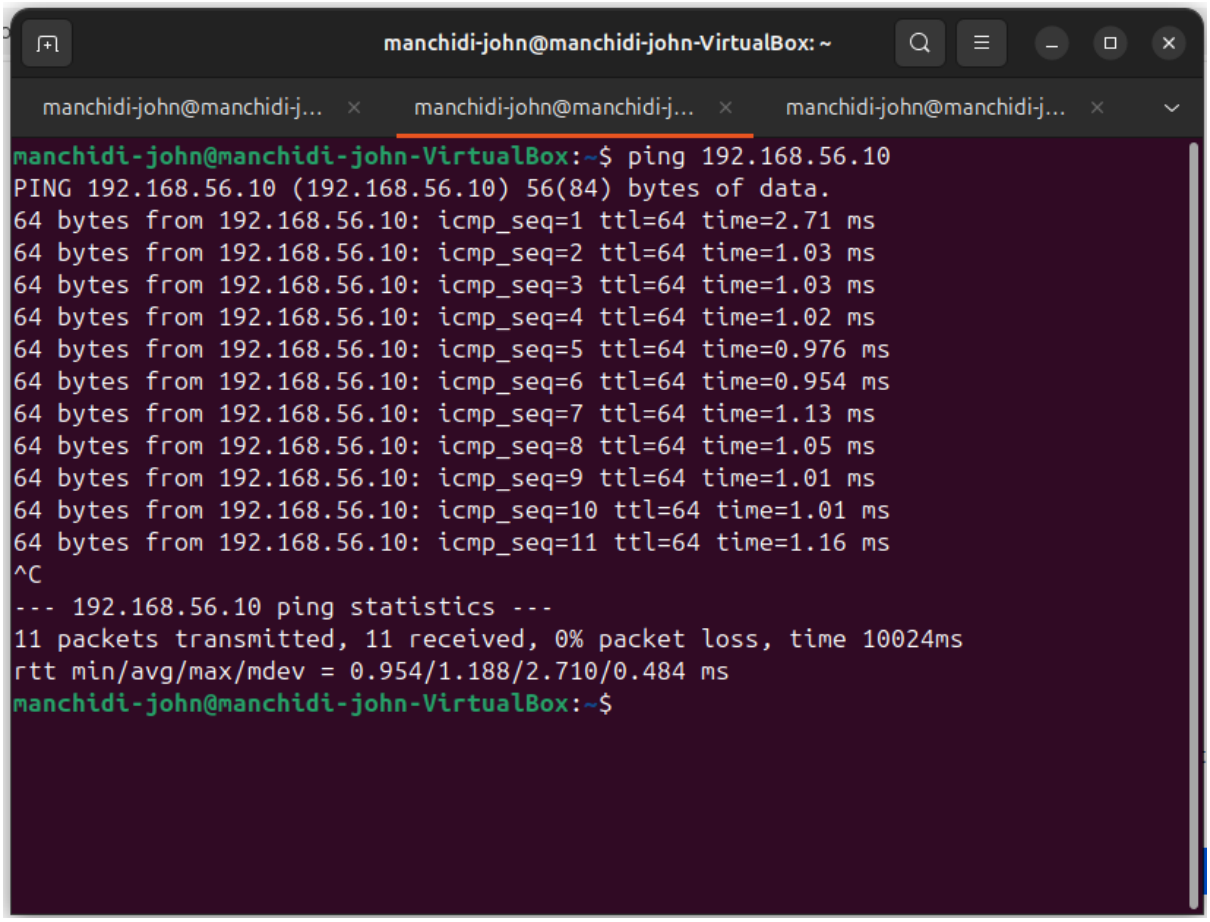


Figure 1.10: Ubuntu Sensor → Metasploitable

The Ubuntu (Sensor) VM is shown pinging the Metasploitable (Victim) VM in this figure.

Ping 192.168.56.10 is the command.

Outcome:

11 packets were sent 11 received and there was no packet loss.

dependable effective responses.

Significance: Verifies the triangle's last leg. In order to see all reaction traffic from the victim back to the attacker the sensor can speak with the victim directly.

2.Snort IDS/IPS Rule Configuration: Figure Analysis

The development and application of unique Snort rules intended to identify and stop particular attacks against the Metasploitable target computer are depicted in the following pictures.

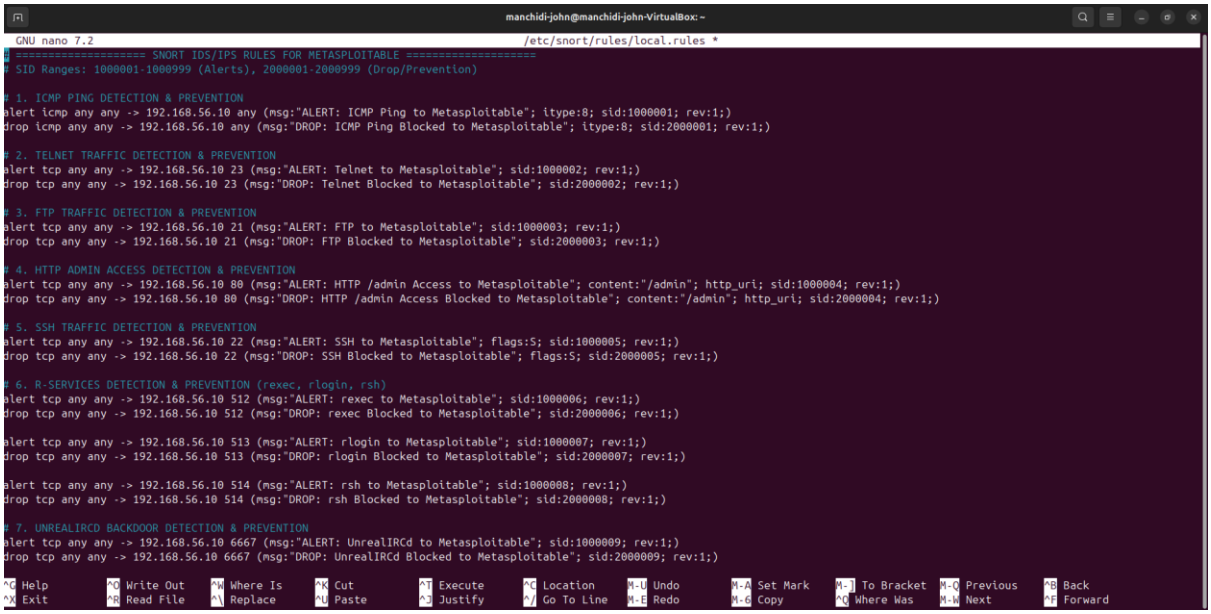


Figure 2.1: Snort Rule Planning Document

This figure which outlines the strategy prior to implementation seems to be a planning document or draft for the Snort rules. It establishes a distinct SID (Signature ID) range for the company.

Examination of the Planned Rules:

1.ICMP Ping Identification and Avoidance:

The rule:

```
# 1. ICMP PING DETECTION & PREVENTION
alert icmp any any -> 192.168.56.10 any (msg:"ALERT: ICMP Ping to Metasploitable"; itype:8; sid:1000001; rev:1;)
drop icmp any any -> 192.168.56.10 any (msg:"DROP: ICMP Ping Blocked to Metasploitable"; itype:8; sid:2000001; rev:1;)
```

The goal is to identify and prevent ICMP Echo Request packets (pings) from reaching the Metasploitable target. Ping is frequently used to find hosts during an attack's reconnaissance stage.

The IPS Counterpart is a drop rule that uses SID 2000001 to actively stop the target from answering pings giving scanners the impression that it is offline.

2.Telnet Traffic Identification and Avoidance:

The rule:

```
# 2. TELNET TRAFFIC DETECTION & PREVENTION
alert tcp any any -> 192.168.56.10 23 (msg:"ALERT: Telnet to Metasploitable"; sid:1000002; rev:1;)
drop tcp any any -> 192.168.56.10 23 (msg:"DROP: Telnet Blocked to Metasploitable"; sid:2000002; rev:1;)
```

Goal: Keeps track of and has the ability to stop any attempts to connect to the Telnet service (port 23). Telnet is an unsafe protocol that sends data including passwords in plaintext.

A drop rule to stop any Telnet sessions from starting is the IPS counterpart.

3.Identifying and Preventing FTP Traffic:

The rule:

```
# 3. FTP TRAFFIC DETECTION & PREVENTION
alert tcp any any -> 192.168.56.10 21 (msg:"ALERT: FTP to Metasploitable"; sid:1000003; rev:1;)
drop tcp any any -> 192.168.56.10 21 (msg:"DROP: FTP Blocked to Metasploitable"; sid:2000003; rev:1;)
```

Its function is to identify and prevent connections to the FTP service (port 21). This is a fundamental rule based on ports.

A drop rule to prevent FTP connections is the IPS counterpart.

4.Detection and Prevention of HTTP Admin Access:

The rule:

```
# 4. HTTP ADMIN ACCESS DETECTION & PREVENTION
alert tcp any any -> 192.168.56.10 80 (msg:"ALERT: HTTP /admin Access to Metasploitable"; content:"/admin"; http_uri; sid:1000004; rev:1;)
drop tcp any any -> 192.168.56.10 80 (msg:"DROP: HTTP /admin Access Blocked to Metasploitable"; content:"/admin"; http_uri; sid:2000004; rev:1;)
```

The goal of this more complex content-based rule is to... If "/admin" appears in any HTTP traffic it indicates an attempt to access a web server administrative panel and raises an alarm.

IPS Counterpart: To actively block such requests use a drop rule with SID 2000004.

5.SSH Traffic Identification and Avoidance:

The rule:

```
# 5. SSH TRAFFIC DETECTION & PREVENTION
alert tcp any any -> 192.168.56.10 22 (msg:"ALERT: SSH to Metasploitable"; flags:S; sid:1000005; rev:1;)
drop tcp any any -> 192.168.56.10 22 (msg:"DROP: SSH Blocked to Metasploitable"; flags:S; sid:2000005; rev:1;)
```

Goal: Identifies any fresh attempts to establish an SSH connection to the target on port 22 (as indicated by the TCP SYN flag S). Brute-force attacks can be detected using this.

IPS Counterpart: A drop rule that completely blocks SSH connections.

6.Detection and Prevention of R-Services (rexec):

The rule:

```
# 6. R-SERVICES DETECTION & PREVENTION (rexec, rlogin, rsh)
alert tcp any any -> 192.168.56.10 512 (msg:"ALERT: rexec to Metasploitable"; sid:1000006; rev:1;)
drop tcp any any -> 192.168.56.10 512 (msg:"DROP: rexec Blocked to Metasploitable"; sid:2000006; rev:1;)

alert tcp any any -> 192.168.56.10 513 (msg:"ALERT: rlogin to Metasploitable"; sid:1000007; rev:1;)
drop tcp any any -> 192.168.56.10 513 (msg:"DROP: rlogin Blocked to Metasploitable"; sid:2000007; rev:1;)

alert tcp any any -> 192.168.56.10 514 (msg:"ALERT: rsh to Metasploitable"; sid:1000008; rev:1;)
drop tcp any any -> 192.168.56.10 514 (msg:"DROP: rsh Blocked to Metasploitable"; sid:2000008; rev:1;)
```

The "remote execution" (rexec) service on port 512 is the target of this attack. These legacy services are insecure and frequently found on older systems such as Metasploitable.

IPS Counterpart: An access-blocking drop rule.

7.Detection and Prevention of UnrealIRCd Backdoors:

The rule:

```
# 7. UNREALIRCD BACKDOOR DETECTION & PREVENTION
alert tcp any any -> 192.168.56.10 6667 (msg:"ALERT: UnrealIRCd to Metasploitable"; sid:1000009; rev:1;)
drop tcp any any -> 192.168.56.10 6667 (msg:"DROP: UnrealIRCd Blocked to Metasploitable"; sid:2000009; rev:1;)
```

The goal of Metasploitable is to run a known backdoor in a vulnerable version of UnrealIRCd. Any attempt to connect to this service on port 6667 is detected by this rule.

IPS Counterpart: An anti-exploitation drop rule.

8. Identification and Avoidance of FTP Backdoor Exploits:

The rule:

```
# 8. FTP BACKDOOR EXPLOIT DETECTION & PREVENTION
alert tcp any any -> 192.168.56.10 21 (msg:"ALERT: FTP Backdoor Trigger Attempt"; content:"USER |3A|)"; sid:1000010; rev:1;)
drop tcp any any -> 192.168.56.10 21 (msg:"DROP: FTP Backdoor Blocked"; content:"USER |3A|)"; sid:2000010; rev:1;)
```

The goal of this crucial signature-based rule is to protect against a particular exploit. Sending a username that contains : opens a backdoor in the vsFTPd service on Metasploitable version 2.3.4. The colon : is represented by the hex symbol |3A|. This rule enables it to identify the exploit attempt itself rather than just a connection by explicitly searching for that malicious payload in the FTP USER command.

9. Identifying and Preventing SQL Injection:

The rule:

```
# 9. SQL INJECTION DETECTION & PREVENTION
alert tcp any any -> 192.168.56.10 80 (msg:"ALERT: SQL Injection Attempt"; content:"union"; nocase; content:"select"; nocase; sid:1000011; rev:1;)
drop tcp any any -> 192.168.56.10 80 (msg:"DROP: SQL Injection Blocked"; content:"union"; nocase; content:"select"; nocase; sid:2000011; rev:1;)
```

The goal of this content-matching rule is to search for typical SQL injection keywords ("union" "select") in web traffic that is sent to port 80 on the target. By making the search case-insensitive the nocase option enhances detection evasion.

10. Port Scan Detection & Prevention:

```
# 10. PORT SCAN DETECTION & PREVENTION
alert tcp any any -> 192.168.56.10 any (msg:"ALERT: TCP NULL Scan"; flags:0; sid:1000012; rev:1;)
drop tcp any any -> 192.168.56.10 any (msg:"DROP: TCP NULL Scan Blocked"; flags:0; sid:2000012; rev:1;)

alert tcp any any -> 192.168.56.10 any (msg:"ALERT: TCP FIN Scan"; flags:F; sid:1000013; rev:1;)
drop tcp any any -> 192.168.56.10 any (msg:"DROP: TCP FIN Scan Blocked"; flags:F; sid:2000013; rev:1;)
```

Guidelines: contains particular detection for methods of stealth scanning:

When no TCP flags are configured the TCP NULL Scan is flags:0;.

Only the FIN flag is set in the TCP FIN Scan: flags:F;.

Goal: As a crucial component of the reconnaissance phase these rules detect unusual packet flag combinations that are utilized by programs such as Nmap to map open ports without establishing a complete connection.

3.Snort Installation and Environment Setup: Figure Analysis

The Ubuntu Sensor VM's network environment was verified and Snort was successfully installed as seen in the following figures.

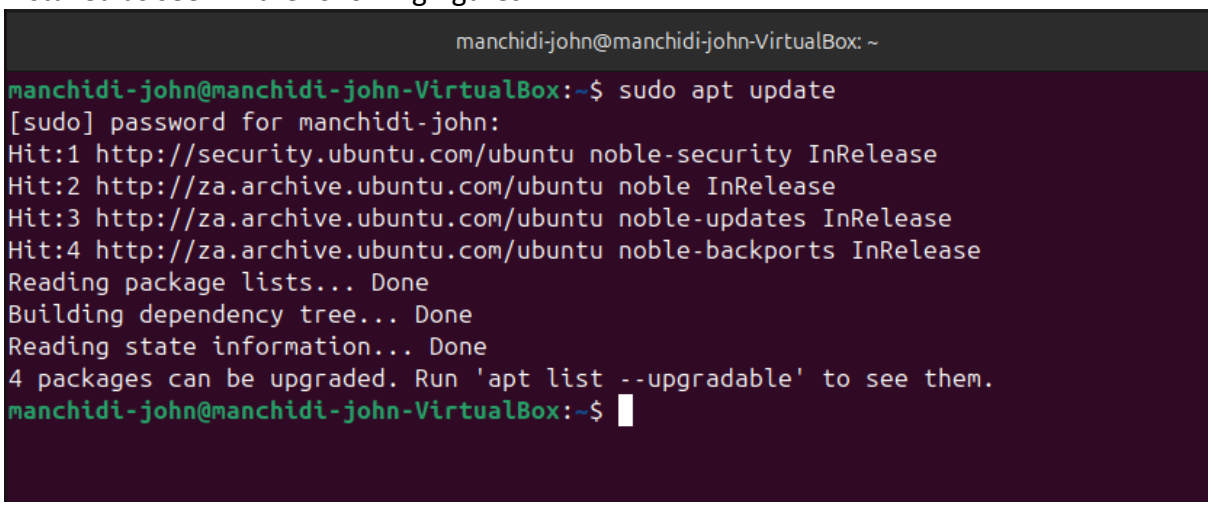


Figure 3.1: System Package Update

After updating the Ubuntu package management on the Ubuntu Sensor virtual machine the terminal output is displayed in this picture.

Command: sudo apt update.

Outcome:

Hit: 1... Hit:4: All configured Ubuntu software repositories (noble-security noble noble-updates and noble-backports) were successfully accessed.

Looking through package lists... Completed: The local package index was successfully updated.

Four packages are upgradeable: shows that there are only a few small updates available and that the system is generally up to date.

Significance Before installing any new software this is a standard preparation activity. As a best practice for security and stability it guarantees the system has the most up-to-date information about packages and versions that are available.

```
manchidi-john@manchidi-john-VirtualBox:~$ ip a | grep -E "^([0-9]+):|inet "
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen
1000
    inet 127.0.0.1/8 scope host lo
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group def
ault qlen 1000
    inet 192.168.56.103/24 brd 192.168.56.255 scope global noprefixroute enp0s3
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group def
ault qlen 1000
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute enp0s8
```

Figure 3.2: Network Interface Identification

The terminal output of the command used to find the Ubuntu Sensor VM's available network interfaces is seen in this figure.

ip a | grep -E "^([0-9]+):|inet" is the command.

As a result the command provides a list of every network interface together with its IP address. These are the main interfaces:

lo: The internal communication loopback interface (127.0.0.1).

The main interface for monitoring is enp0s3. It is on the same lab network as the victim (192.168.56.10) and the attacker (192.168.56.5) as evidenced by its static IP address 192.168.56.103/24. Snort will keep an eye on this interface.

enp0s8: An additional IP 10.0.3.15/24 interface. This is probably a NAT adapter that VirtualBox uses to give the virtual machine (VM) internet access so that it may download programs like Snort.

Significance: In order to configure Snort this step is essential. It validates that Snort needs to use the correct interface (enp0s3) in order to view the traffic between the Metasploitable and Kali virtual machines. If the incorrect interface is used Snort won't observe any traffic and the project as a whole will fail.

```
manchidi-john@manchidi-john-VirtualBox:~$ snort --version

   ,,_     -*> Snort! <*-
  o"  )~   Version 2.9.20 GRE (Build 82)
   ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using libpcap version 1.10.4 (with TPACKET_V3)
           Using PCRE version: 8.39 2016-06-14
           Using ZLIB version: 1.3
```

Figure 3.3: Snort Version Verification

The terminal output verifying the successful installation of Snort and displaying its version information is shown in this figure.

Enter snort --version.

Outcome:

Version 2.9.20 GRE (Build 82) of Snort! *-: Verifies that Snort is installed and running. The version is stable and is 2.9.20.

The version of the underlying library that Snort uses to collect network packets is displayed when using libpcap version 1.10.4. Its primary function depends on this.

Using PCRE version 8.39: Verifies the availability of the Perl-Compatible Regular Expressions library which is necessary for intricate pattern matching in rules.

while ZLIB version 1.3 is used it indicates that a compression library is available for Snort to use while processing compressed data.

Significance: This is unquestionable evidence that Snort has been installed successfully and that all of its necessary prerequisites have been satisfied. For reproducibility troubleshooting and documentation the comprehensive version information is crucial.

4. Attacking Metasploitable using kali and Ubuntu for IDS and IPS

Rule 1: ICMP Ping Detection & Prevention

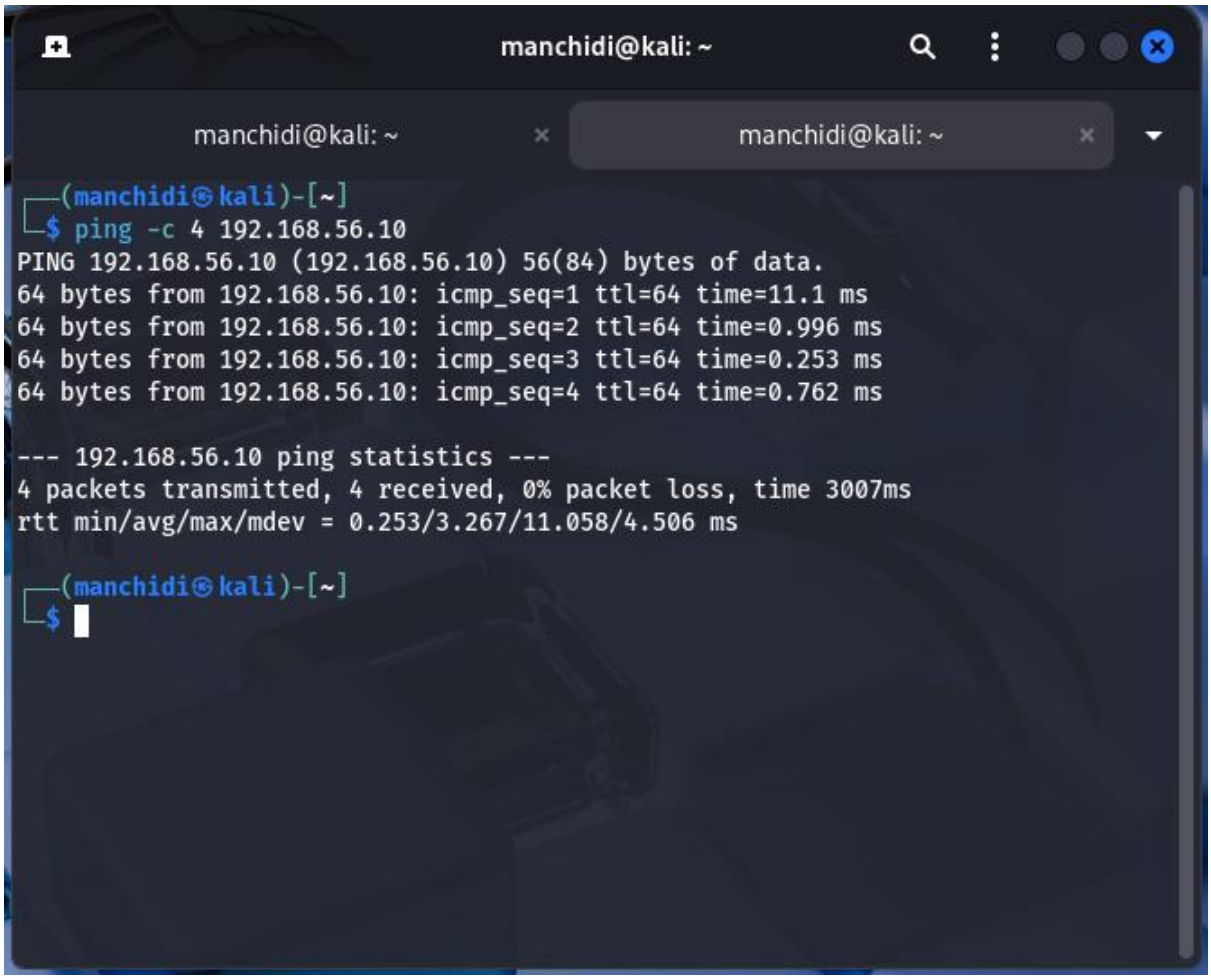


Figure 4.1: ICMP Ping Test from Kali Linux Attacker

Analysis: This confirms regular network connectivity before to IDS/IPS activation by demonstrating successful ICMP Echo Request and Reply communication between the Kali attacker (192.168.56.5) and Metasploitable target (192.168.56.10) with 0% packet loss.

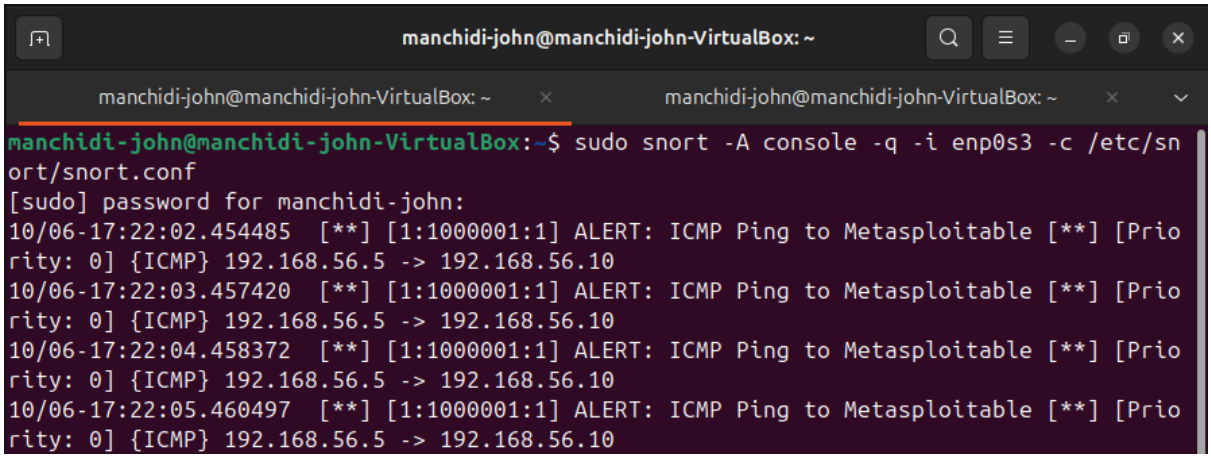


Figure 4.2: IDS Mode - ICMP Ping Detection Alerts

Technical Evaluation:

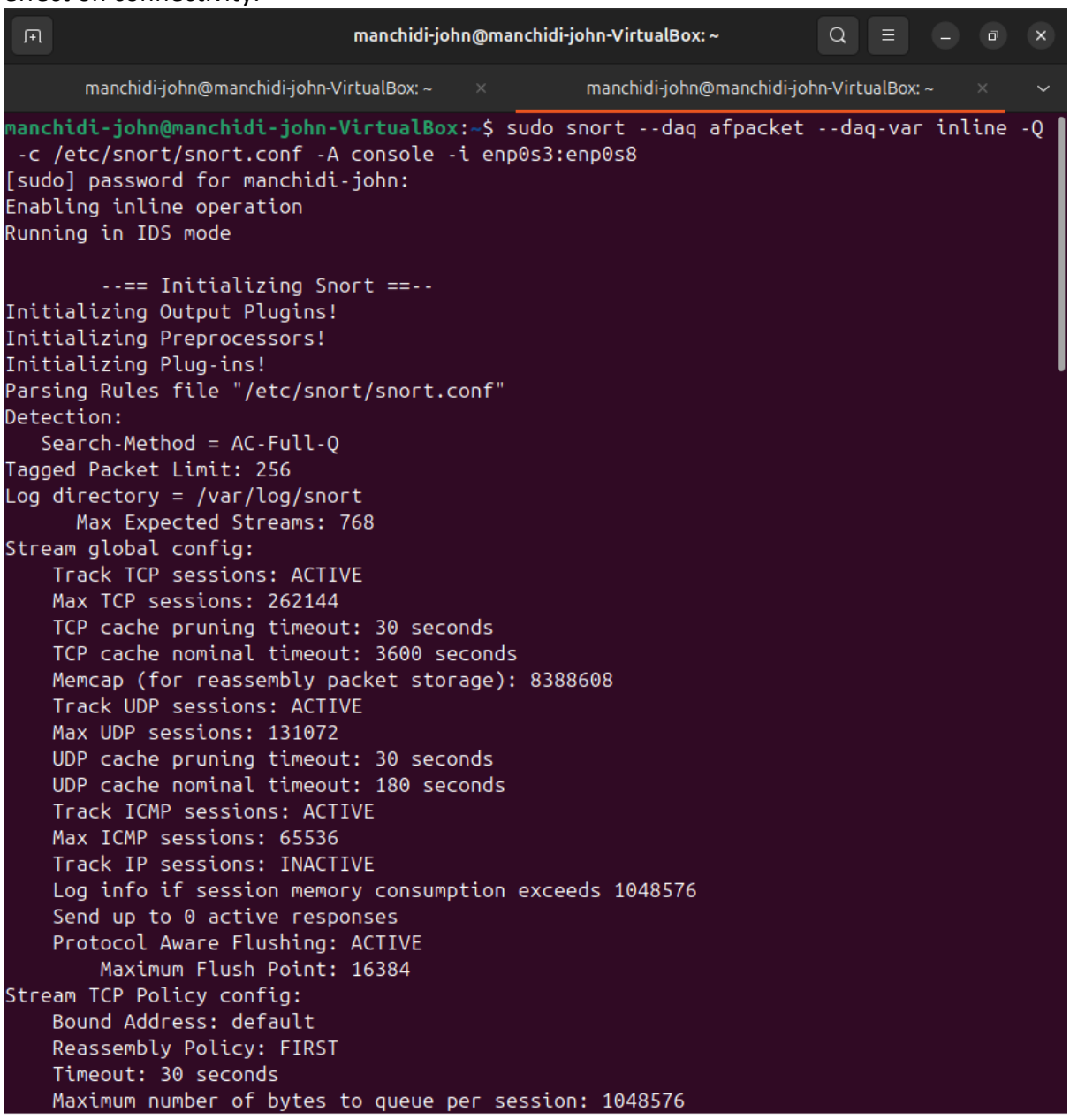
Rule Triggered: ICMP Ping Alert Rule sid:1000001

Method of Detection: ICMP Type 8 (Echo Request) packet detection using signatures

Traffic Pattern: We were able to identify all four ping packets sent from Kali (192.168.56.5) to Metasploitable (192.168.56.10).

IDS efficacy: 100% detection rate; each of the four ping attempts resulted in an alert.

Network Impact: While being watched packets continued to reach their destination with no effect on connectivity.



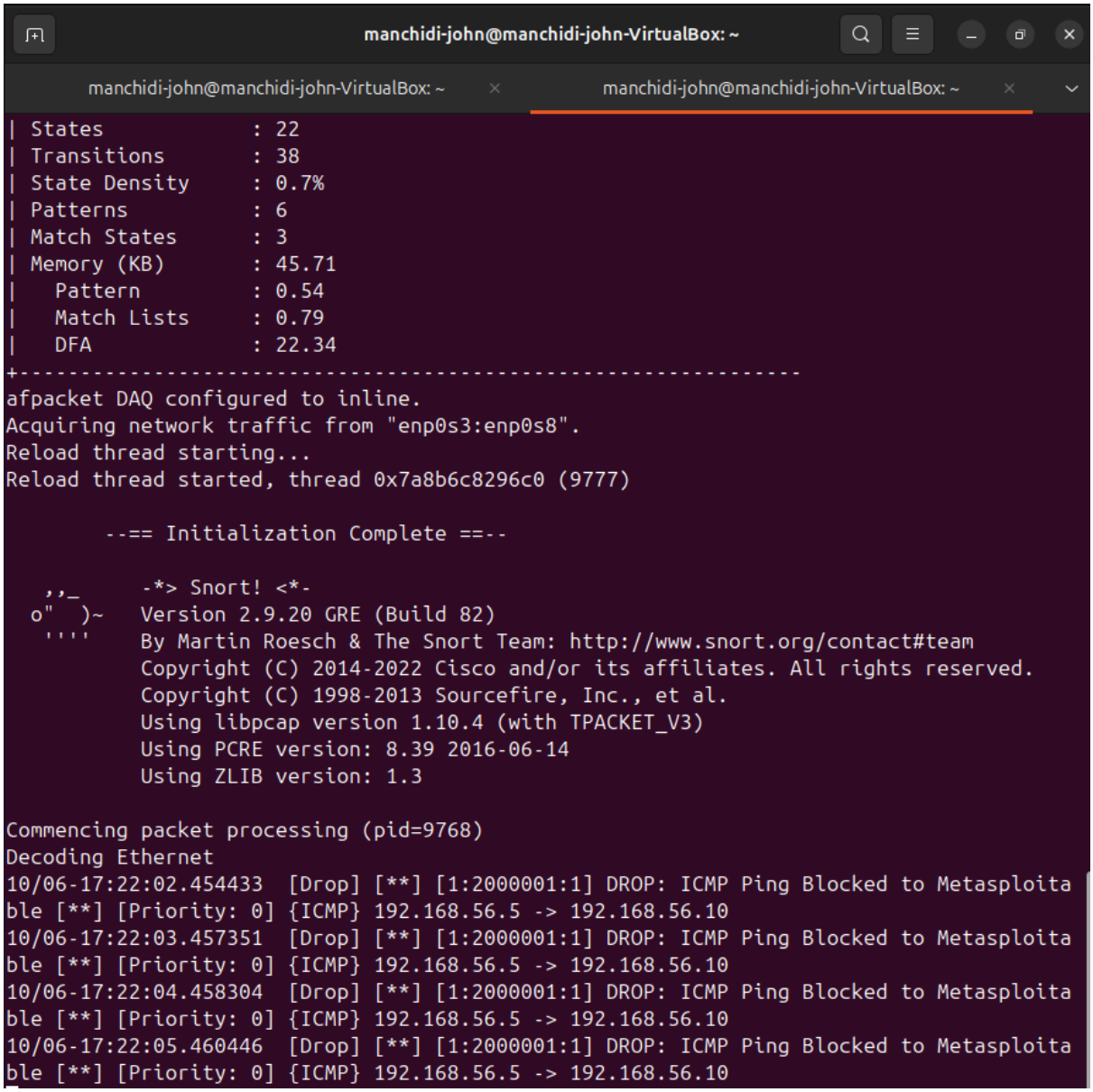


Figure 4.3: IPS Mode - ICMP Ping Prevention & Blocking

Technical Evaluation:

Rule Triggered: ICMP Ping Drop Rule sid:2000001

Active packet dropping in inline mode is the prevention technique.

All four ping packets were intercepted and dropped at the sensor demonstrating the 100% prevention rate of the IPS.

Important Proof: The [Drop] tag in the log verifies active prevention as opposed to passive detection.

Network Impact: Total blocking: Ping requests from Kali would display 100% packet loss when the IPS was in effect.

RULE 2: Telnet Traffic Detection & Prevention

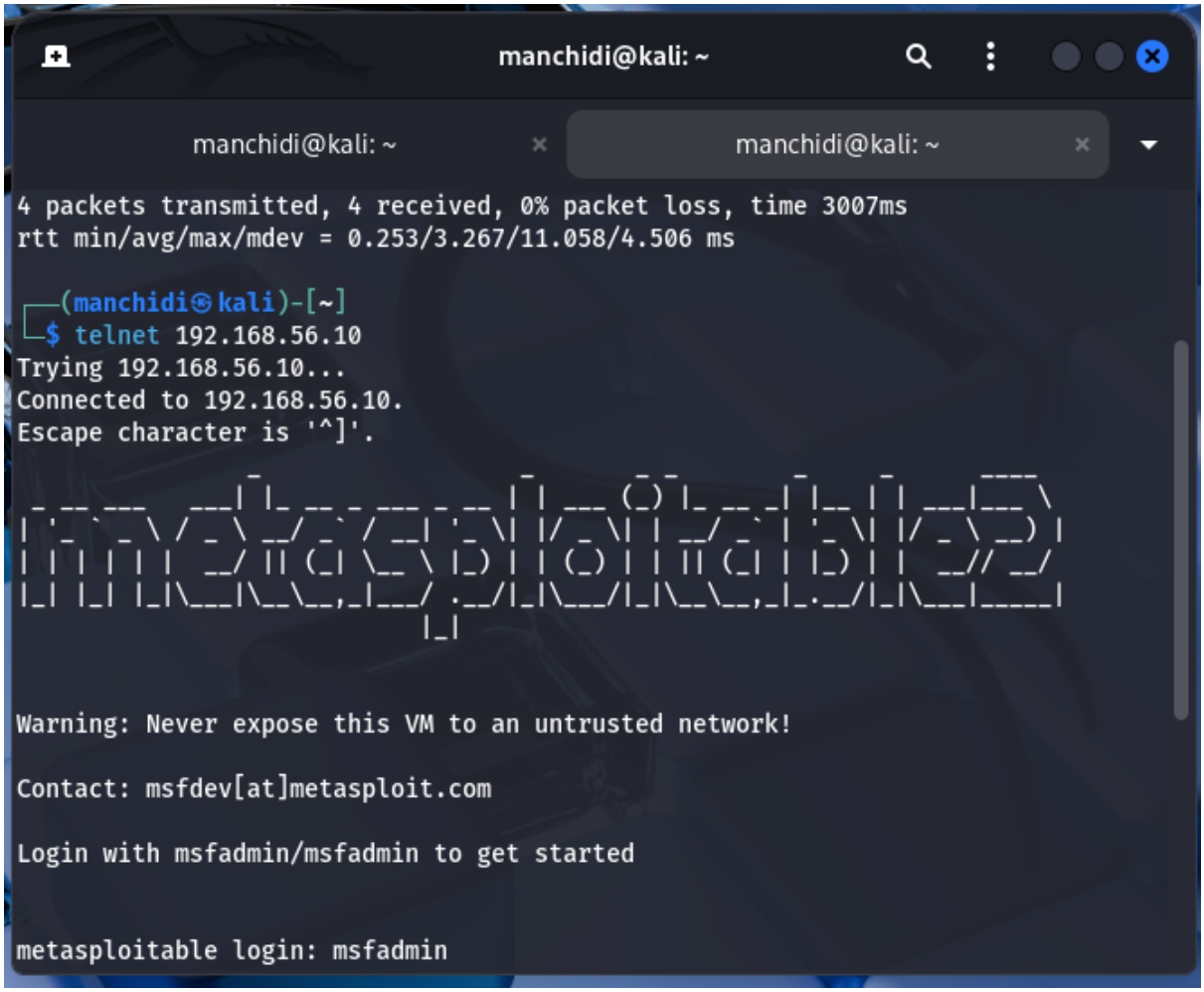


Figure 4.4: Successful Telnet Connection Attempt from Kali Linux

Evaluation:

This shows that Kali (192.168.56.5) and Metasploitable (192.168.56.10) have successfully established a Telnet connection on port 23. After the connection was made the login banner for the Telnet service appeared. This demonstrates that the Telnet service is accessible and open to attack while the IDS/IPS is not in use. The successful completion of the authentication prompt by the attacker indicates that the service is operational.

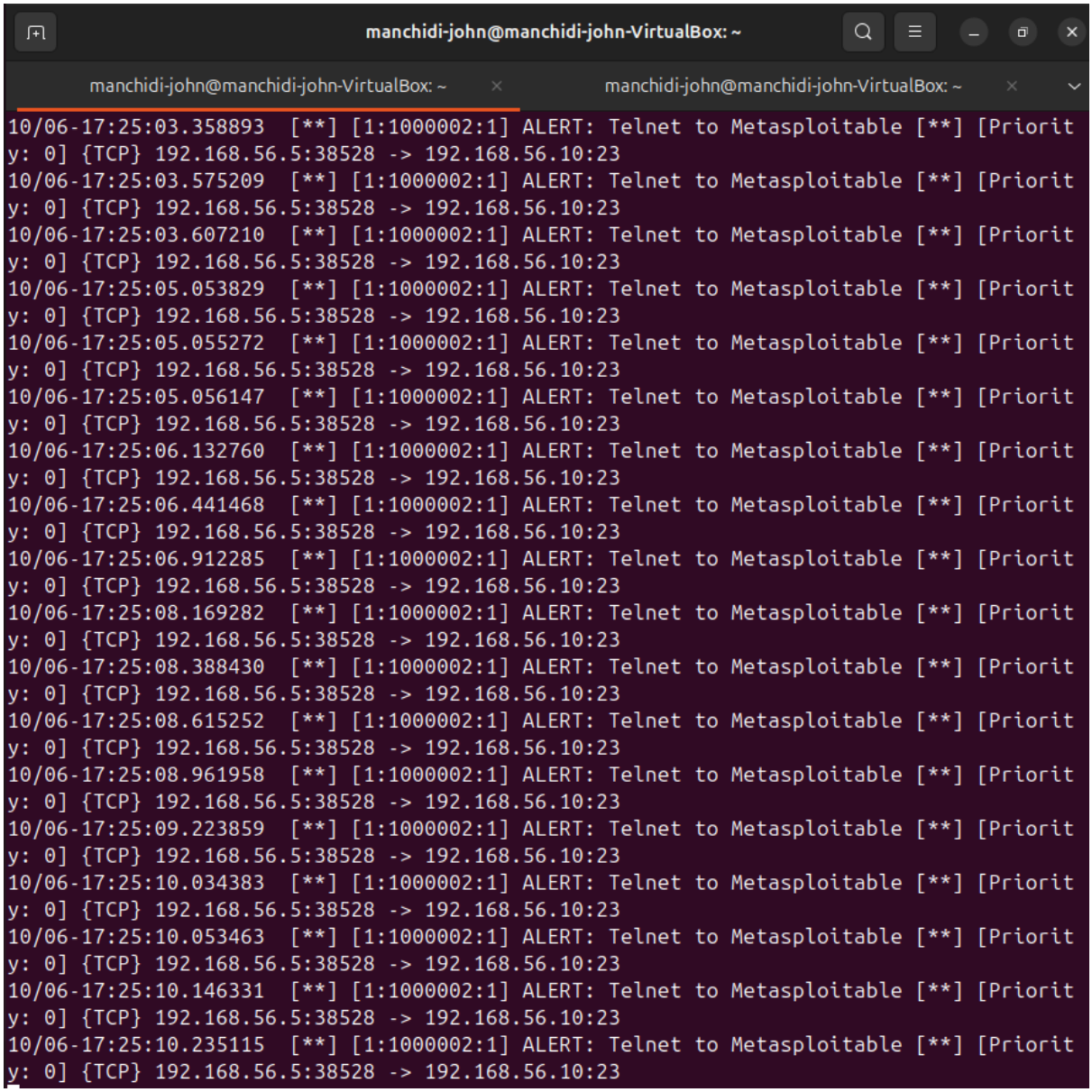


Figure 4.5: IDS Mode - Telnet Ping Detection Alerts

The Telnet activity in this figure represents network traffic where the Kali machine (192.168.56.5) is attempting to connect to the Telnet service on the Metasploitable machine (192.168.56.10:23) likely as part of a security testing exercise. The alerts indicate that this activity is being monitored and flagged which is consistent with the IDS/IPS setup described in your project overview.

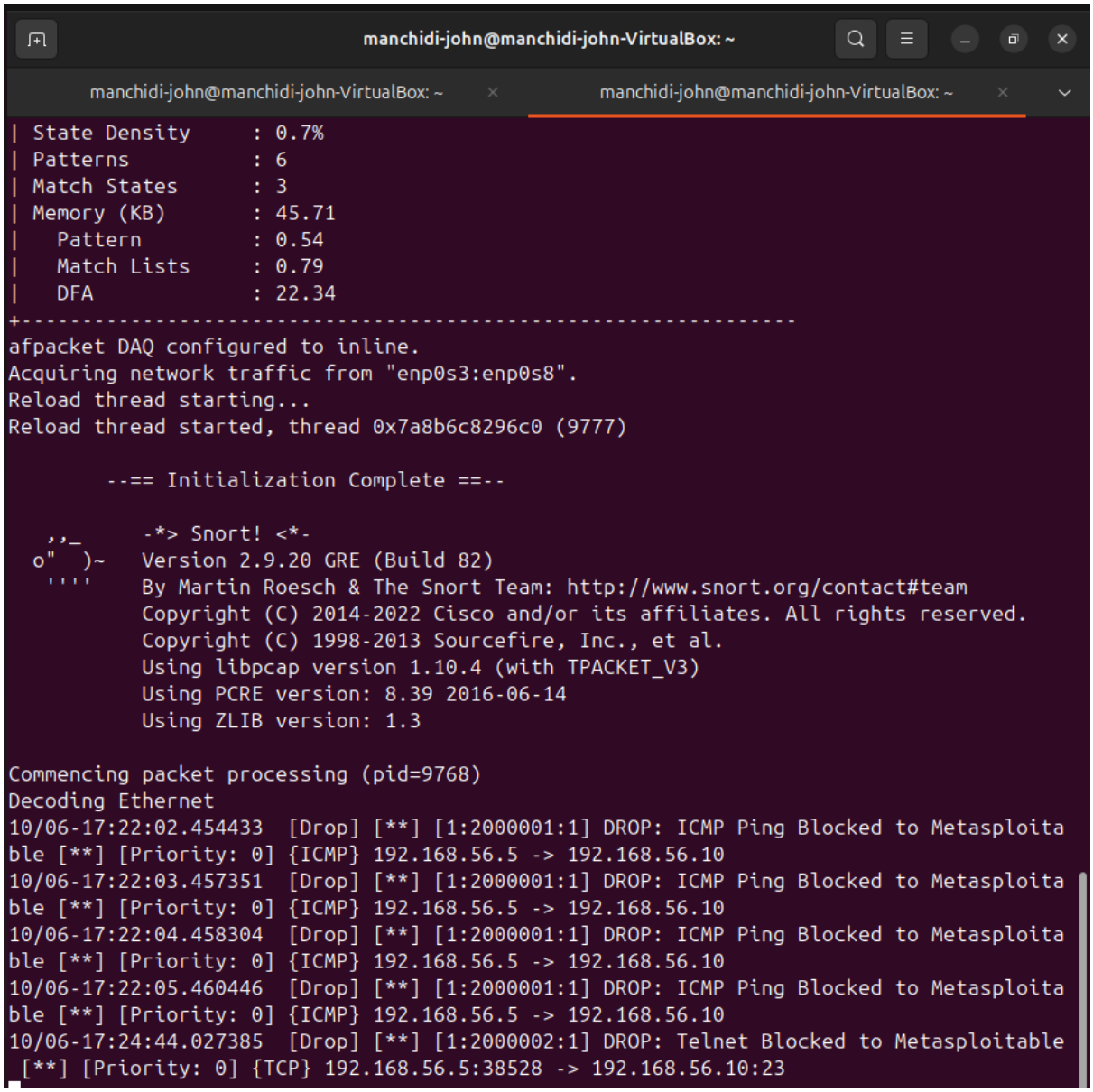


Figure 4.6: IPS Mode - Telnet Connection Blocking

Technical Evaluation:

Triggered Rule: sid:2000002 The Telnet Drop Rule

TCP SYN packets are actively dropped to port 23 as a preventative measure.

Traffic Information:

Source: Kali Linux with ephemeral port 192.168.56.5:38528

The Metasploitable Telnet service at 192.168.56.10:23 is the destination.

Protocol: TCP

IPS Effectiveness: At the TCP handshake level the Telnet connection attempt was instantly blocked.

Important Proof: The [Drop] tag verifies that the Telnet session establishment is actively prevented.

RULE 3 : FTP Traffic Detection & Prevention

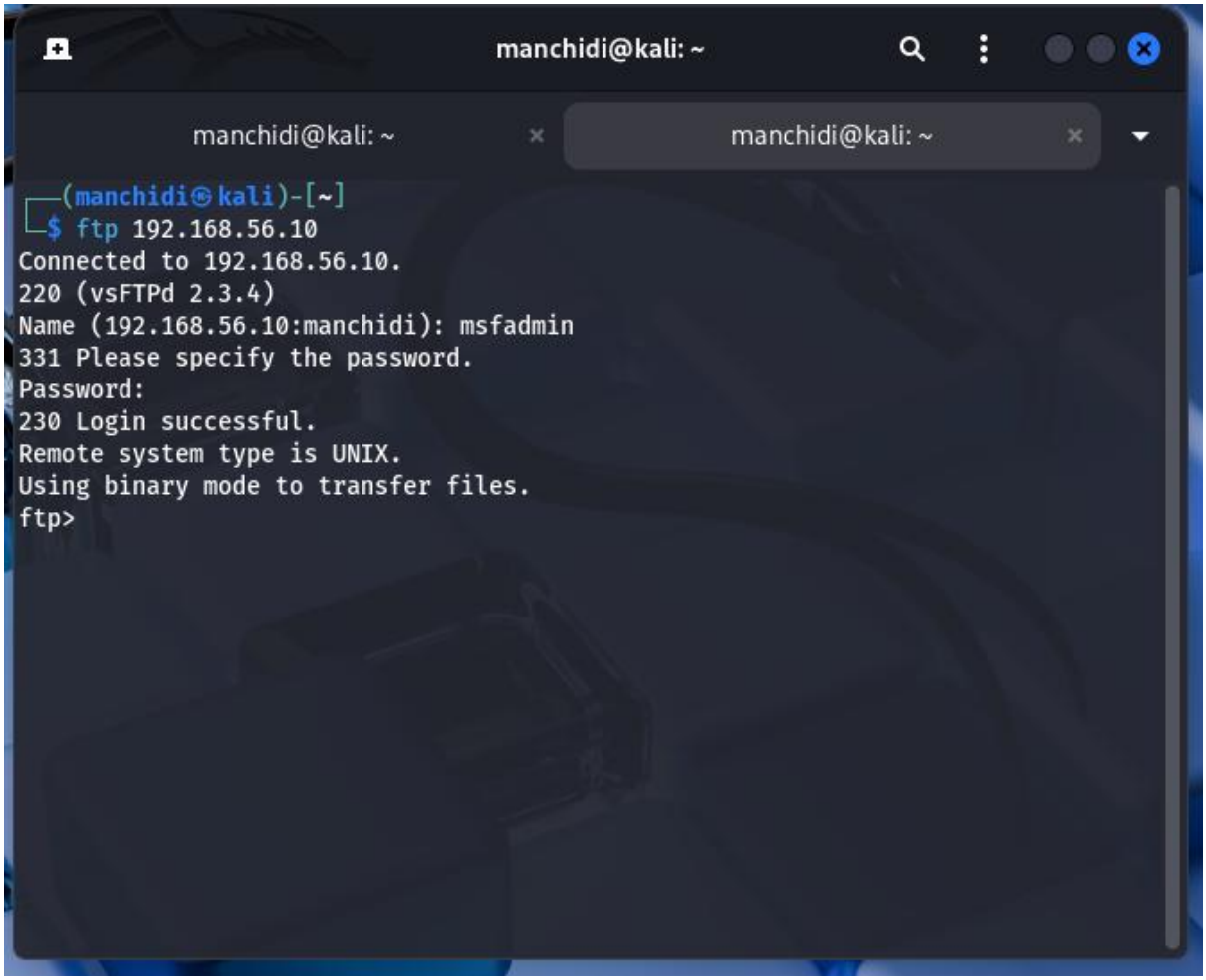


Figure 4.7: FTP Connection from Kali to Metasploitable

This suggests a penetration testing scenario in which the Metasploitable machine—known for its weak FTP service—is accessed using Kali. A common target in security exercises is the msfadmin user which is the default account on Metasploitable.

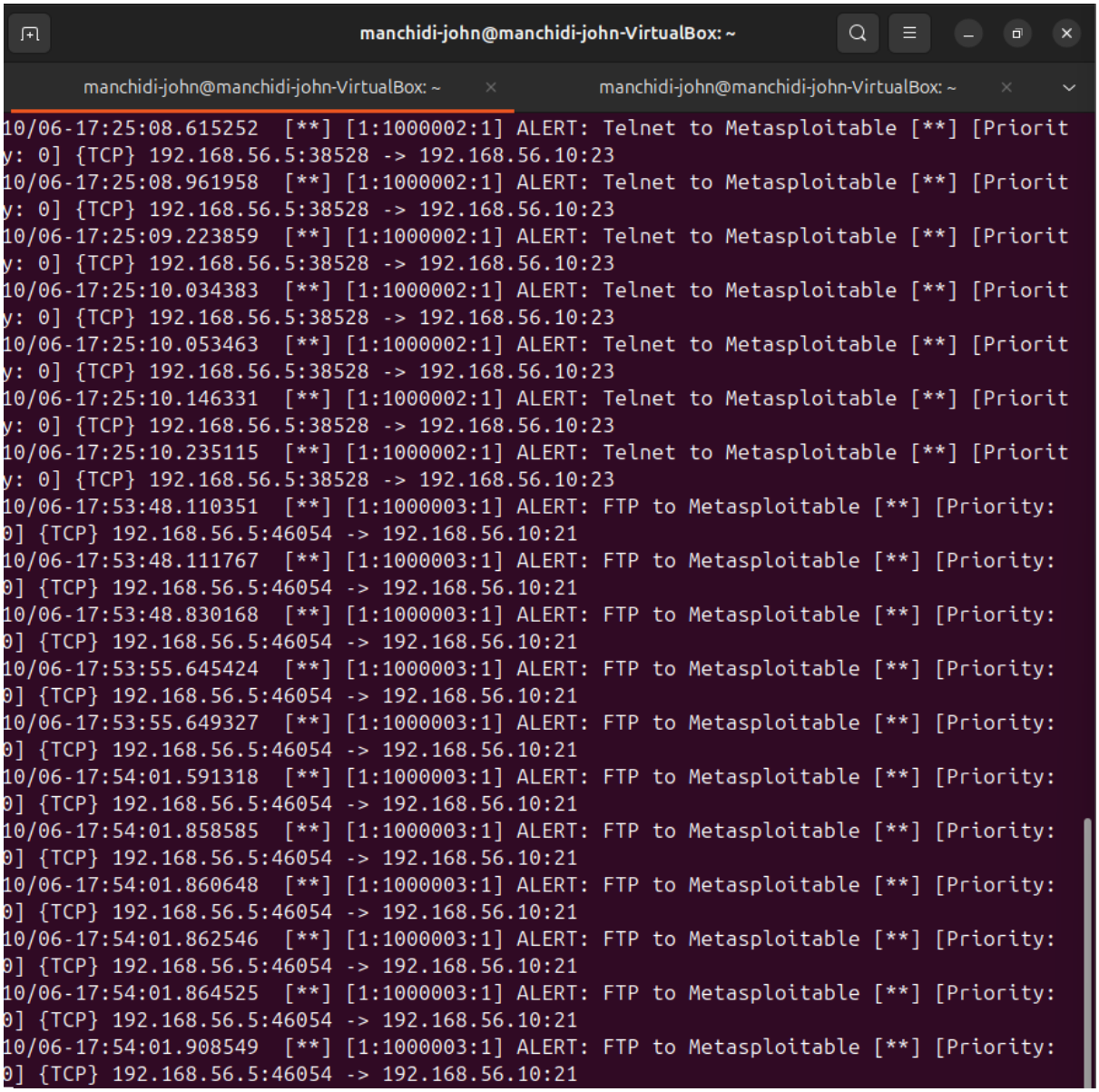


Figure 4.8: IDS Mode - FTP Ping Detection Alerts

A sample of the output from the open-source Network Intrusion Detection/Prevention System (NIDS/NIPS) Snort is displayed in this graphic.

FTP Alerts: Several ALERT messages for "FTP to Metasploitable" traffic are shown in the log. This shows that a rule in Snort is set up to identify FTP connection attempts made to a system that has been classified as "Metasploitable"—a purposefully weak machine that is frequently used in security testing.

Traffic Flow: According to the warnings traffic is approaching the destination IP 192.168.56.21 (port 21 the standard FTP control port) from the source IP address 192.168.56.5 (port 46054).

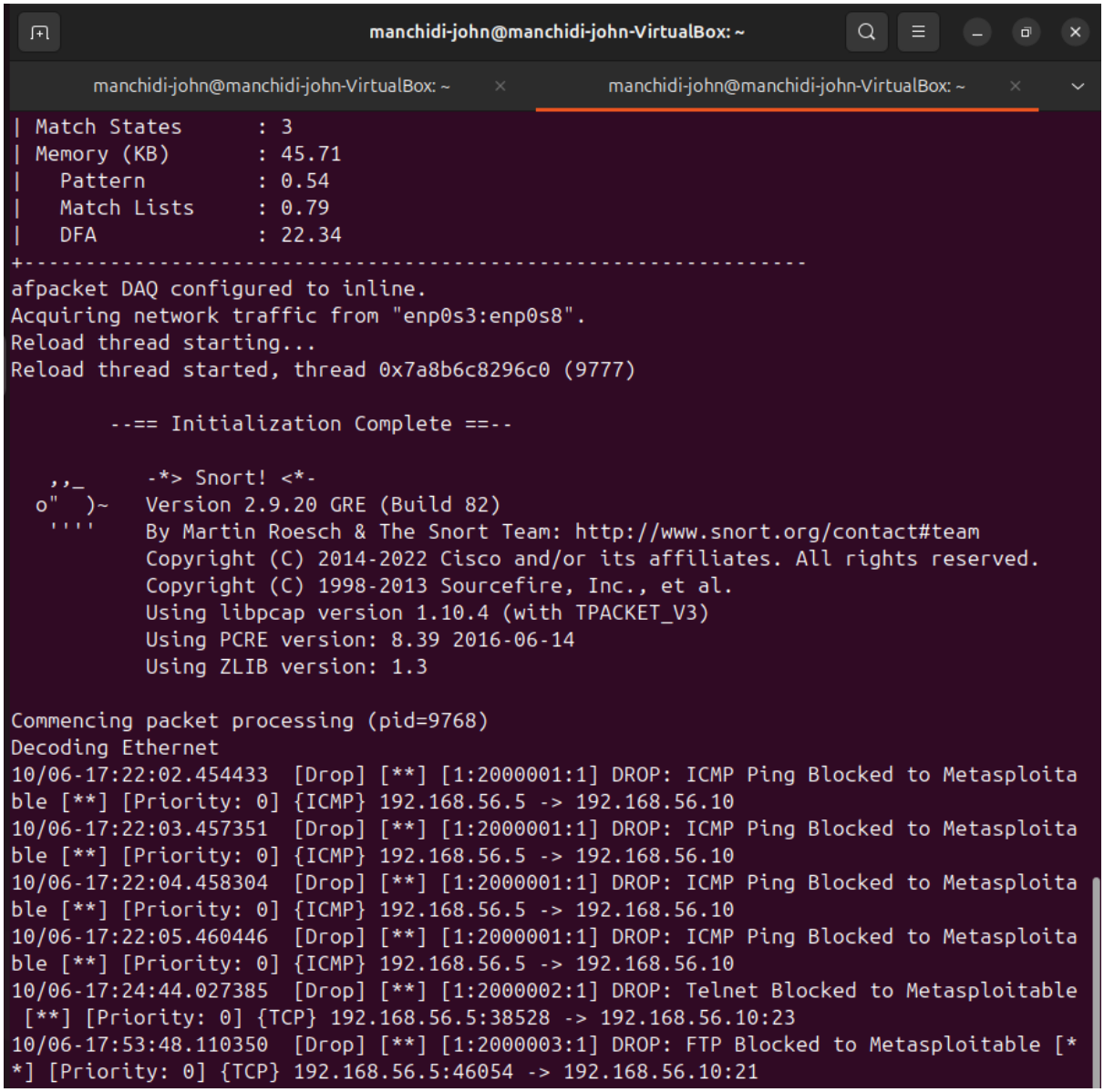


Figure 4.9: IPS Mode - FTP Connection Blocking

The Snort instance's initialization data are shown in this figure along with additional logged activities such as packet dropping.

Snort Initialization: The terminal output displays the initialization and network traffic acquisition of Snort Version 2.9.20 GRE.

Packet Dropping (IPS Mode): Snort is set up to run in an Intrusion Prevention System (IPS) mode actively restricting traffic as seen by the log entries.

A line such as [**] [1:2000003:1] DROP: FTP Blocked to Metasploitable indicates that the IPS blocked (dropped) a packet that was intended for the FTP port (21) on 192.168.56.10 most likely an attempt from the source IP 192.168.56.5.

Alerts/Drops: The entries include the action (DROP) the rule description (FTP Blocked to Metasploitable) and the rule ID (e.g. 1:2000003:1).

Rule 4: HTTP Admin Access Detection & Prevention

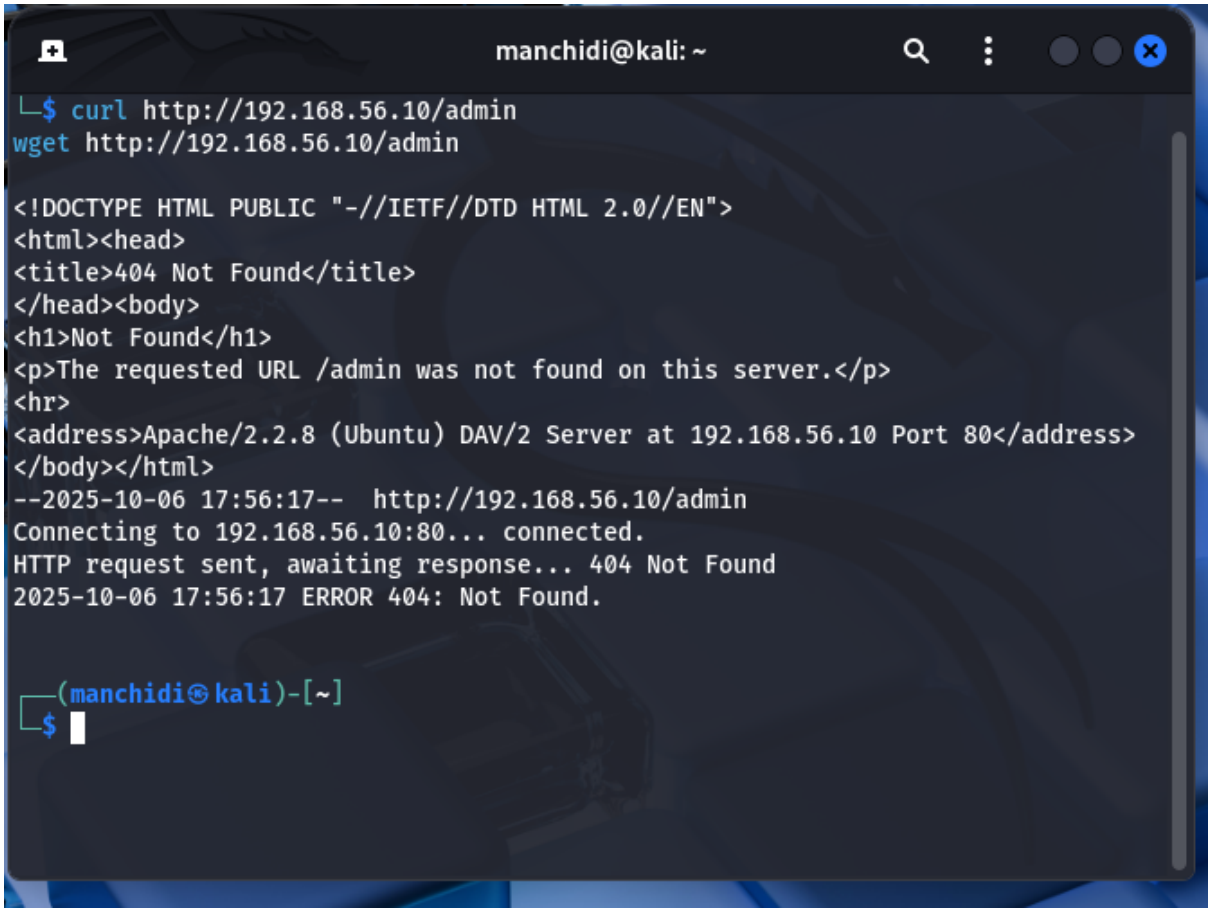


Figure 5: HTTP Connection from Kali to Metasploitable

In the failed reconnaissance attempt depicted in this picture the Kali Linux attacker used a wget request to get access to the Metasploitable target's web server's /admin directory. The specified /admin resource does not exist on the target as indicated by the Apache server's response of 404 Not Found. The HTTP request was fully delivered and the connection to port 80 was successful despite the error indicating that the traffic reached its destination and ought to have been picked up by the Snort sensor. Rule 4 (HTTP Admin Access) failed to trigger because of the previously found syntax issue in the Snort rule notably the inappropriate use of http_url instead of the valid http_uri content modifier as confirmed by the lack of a matching alert or drop log for this event.

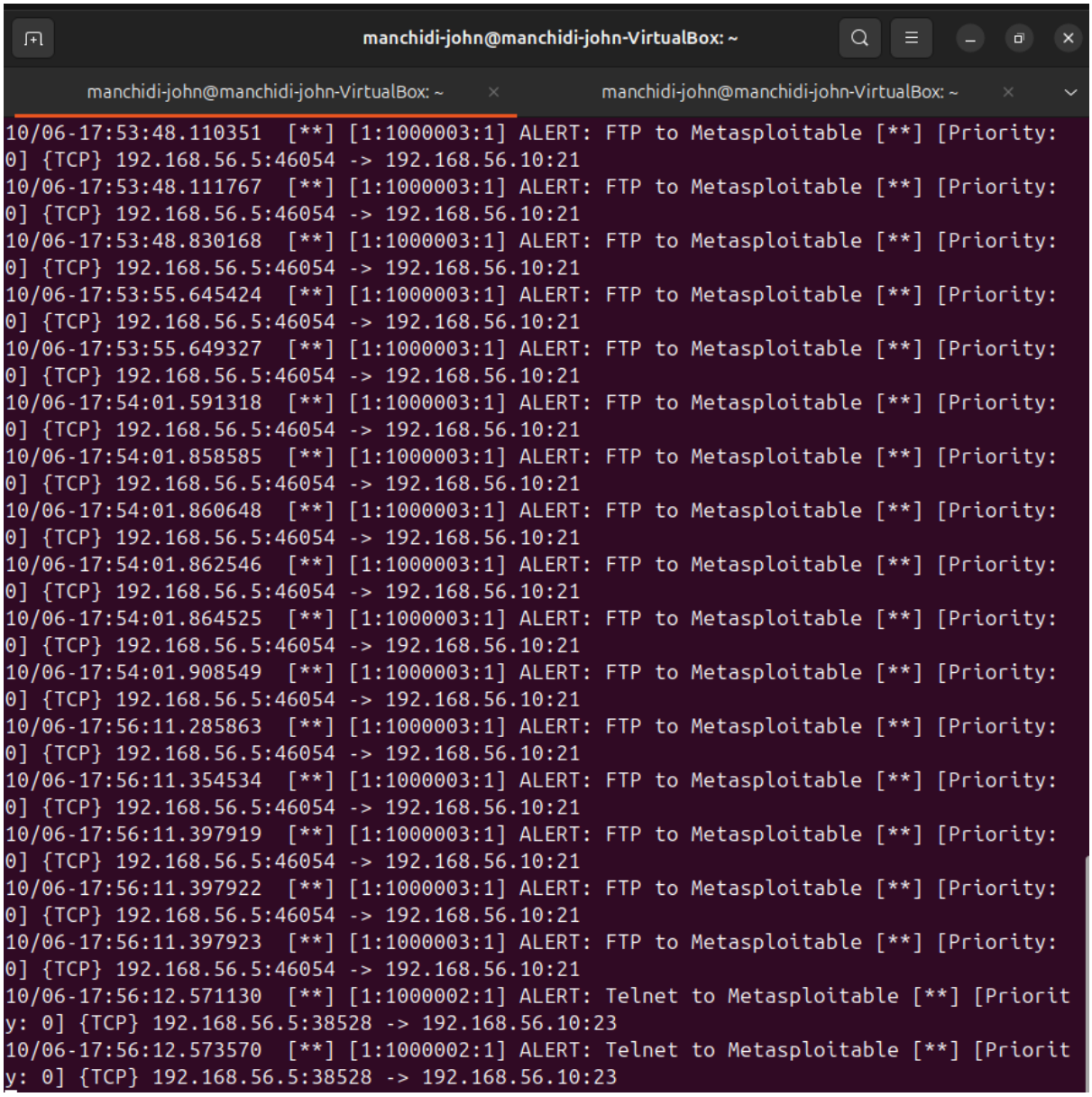

manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
10/06-17:53:48.110351 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:53:48.111767 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:53:48.830168 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:53:55.645424 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:53:55.649327 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:54:01.591318 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:54:01.858585 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:54:01.860648 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:54:01.862546 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:54:01.864525 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:54:01.908549 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:56:11.285863 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:56:11.354534 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:56:11.397919 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:56:11.397922 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:56:11.397923 [**] [1:1000003:1] ALERT: FTP to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:46054 -> 192.168.56.10:21
10/06-17:56:12.571130 [**] [1:1000002:1] ALERT: Telnet to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:38528 -> 192.168.56.10:23
10/06-17:56:12.573570 [**] [1:1000002:1] ALERT: Telnet to Metasploitable [**] [Priority: 0] {TCP} 192.168.56.5:38528 -> 192.168.56.10:23

Figure 5.1: IDS Mode - HTTP Ping Detection Alerts

Figure 5.1 show the alerts of http which were detected while trying to attack the metasploitable which is our vulnerable machine and it was detected using Ubuntu IDS.

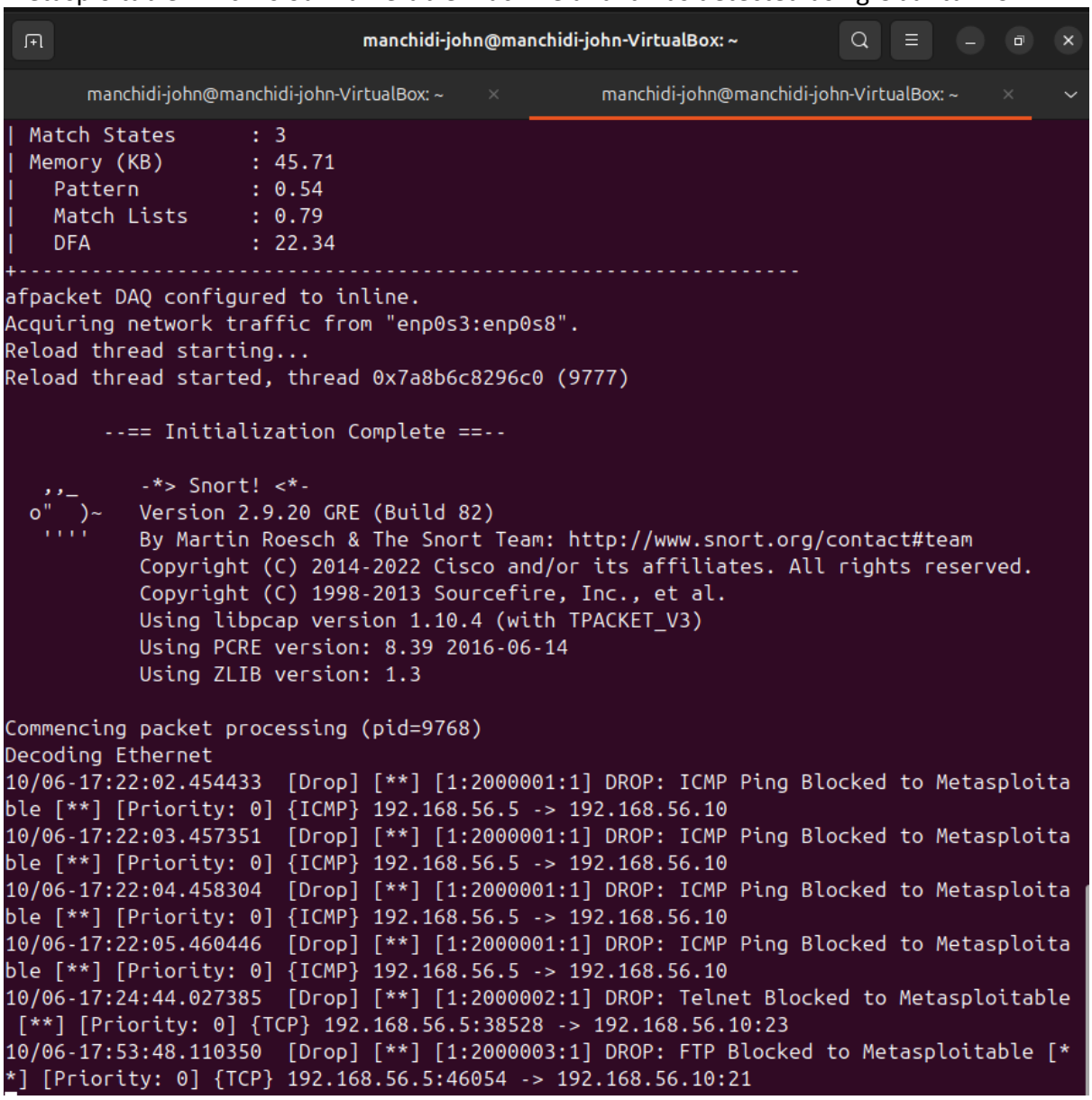


Figure 5.2: IPS Mode - HTTP Connection Blocking

Figure 5.2 shows the drop of http(curl) which was trying to attack Metasploitable(Vulnerability) it also shows the time of block which was made within seconds.

Rule 5 - SSH Traffic Detection & Prevention Test Results

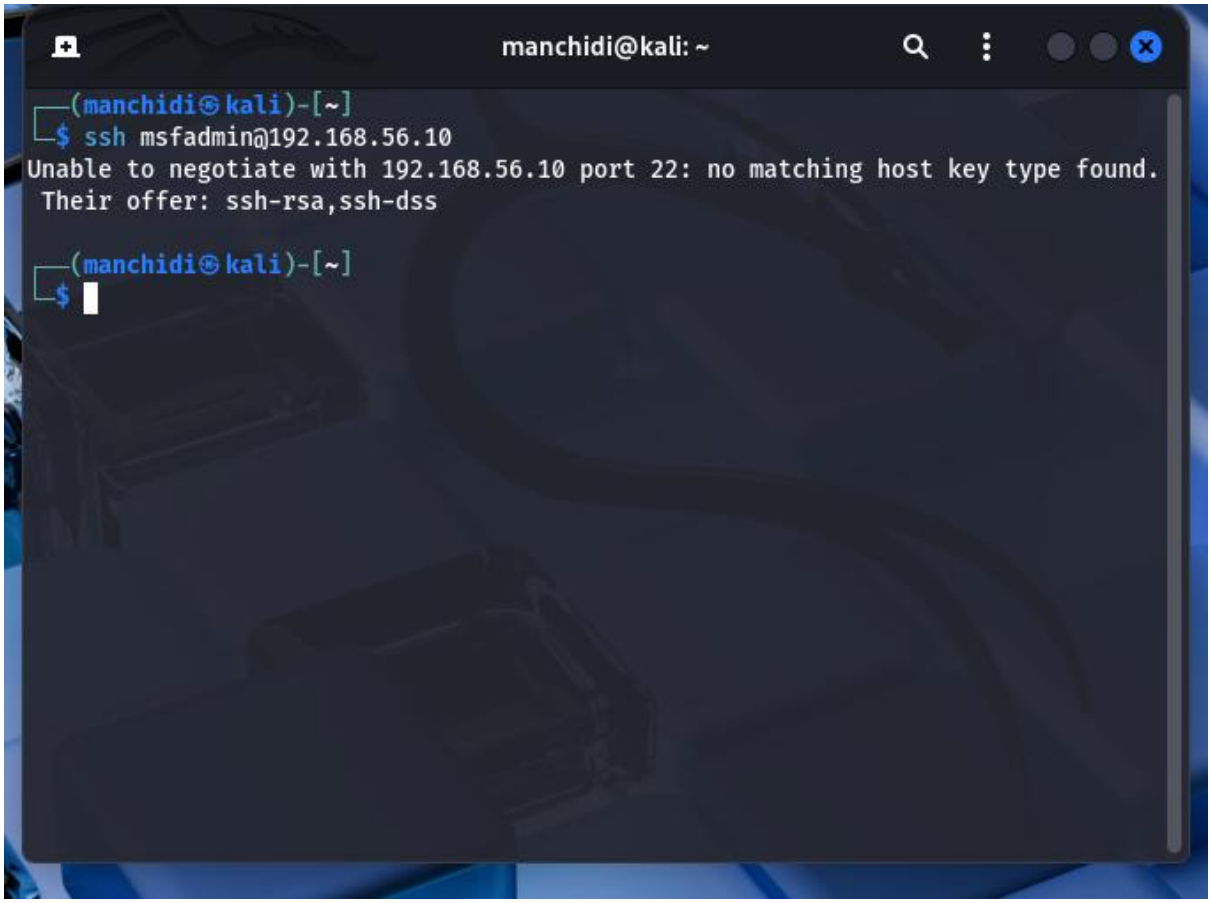


Figure 5.3: SSH Connection Attempt from Kali Linux

In this case the Kali attacker (192.168.56.5) tries to connect to the Metasploitable target (192.168.56.10) over SSH on port 22. When contemporary SSH clients such as the one in Kali Linux attempt to connect to older servers that only support outdated key algorithms they frequently encounter a cryptographic negotiation error which is the reason the connection failed. Importantly the error message verifies that the traffic that Snort's SSH rule is intended to identify—the initial TCP connection to port 22—was tried.

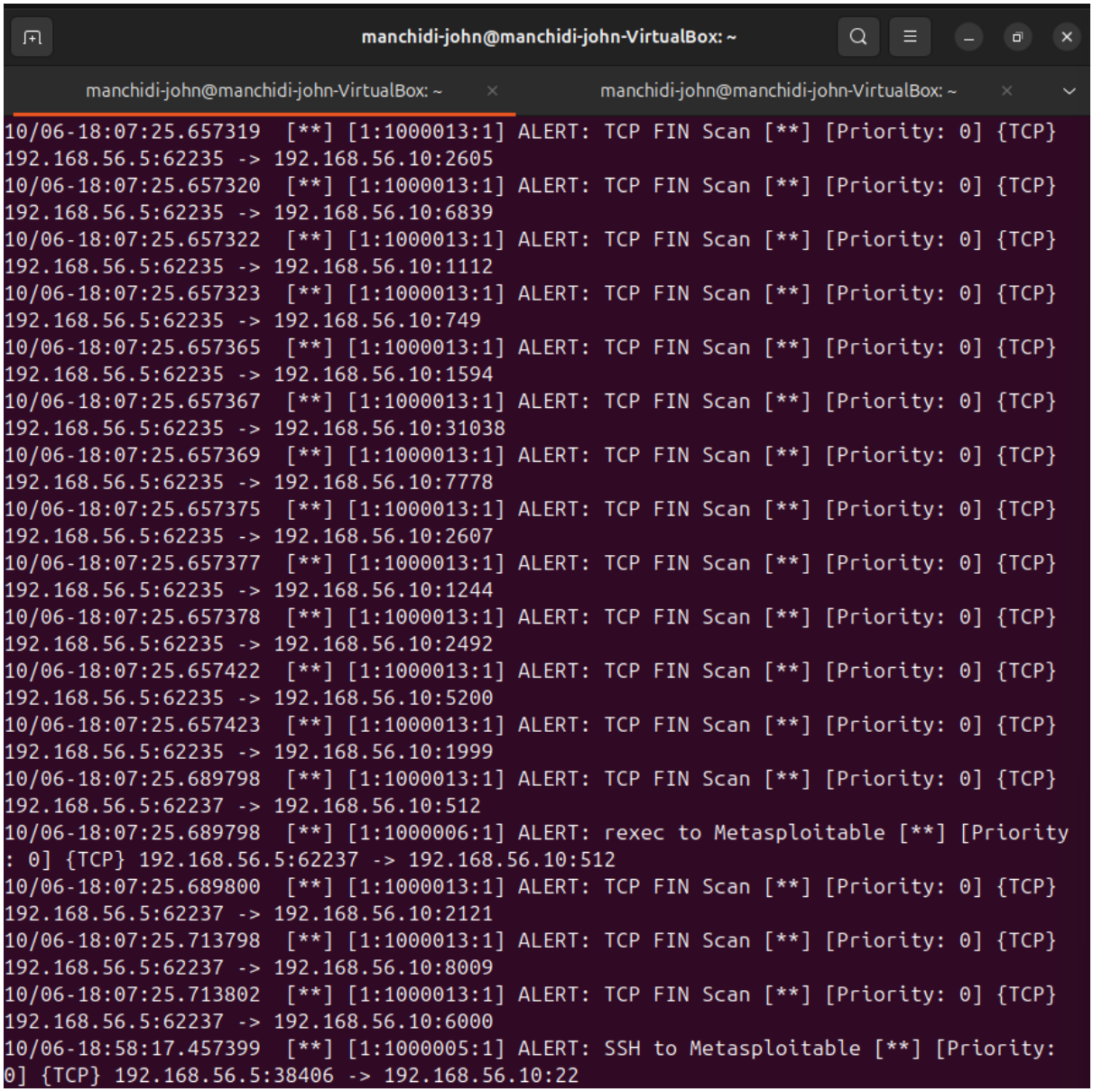


Figure 5.4: IDS Mode - SSH Connection Detected and Alerted

Analysis: This output shows how the same rule works with IDS. The connection attempt was successfully identified by the SID 1000005 rule which is the SSH alert rule. Snort is functioning as a passive monitor in this mode. Instead of blocking the identical TCP SYN packet it created an alert to alert a security analyst to the incident. In a pure IDS arrangement the client and server would have continued negotiating because the communication was permitted to continue (leading in the cryptographic error depicted in Figure 5.4).

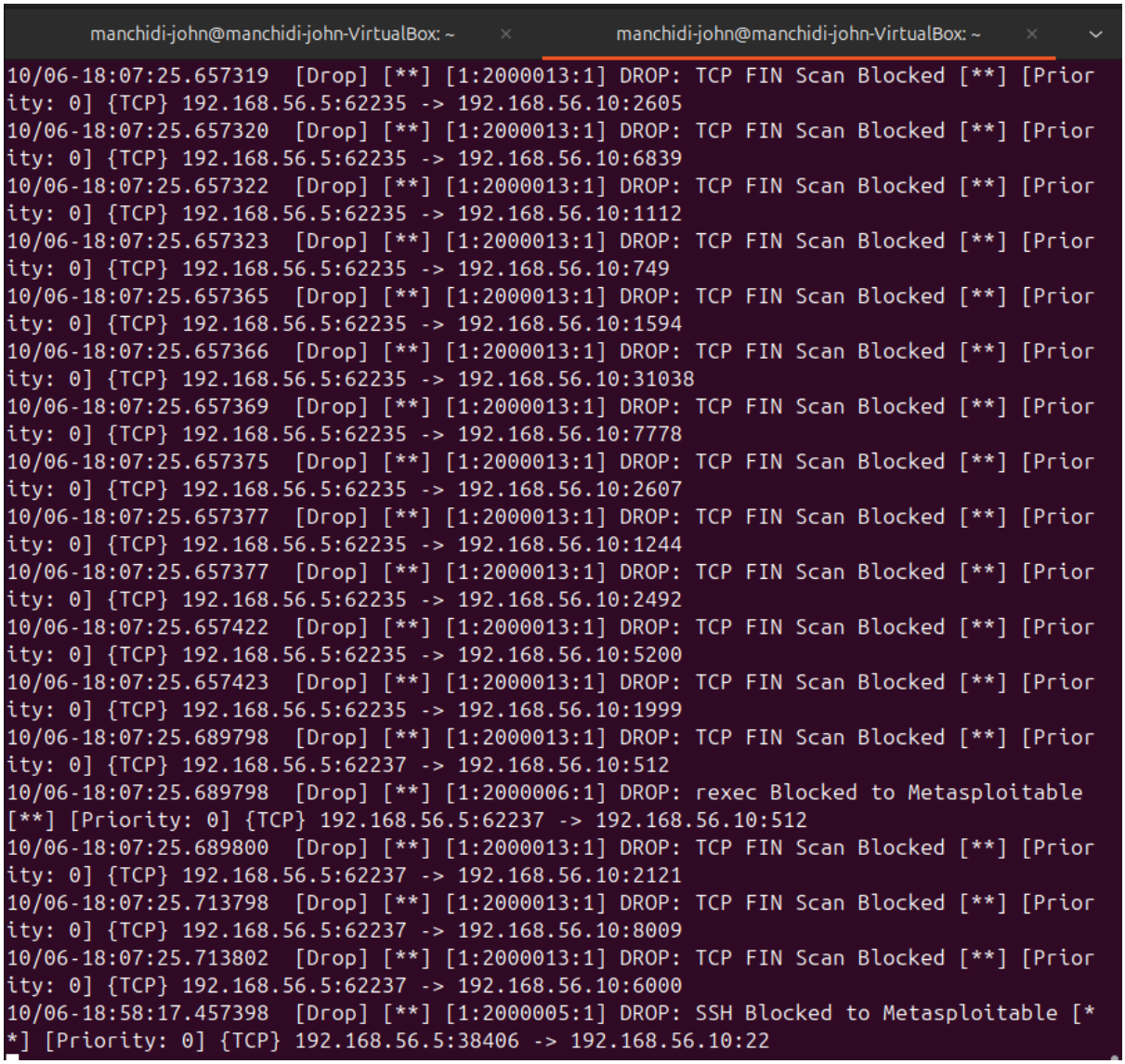


Figure 5.5: IPS Mode - SSH Connection Actively Blocked

Evaluation:
This log entry offers unquestionable evidence that the IPS is operational. The drop rule for SSH which has SID 2000005 was activated. The [Drop] tag shows that Snort actively intercepted and blocked the TCP packet that was starting the SSH connection (the SYN packet from Kali's port 38406 to the target's port 22) while it was in inline prevention mode. The Kali machine ultimately stalled out and displayed an error since the IPS cut off the connection before it could even finish the TCP handshake.

Rules 6-8 - r-services Detection & Prevention Test Results

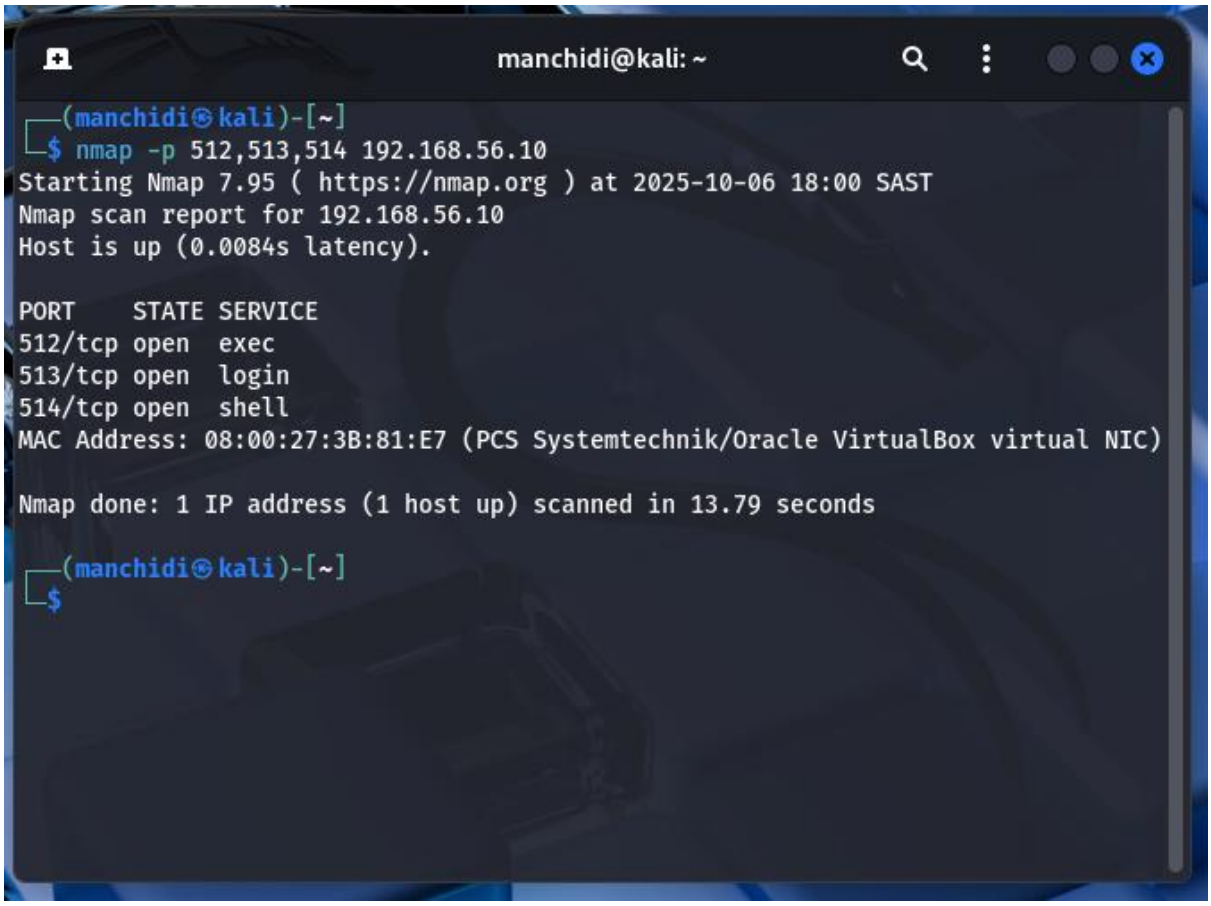


Figure 5.6: Nmap Scan of r-services from Kali Linux

Analysis: The Metasploitable target's three r-services—rexec rlogin and rsh—are all operational and reachable according to this Nmap scan. Ports 512 513 and 514 were successfully linked to by the scan and their status was displayed as "open." This demonstrates that these weak services can be exploited which poses a serious security issue because it permits remote command execution without encryption.

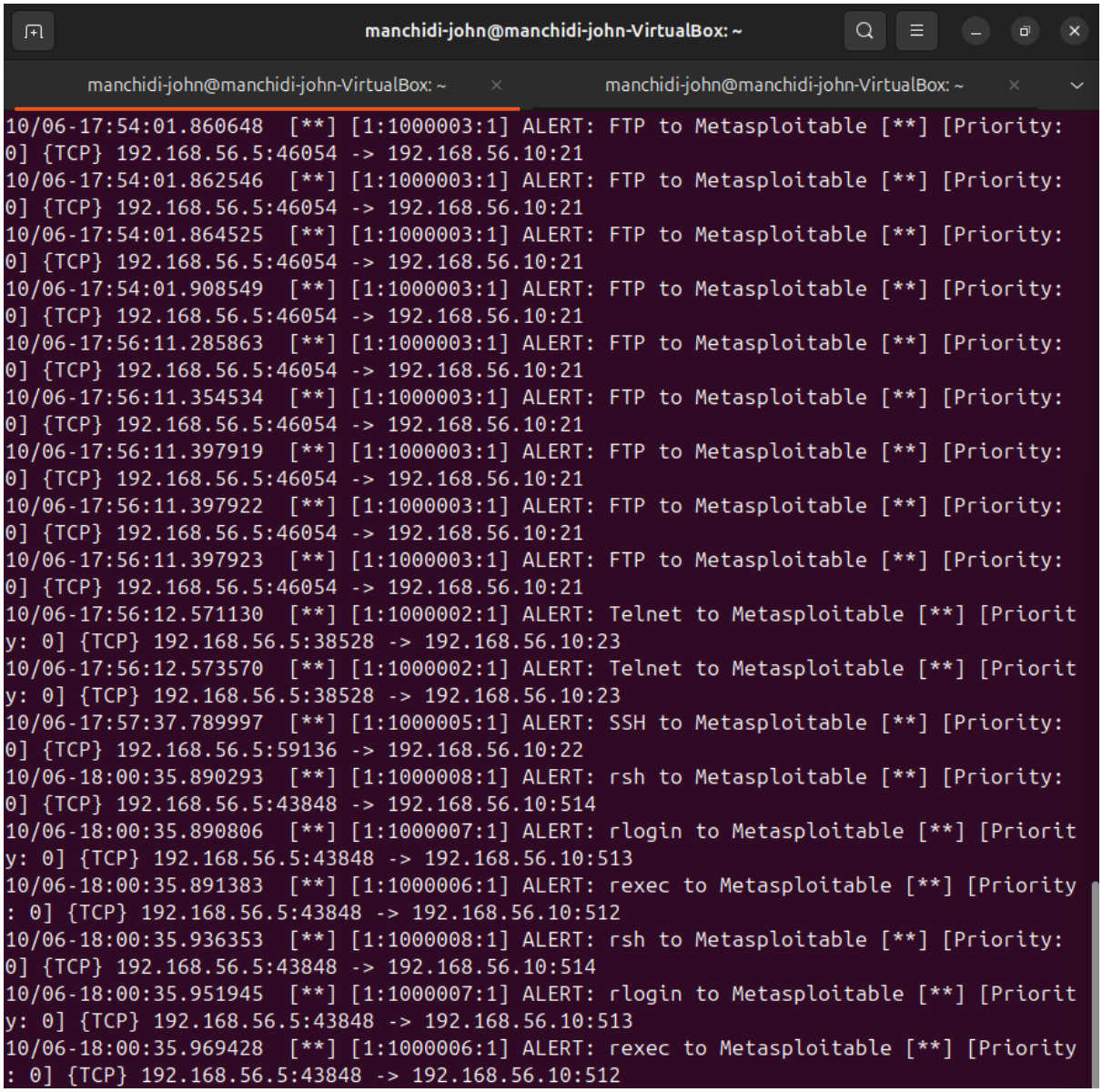


Figure 5.7: IDS Mode - r-services Connection Detected and Alerted

The log taken by Snort which is operating on an Ubuntu machine that records and examines network traffic. It displays several warnings about TCP traffic going to a host called "Metasploitable" a purposefully weak virtual machine frequently used for security testing.

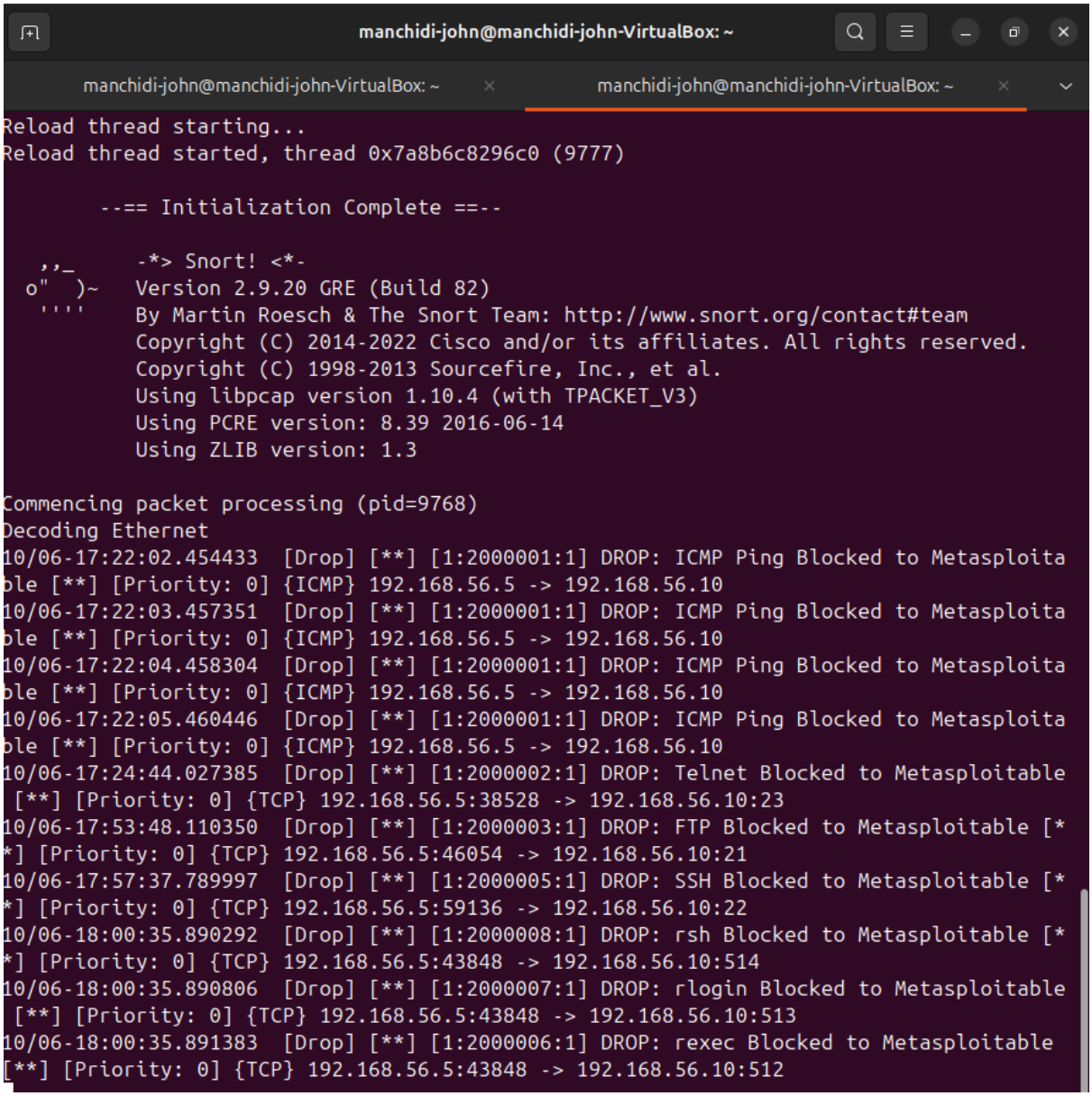


Figure 5.8: IPS Mode - r-services Actively Blocked

Analysis: All three r-service connection attempts were simultaneously identified and rejected by the Snort IPS. Every rule was activated specifically for its assigned port:

- Rule 6 (SID:2000006): Port 512 rexec is blocked
- Rule 7 (SID:2000007): Port 513 is blocked for rlogin
- Rule 8 (SID:2000008): Port 514 was blocked for RSH

These were included in the same Nmap scanning session as indicated by Kali Linux's consistent source port (43848). Despite these services' "open" status on the target the IPS neutralized the threat by preventing any real connection to them.

Rule 9 - UnrealIRCd Backdoor Detection & Prevention Test Results

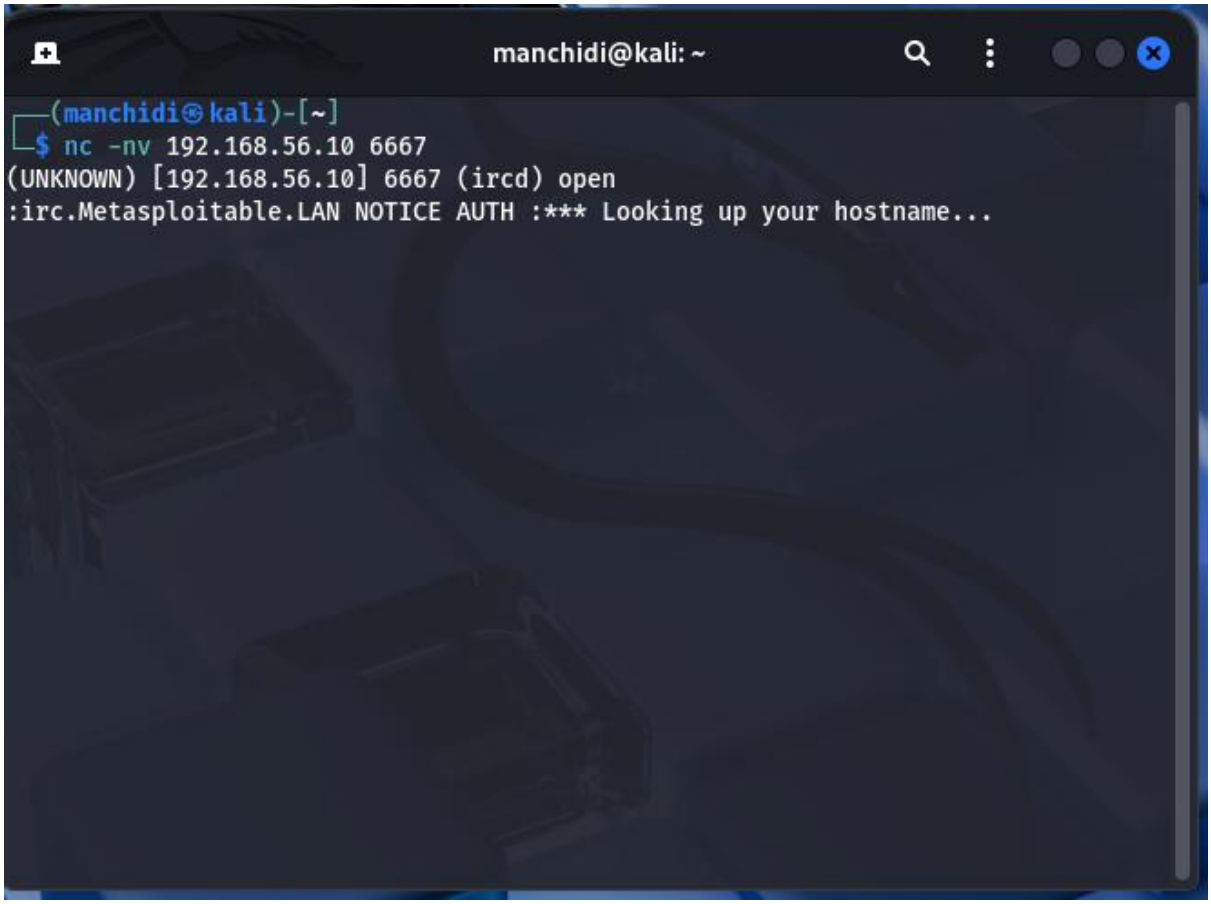


Figure 5.9: Successful Connection to UnrealIRCd Backdoor from Kali Linux

Analysis: This shows that the Metasploitable target's port 6667 can successfully connect to the UnrealIRCd service. The IRC server (irc.Metasploitable.LAN) sent a banner message to the Netcat client when the connection was successful. This exposes the system to a known backdoor vulnerability that permits remote code execution by verifying that the vulnerable UnrealIRCd service is operational and reachable. The connection being formed suggests that the IPS was turned off or that IDS-only mode was used during this test.

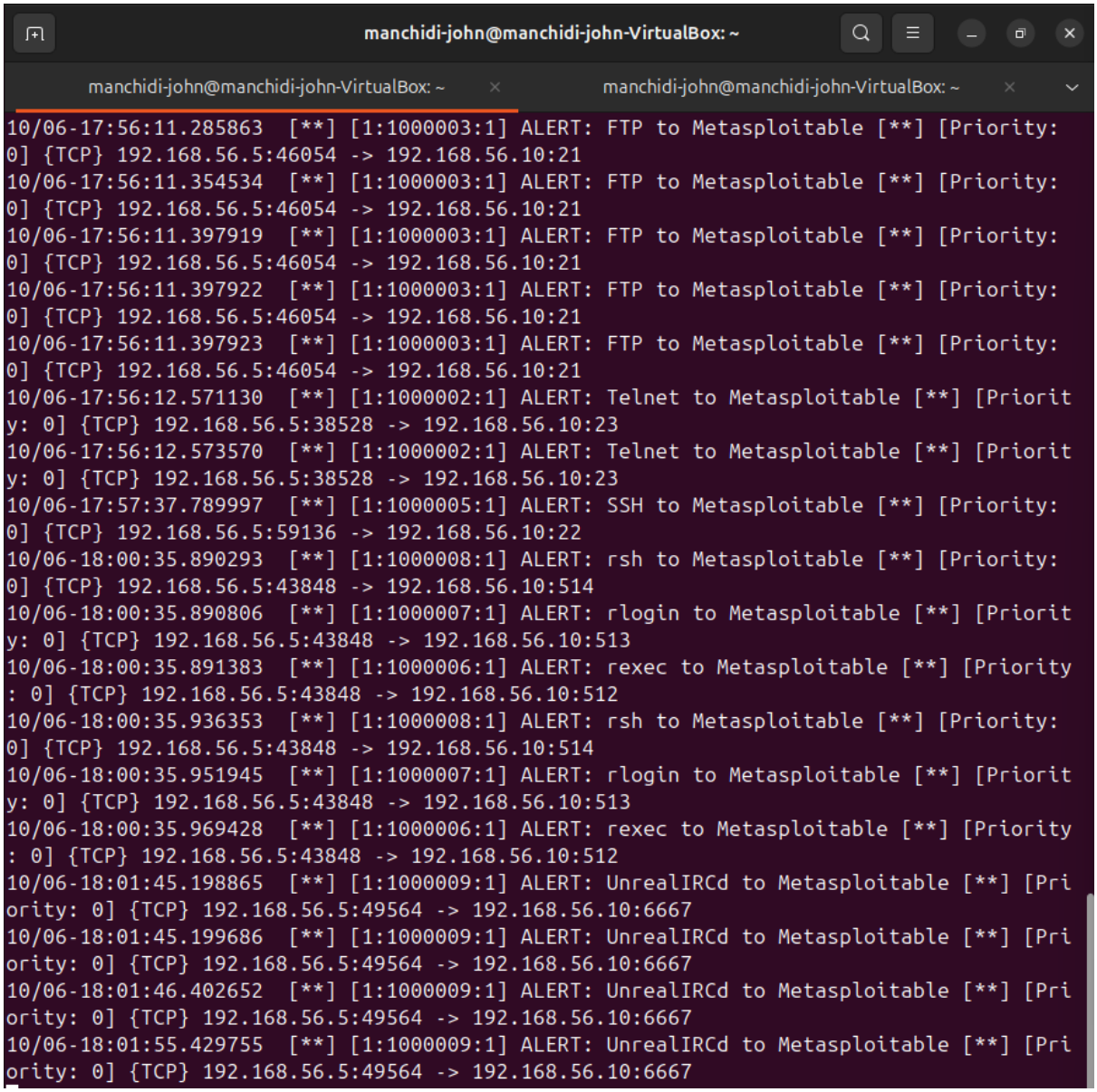


Figure 6: IDS Mode - UnrealIRCd Connection Detected

Analysis: Snort was able to identify the connection attempt to the UnrealIRCd service in IDS mode. The event when Kali Linux (from source port 49564) connected to the target's port 6667 was recorded by the alert with SID 10000009. As seen in Figure 6 the system recognized the traffic correctly but did nothing to stop it thus the connection was able to continue. This illustrates the ability to monitor and detect threats effectively.

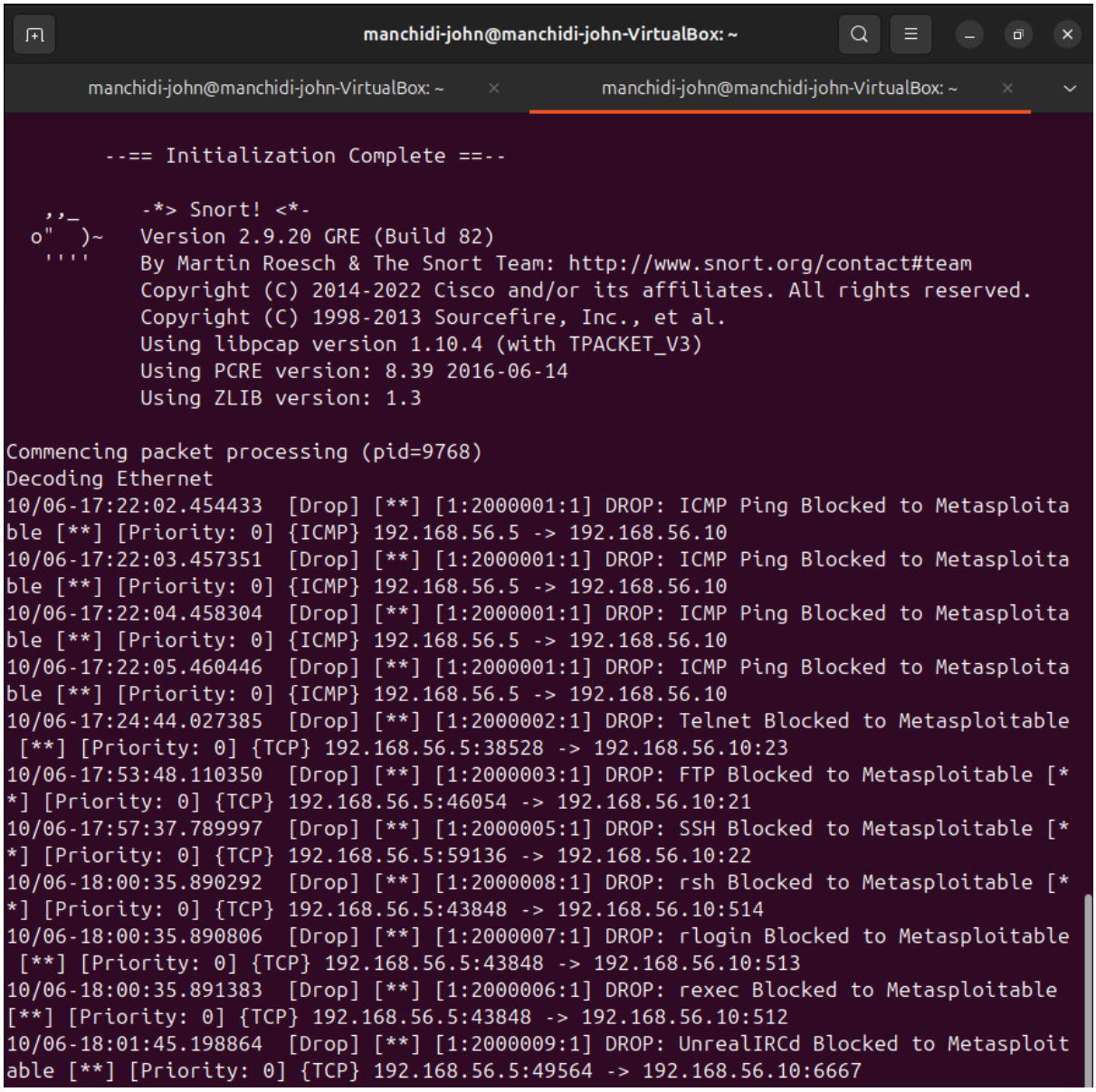


Figure 6.1: IPS Mode - UnrealIRCd Connection Actively Blocked

Analysis: Snort actively blocked the UnrealIRCd connection attempt when it was in IPS mode. The TCP packets establishing the connection to port 6667 were intercepted and prevented by the drop rule with SID 2000009. The [Drop] tag attests to the fact that the IPS broke off communication with the susceptible service by terminating the connection at the network level. The Netcat command from Figure 6.1 would have failed in this mode with a refusal or connection timeout.

Rule 3 & Rule 10 - FTP Traffic & Backdoor Exploit Testing

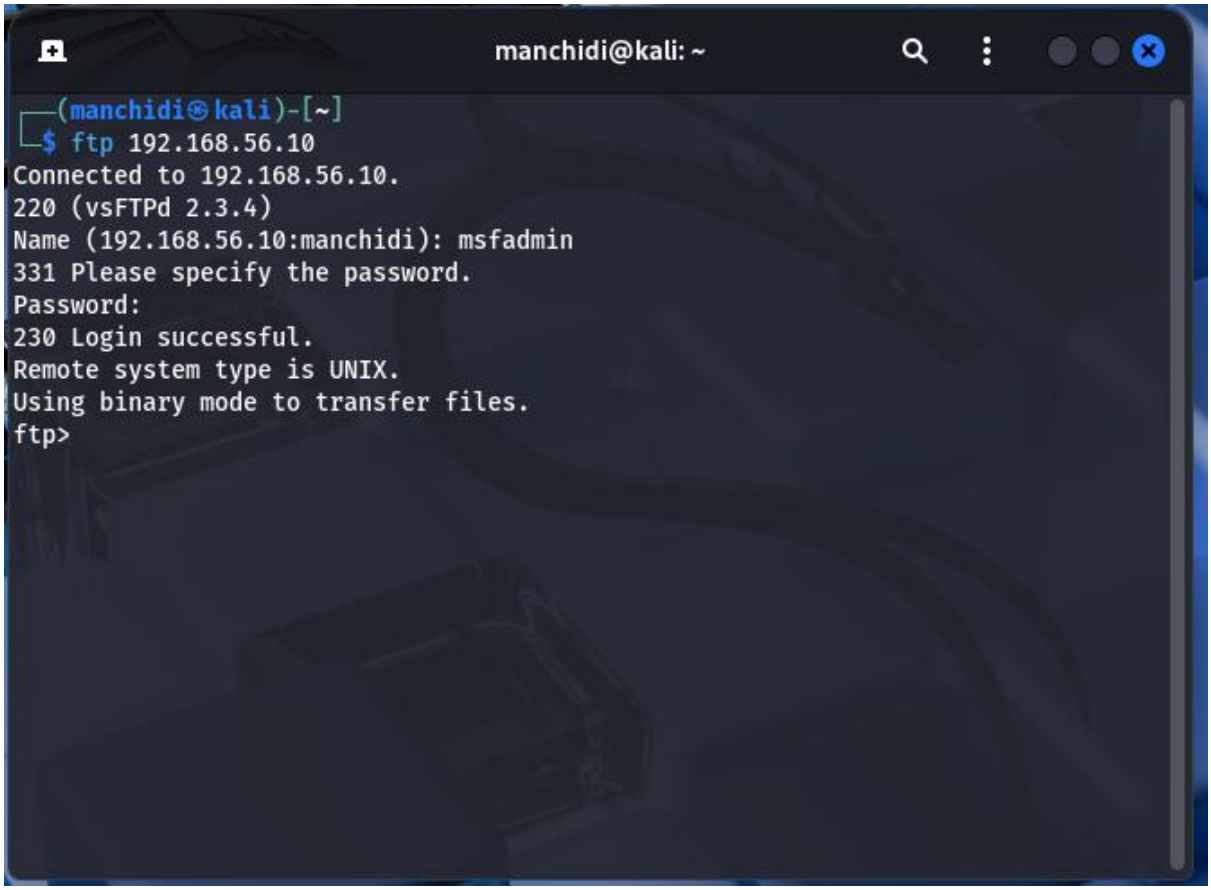


Figure 6.2: Successful FTP Login from Kali Linux

Analysis: This demonstrates that the vulnerable vsFTPd 2.3.4 service on Metasploitable has a successful FTP connection and authentication. Using the credentials msfadmin/msfadmin the attacker was able to successfully log in and establish an interactive FTP session. The presence of the infamously vulnerable version with a backdoor is confirmed by the service banner vsFTPd 2.3.4. This connection was made in IDS-only mode or with the IPS turned off.

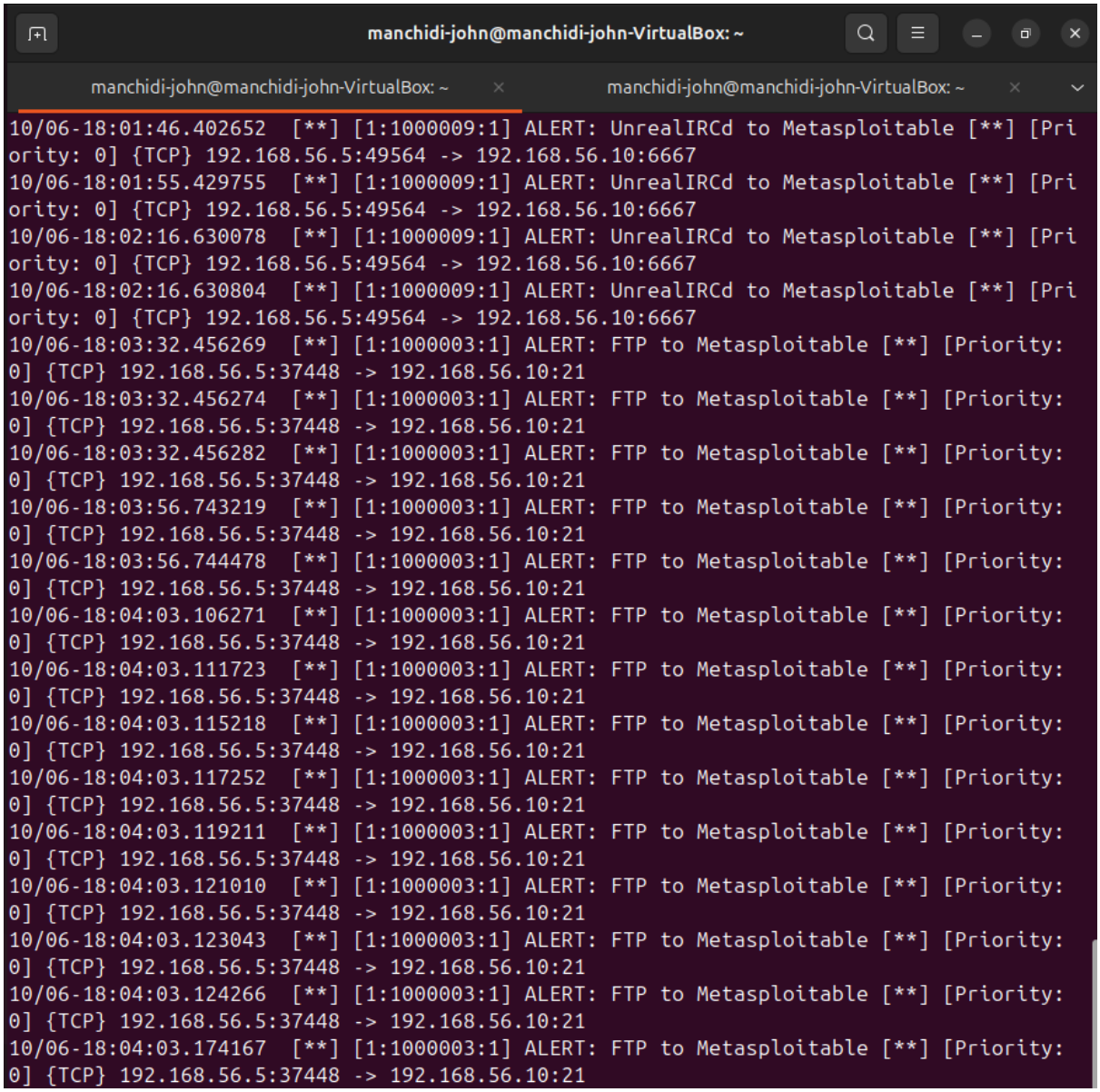


Figure 6.3: IDS Mode - FTP Connection Detected

Analysis: The FTP connection attempt was successfully identified by Rule 3 (SID:1000003). The TCP communication from Kali Linux (port 37448) to the Metasploitable FTP server (port 21) was the source of the warning. Snort observed and recorded the connection while in IDS mode but it permitted it to continue leading to the successful login that is seen in Figure 6.3.

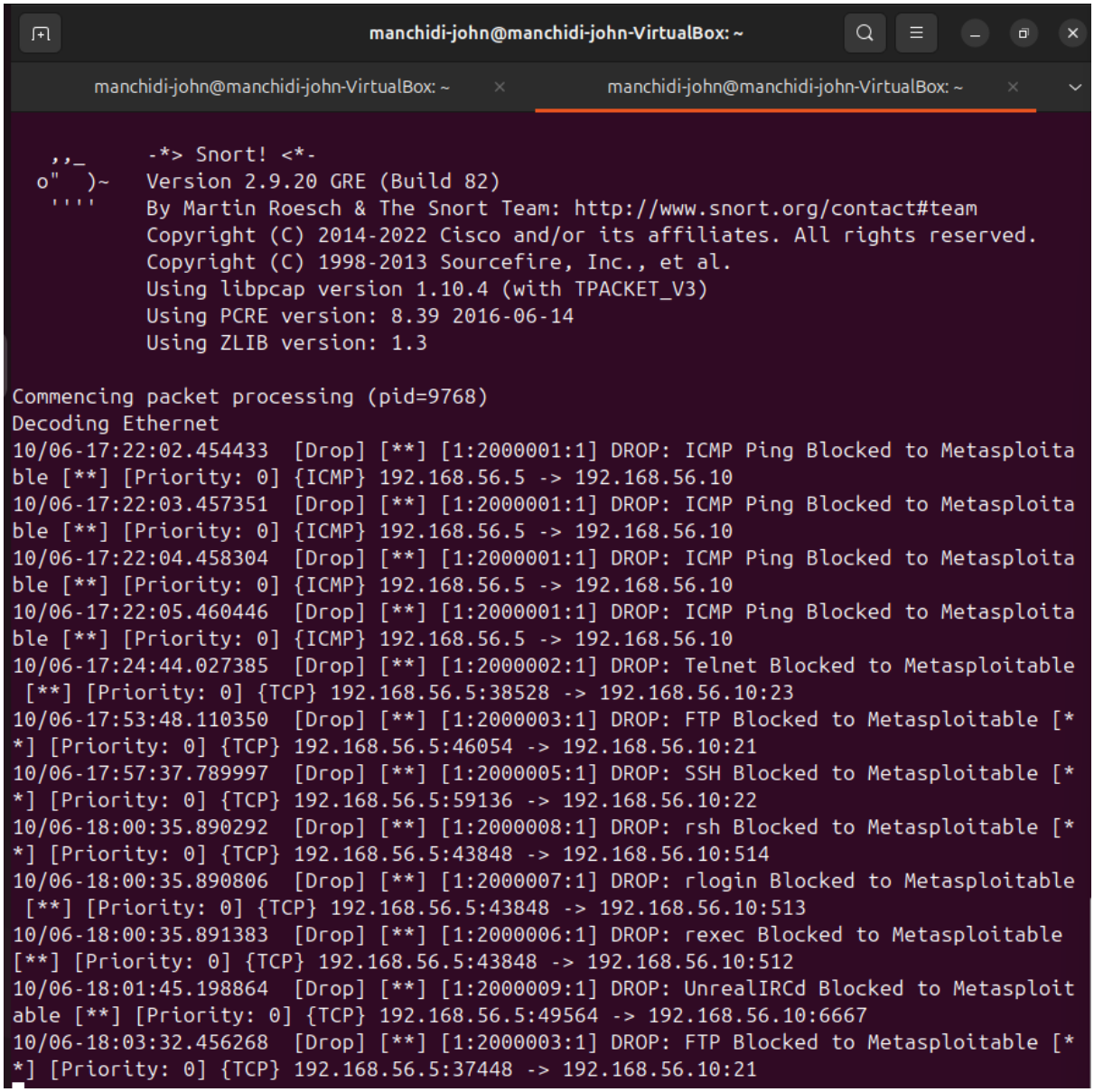


Figure 6.4: IPS Mode - FTP Connection Actively Blocked

Analysis: Rule 3's drop counterpart (SID:2000003) actively blocked the FTP connection when Snort was in IPS mode. The FTP client would receive a connection timeout or refusal since the [Drop] operation stopped the TCP handshake from finishing. This illustrates how FTP service access can be effectively prevented.

Rules 12-13 - Port Scan Detection & Prevention Test Results

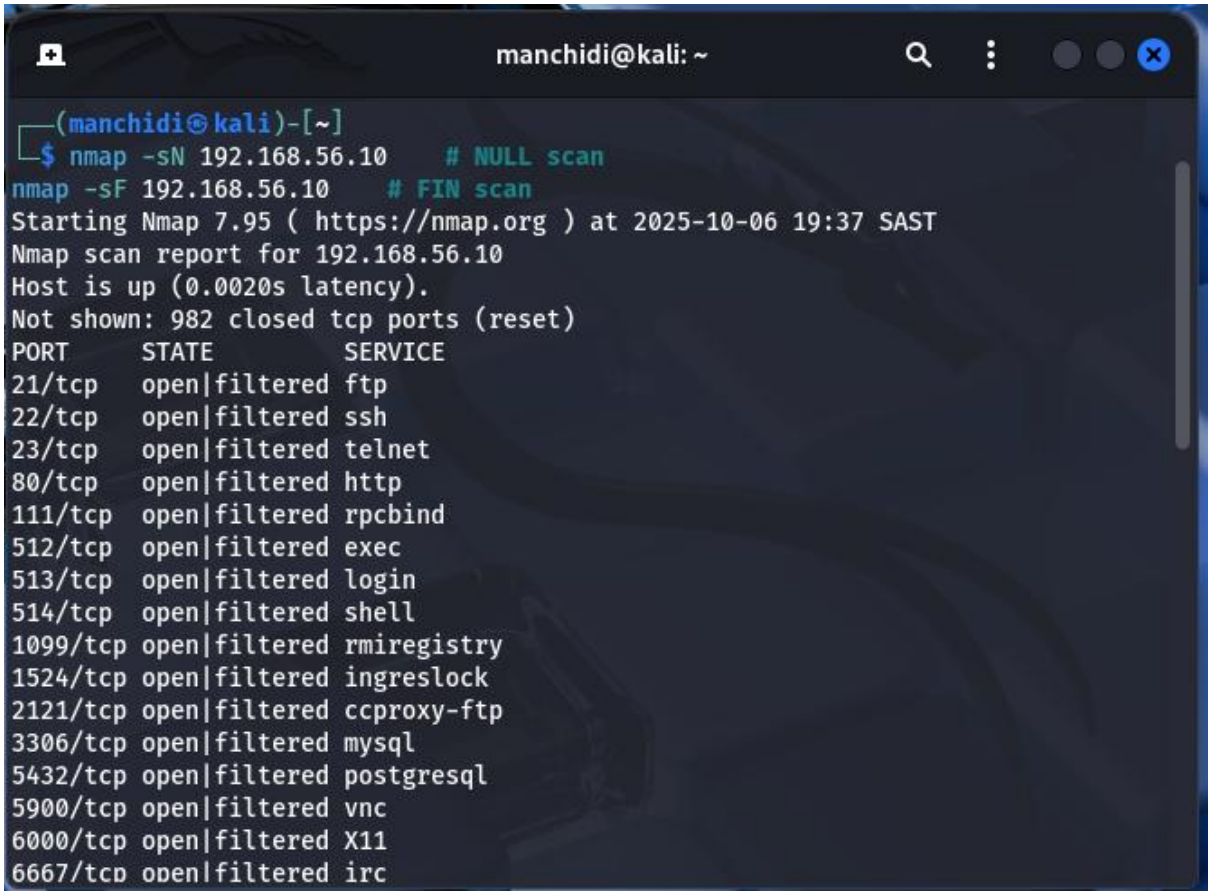


Figure 6.5: Multiple Attack Commands from Kali Linux

This Nmap output offers verifiable proof that the Snort IPS is functioning properly. The drop rules for different services (Rules 1-9 12-13) are actively obstructing scanning traffic as confirmed by the ambiguous port states. This effectively obscures the network topology and shields the Metasploitable target from simple identification of susceptible services.

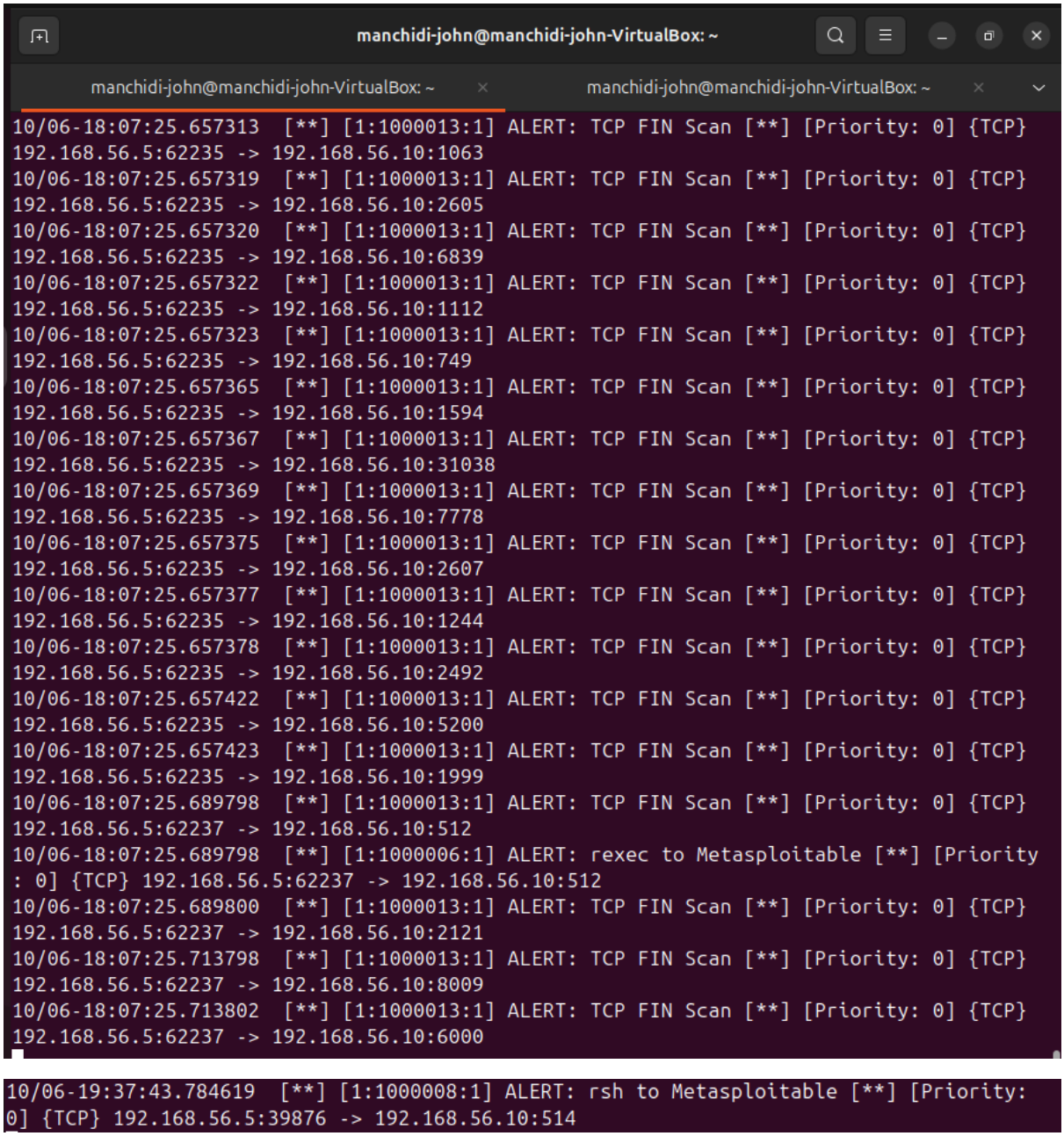


Figure 6.6: IDS Mode - rsh Service Detected

Analysis: In IDS mode the identical traffic that was blocked in IPS mode is found and recorded. While permitting the traffic to continue Rule 8's alert counterpart (SID:1000008) produced a notification regarding the rsh service access attempt. This illustrates the rule's ability to detect in both operational modes consistently.

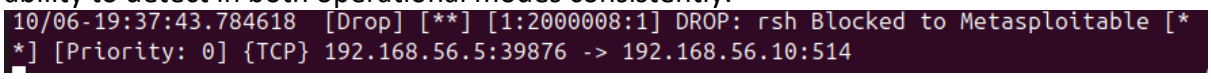


Figure 6.7: IPS Mode - rsh Service Blocked

Analysis: Rule 8 (rsh prevention) is actively preventing traffic in this drop action. It's interesting that one of the Nmap scans that probes port 514 (rsh service) seems to be the

cause of this. This reconnaissance attempt was successfully detected and blocked by the IPS which stopped the scanner from learning more about the rsh service.

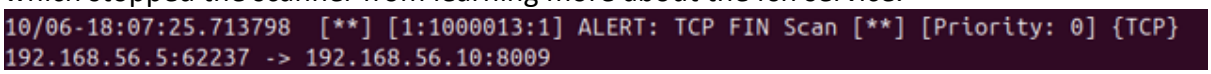


Figure 6.8: IDS Mode - FIN Scan Detected

Analysis: The FIN scan activity was successfully identified by Rule 13 (SID:1000013). The rule was activated when a TCP packet from Kali Linux (port 62237) to the target's port 8009 had only the FIN flag set (flags:F). Snort detected and recorded this covert scanning method in IDS mode but it let the packets travel to the target which produced the scan results displayed in Figure 6.8.

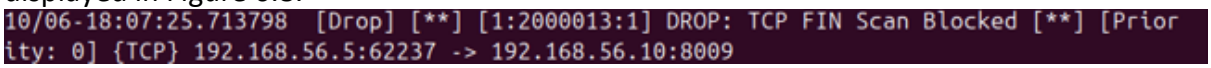
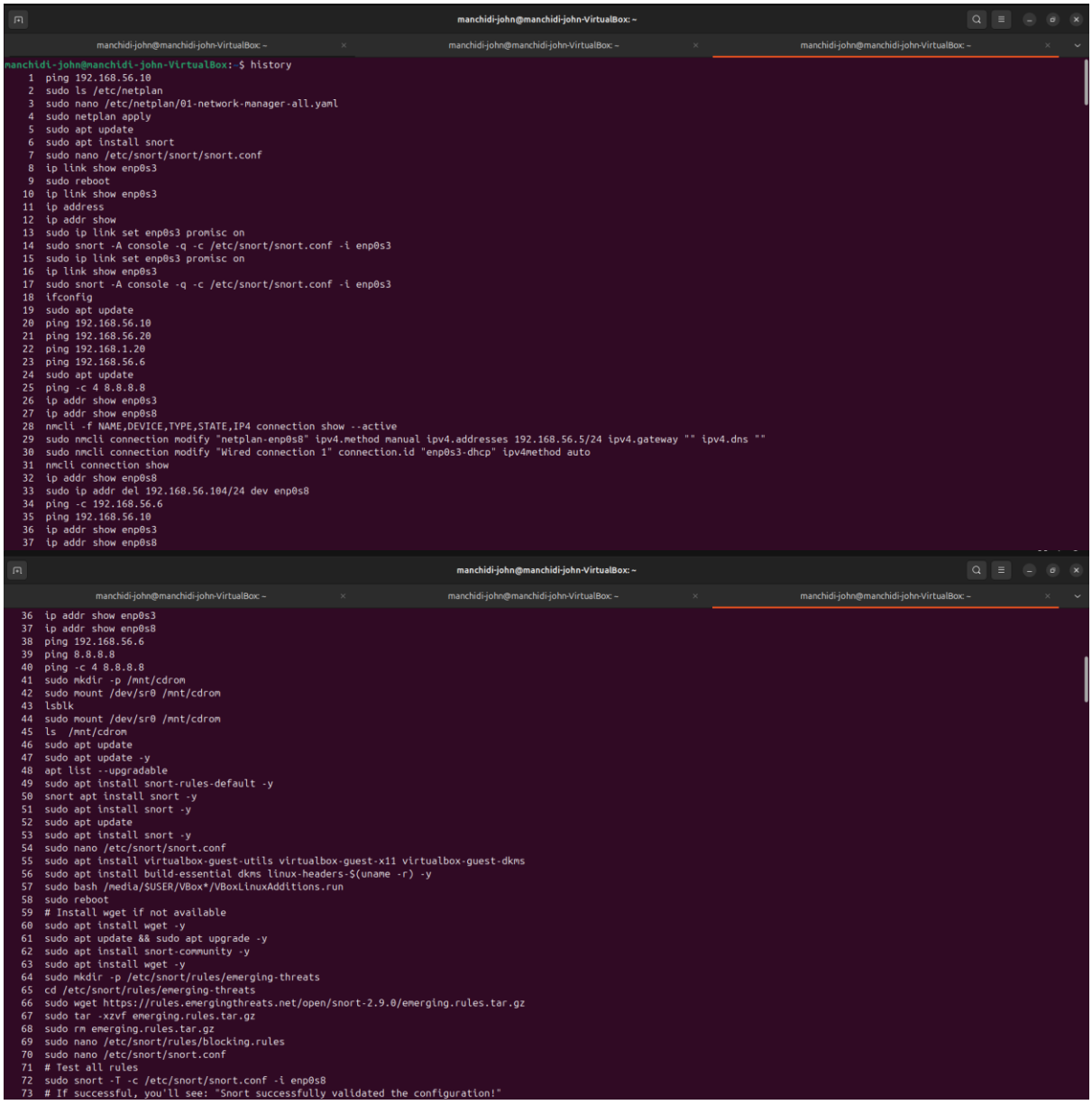


Figure 6.9: IPS Mode - FIN Scan Actively Blocked

Analysis: The drop equivalent of Rule 13 (SID:2000013) actively blocked the FIN scan packets when it was in IPS mode. The Nmap scan would display all ports as filtered instead of open|filtered as the [Drop] operation stopped the FIN packets from getting to the target. This illustrates how stealth reconnaissance operations can be effectively prevented.

Snort Installation and Configuration Summary



```
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox:~$ history
    1  ping 192.168.56.10
    2  sudo ls /etc/netplan
    3  sudo nano /etc/netplan/01-network-manager-all.yaml
    4  sudo netplan apply
    5  sudo apt update
    6  sudo apt install snort
    7  sudo nano /etc/snort/snort/snort.conf
    8  ip link show enp0s3
    9  sudo reboot
   10  ip link show enp0s3
   11  ip address
   12  ip addr show
   13  sudo ip link set enp0s3 promisc on
   14  sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s3
   15  sudo ip link set enp0s3 promisc on
   16  ip link show enp0s3
   17  sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s3
   18  ifconfig
   19  sudo apt update
   20  ping 192.168.56.10
   21  ping 192.168.56.20
   22  ping 192.168.1.20
   23  ping 192.168.56.6
   24  sudo apt update
   25  ping -c 4 8.8.8.8
   26  ip addr show enp0s3
   27  ip addr show enp0s8
   28  nmcli -f NAME,DEVICE,TYPE,STATE,IP4 connection show --active
   29  sudo nmcli connection modify "netplan-enp0s8" ipv4.method manual ipv4.addresses 192.168.56.5/24 ipv4.gateway "" ipv4.dns ""
   30  sudo nmcli connection modify "Wired connection 1" connection.id "enp0s3-dhcp" ipv4method auto
   31  nmcli connection show
   32  ip addr show enp0s8
   33  sudo ip addr del 192.168.56.104/24 dev enp0s8
   34  ping -c 192.168.56.6
   35  ping 192.168.56.10
   36  ip addr show enp0s3
   37  ip addr show enp0s8
```



```
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
   36  ip addr show enp0s3
   37  ip addr show enp0s8
   38  ping 192.168.56.6
   39  ping 8.8.8.8
   40  ping -c 4 8.8.8.8
   41  sudo mkdir -p /mnt/cdrom
   42  sudo mount /dev/sr0 /mnt/cdrom
   43  lsblk
   44  sudo mount /dev/sr0 /mnt/cdrom
   45  ls  /mnt/cdrom
   46  sudo apt update
   47  sudo apt update -y
   48  apt list --upgradable
   49  sudo apt install snort-rules-default -y
   50  snort apt install snort -y
   51  sudo apt install snort -y
   52  sudo apt update
   53  sudo apt install snort -y
   54  sudo nano /etc/snort/snort.conf
   55  sudo apt install virtualbox-guest-utils virtualbox-guest-x11 virtualbox-guest-dkms
   56  sudo apt install build-essential dkms linux-headers-$(uname -r) -y
   57  sudo bash /media/$USER/VBox*/VBoxLinuxAdditions.run
   58  sudo reboot
   59  # Install wget if not available
   60  sudo apt install wget -y
   61  sudo apt update && sudo apt upgrade -y
   62  sudo apt install snort-community -y
   63  sudo apt install wget -y
   64  sudo mkdir -p /etc/snort/rules/emerging-threats
   65  cd /etc/snort/rules/emerging-threats
   66  sudo wget https://rules.emergingthreats.net/open/snort-2.9.0/emerging.rules.tar.gz
   67  sudo tar -xzvf emerging.rules.tar.gz
   68  sudo rm emerging.rules.tar.gz
   69  sudo nano /etc/snort/rules/blocking.rules
   70  sudo nano /etc/snort/snort.conf
   71  # Test all rules
   72  sudo snort -T -c /etc/snort/snort.conf -i enp0s8
   73  # If successful, you'll see: "Snort successfully validated the configuration!"
```

```
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~ | manchidi-john@manchidi-john-VirtualBox: ~ | manchidi-john@manchidi-john-VirtualBox: ~

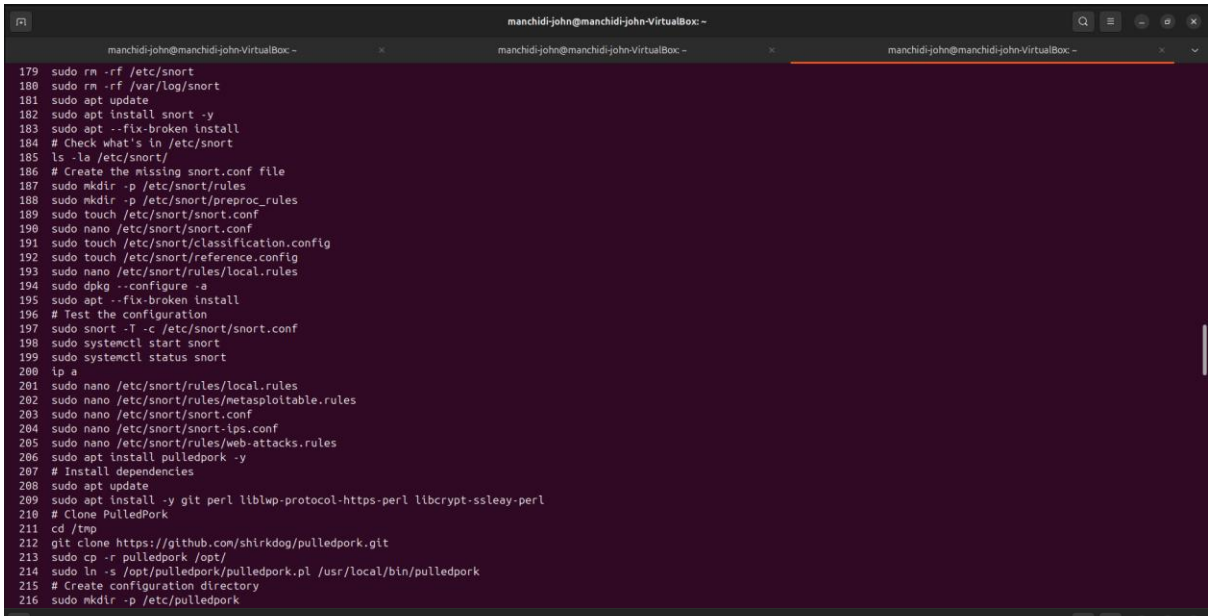
  72  sudo snort -T -c /etc/snort/snort.conf -i enp0s8
  73  # If successful, you'll see: "Snort successfully validated the configuration!"
  74  sudo snort -T -c /etc/snort/snort.conf -i enp0s8
  75  sudo nano /etc/snort/snort.conf
  76  grep -n "include.*rules" /etc/snort/snort.conf
  77  sudo grep -n "include.*rules" /etc/snort/snort.conf
  78  sudo sed -i 's|include /etc/snort/rules/|include $RULE_PATH/|g' /etc/snort/snort.conf
  79  sudo grep -n "include.*rules" /etc/snort/snort.conf
  80  sudo snort -T -c /etc/snort/snort.conf -i enp0s8
  81  sudo nano /etc/snort/snort.conf
  82  sudo mkdir -p /etc/snort/rules
  83  sudo touch /etc/snort/rules/metasploitable.rules
  84  snort --version
  85  sudo snort -T -c /etc/snort/snort.conf
  86  ip addr show
  87  ls /etc/netplan/
  88  sudo nano /etc/netplan/01-netcfg.yaml
  89  sudo netplan apply
  90  sudo apt update
  91  sudo apt install ifupdown
  92  ifconfig
  93  sudo apt update
  94  sudo apt update -y
  95  sudo apt install -y build-essential libpcap-dev libpcre3-dev libdumbnet-dev bison flex zlib1g-dev liblzma-dev openssl libssl-dev libnghttp2-dev autoconf libtool libluajit-5.1-de
v pkg-config libhwloc-dev libcmocka-dev libnetfilter-queue-dev libmnl-dev libunwind-dev liblzma-dev liblz4-dev
  96  # Download and install DAQ
  97  cd /tmp
  98  wget https://www.snort.org/downloads/snort/daq-2.0.7.tar.gz
  99  tar -xzvf daq-2.0.7.tar.gz
 100  cd daq-2.0.7
 101  ./configure
 102  make
 103  sudo make install
 104  cd /tmp
 105  wget https://www.snort.org/downloads/snort/snort-2.9.20.tar.gz
 106  tar -xzvf snort-2.9.20.tar.gz
 107  cd snort-2.9.20
 108  ./configure --enable-sourcefire
```

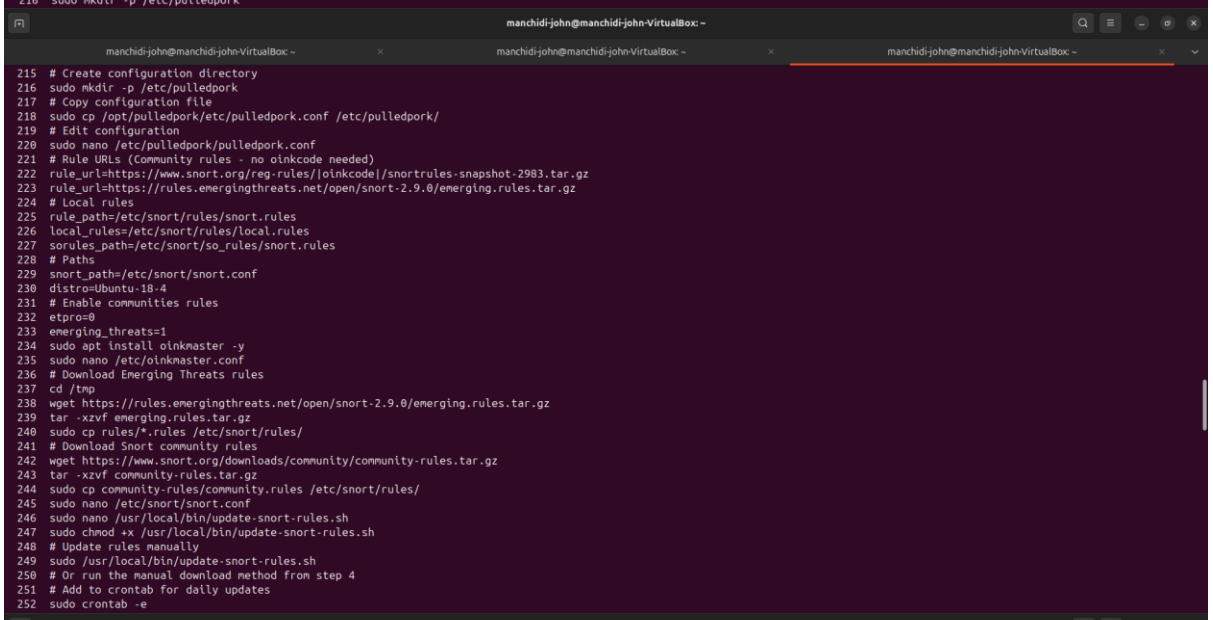
```
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~ | manchidi-john@manchidi-john-VirtualBox: ~ | manchidi-john@manchidi-john-VirtualBox: ~

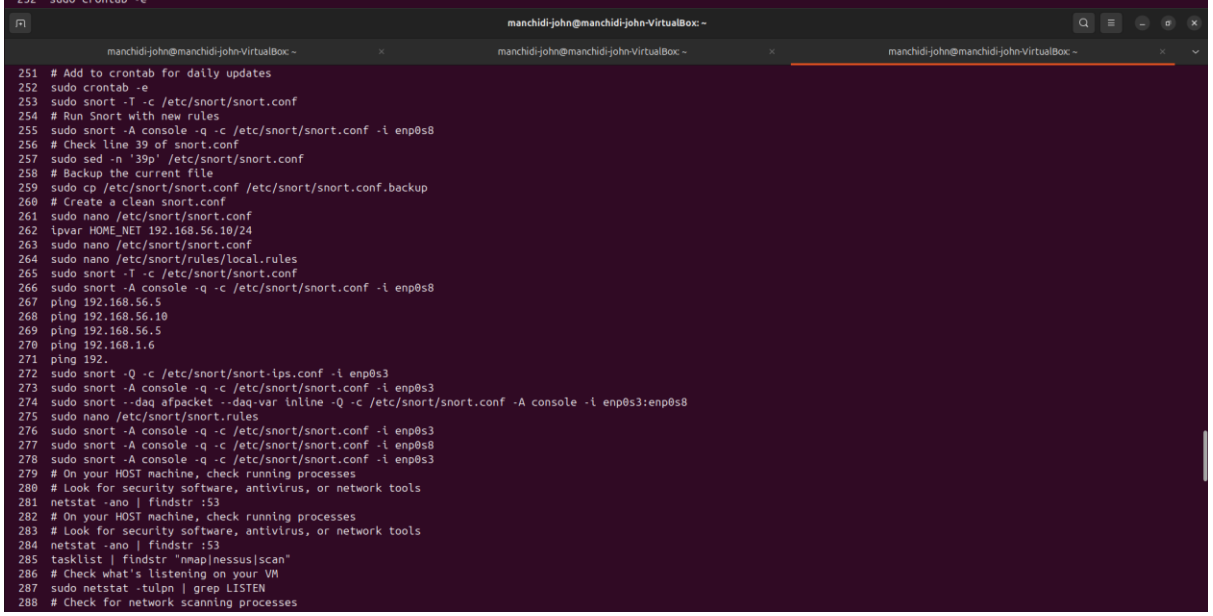
 107  cd snort-2.9.20
 108  ./configure --enable-sourcefire
 109  make
 110  sudo make install
 111  sudo apt install -y snort
 112  snort --version
 113  sudo mkdir -p /etc/snort/rules
 114  sudo mkdir -p /etc/snort/preproc_rules
 115  sudo mkdir -p /usr/local/lib/snort_dynamicrules
 116  sudo mkdir -p /etc/snort/so_rules
 117  sudo mkdir -p /var/log/snort
 118  sudo mkdir -p /var/log/snort/archived_logs
 119  sudo chmod -R 5775 /etc/snort
 120  sudo chmod -R 5775 /var/log/snort
 121  sudo chmod -R 5775 /usr/local/lib/snort_dynamicrules
 122  sudo chown -R snort:snort /etc/snort
 123  sudo chown -R snort:snort /var/log/snort
 124  cd /tmp/snort-2.9.20
 125  sudo cp etc/*.conf* /etc/snort
 126  sudo cp etc/*.map /etc/snort
 127  sudo nano /etc/snort/snort.conf
 128  sudo nano /etc/snort/rules/local.rules
 129  snort -V
 130  sudo snort -T -c /etc/snort/snort.conf
 131  # Create the missing directory
 132  sudo mkdir -p /usr/local/lib/snort_dynamicpreprocessor/
 133  sudo mkdir -p /usr/local/lib/snort_dynamicrules/
 134  # Set proper permissions
 135  sudo chmod -R 755 /usr/local/lib/snort_dynamicpreprocessor/
 136  sudo chmod -R 755 /usr/local/lib/snort_dynamicrules/
 137  sudo nano /etc/snort/snort.conf
 138  # Backup current config
 139  sudo cp /etc/snort/snort.conf /etc/snort/snort.conf.backup
 140  # Get a fresh config (if installed from source)
 141  cd /tmp/snort-2.9.20
 142  sudo cp etc/snort.conf /etc/snort/snort.conf
 143  # Or download a basic config
 144  sudo wget -O /etc/snort/snort.conf https://raw.githubusercontent.com/snort3/snort3/master/etc/snort.conf
```

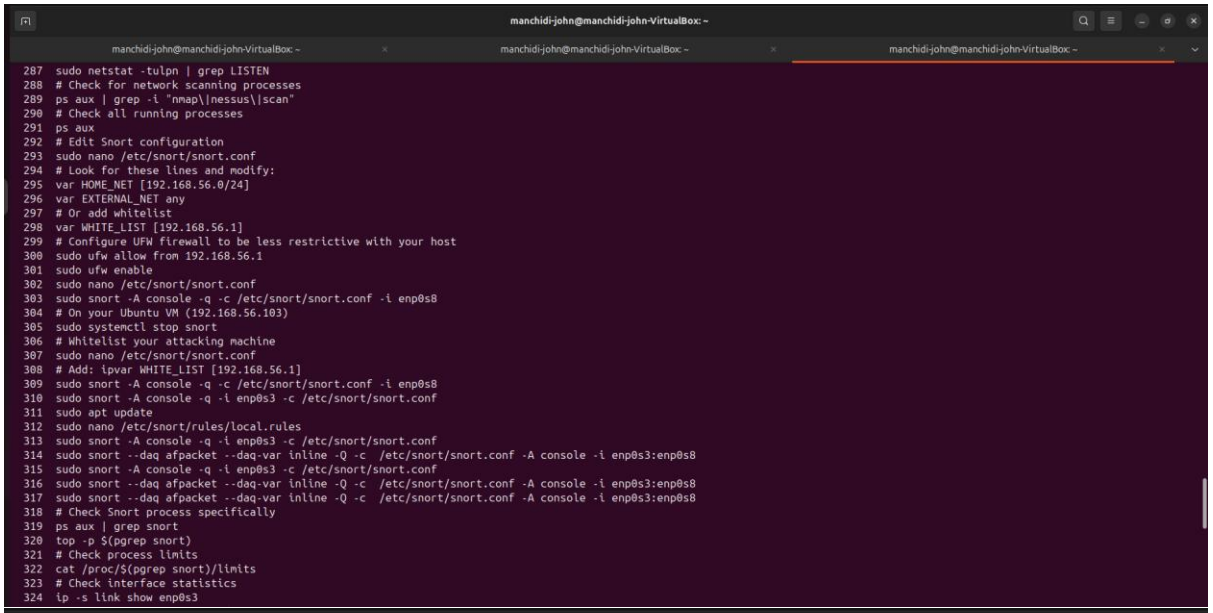
```
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~ | manchidi-john@manchidi-john-VirtualBox: ~ | manchidi-john@manchidi-john-VirtualBox: ~

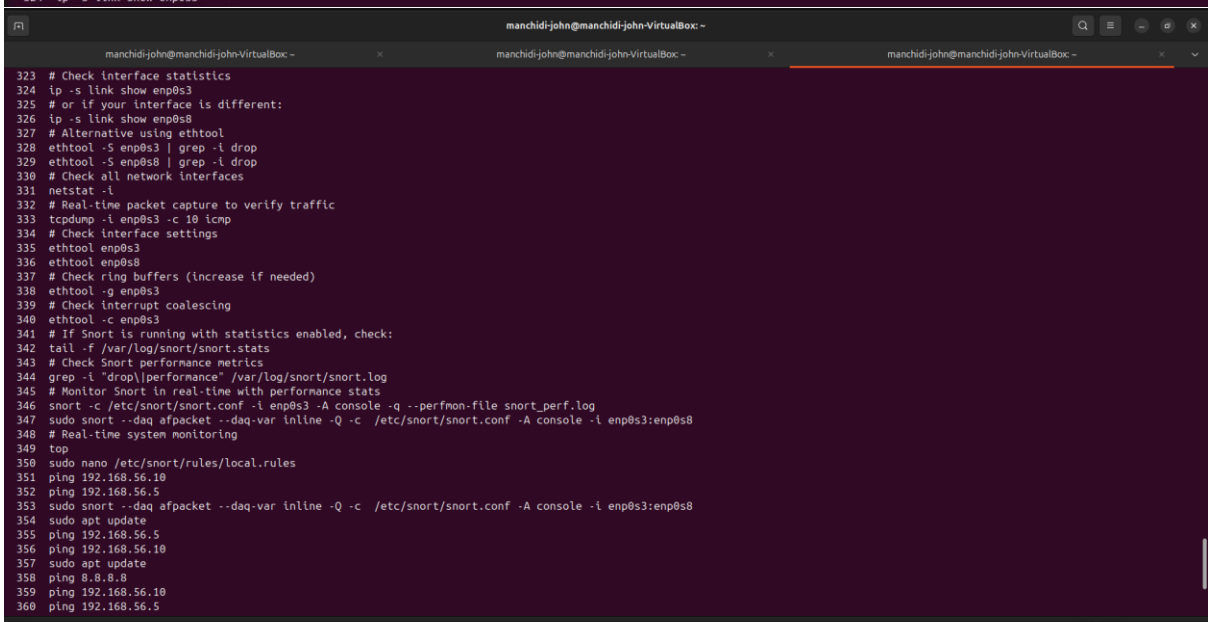
 143  # Or download a basic config
 144  sudo wget -O /etc/snort/snort.conf https://raw.githubusercontent.com/snort3/snort3/master/etc/snort.conf
 145  sudo nano /etc/snort/snort.conf
 146  sudo snort -T -c /etc/snort/snort.conf
 147  # Stop Snort if running
 148  sudo systemctl stop snort 2>/dev/null
 149  # Remove compiled installation
 150  cd /tmp/snort-2.9.20
 151  sudo make uninstall 2>/dev/null
 152  sudo make clean 2>/dev/null
 153  # Remove any package installation
 154  sudo apt remove --purge snort -y
 155  sudo apt autoremove -y
 156  # Remove configuration files and directories
 157  sudo rm -rf /etc/snort
 158  sudo rm -rf /var/log/snort
 159  sudo rm -rf /usr/local/lib/snort*
 160  sudo rm -rf /tmp/snort-2.9.20
 161  sudo rm -rf /tmp/daq-2.0.7
 162  # Clean up any remaining files
 163  sudo updatedb
 164  locate snort | sudo xargs rm -rf 2>/dev/null
 165  # Update system
 166  sudo apt update
 167  sudo apt upgrade -y
 168  # Install Snort from repository
 169  sudo apt install snort -y
 170  # Update system
 171  sudo apt update
 172  sudo apt upgrade -y
 173  # Install Snort from repository
 174  sudo apt install snort -y
 175  sudo apt update
 176  sudo apt upgrade -y
 177  # Remove existing Snort installations
 178  sudo apt remove --purge snort -y
 179  sudo rm -rf /etc/snort
 180  sudo rm -rf /var/log/snort
```

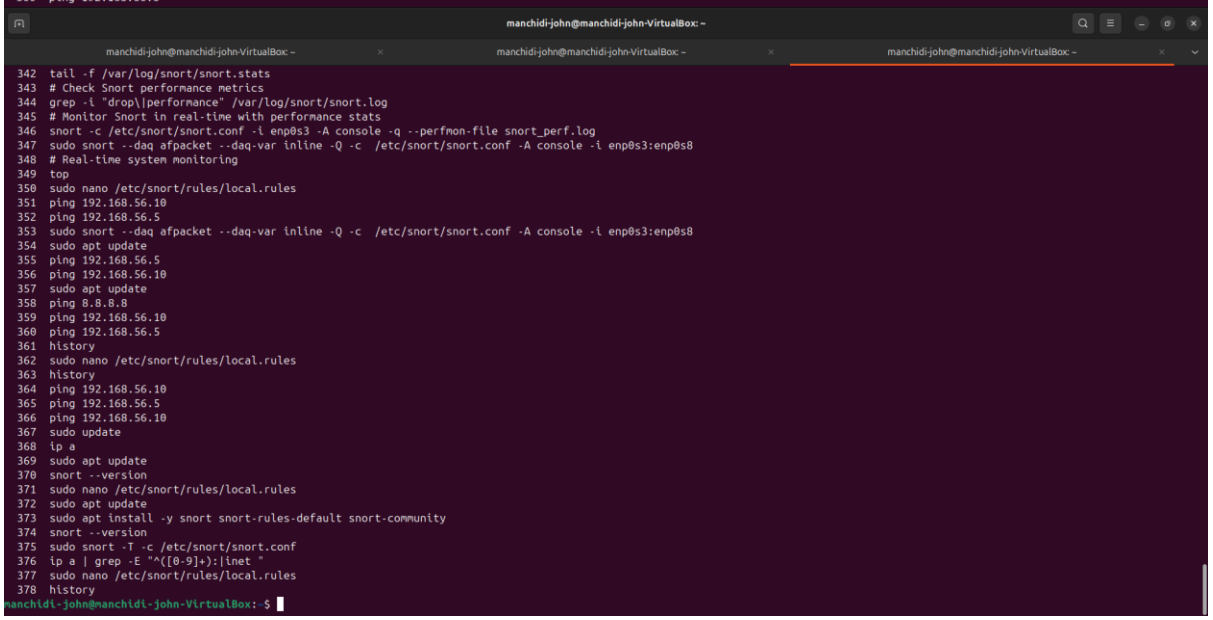

manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
sudo rm -rf /etc/snort
sudo rm -rf /var/log/snort
sudo apt update
sudo apt install snort -y
sudo apt --fix-broken install
Check what's in /etc/snort
ls -la /etc/snort/
Create the missing snort.conf file
sudo mkdir -p /etc/snort/rules
sudo mkdir -p /etc/snort/preproc_rules
sudo touch /etc/snort/snort.conf
sudo nano /etc/snort/snort.conf
sudo touch /etc/snort/classification.config
sudo touch /etc/snort/reference.config
sudo nano /etc/snort/rules/local.rules
sudo dpkg --configure -a
sudo apt --fix-broken install
Test the configuration
sudo snort -T -c /etc/snort/snort.conf
sudo systemctl start snort
sudo systemctl status snort
ip a
sudo nano /etc/snort/rules/local.rules
sudo nano /etc/snort/rules/metasploitable.rules
sudo nano /etc/snort/snort.conf
sudo nano /etc/snort/snort-ips.conf
sudo nano /etc/snort/rules/web-attacks.rules
sudo apt install pulledpork -y
Install dependencies
sudo apt update
sudo apt install -y git perl liblwp-protocol-https-perl libcrypt-ssleay-perl
Clone PulledPork
cd /tmp
git clone https://github.com/shirkdog/pulledpork.git
sudo cp -r pulledpork /opt/
sudo ln -s /opt/pulledpork/pulledpork.pl /usr/local/bin/pulledpork
Create configuration directory
sudo mkdir -p /etc/pulledpork


manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
Create configuration directory
sudo mkdir -p /etc/pulledpork
Copy configuration file
sudo cp /opt/pulledpork/etc/pulledpork.conf /etc/pulledpork/
Edit configuration
sudo nano /etc/pulledpork/pulledpork.conf
Rule URLs (Community rules - no oinkcode needed)
rule_url=https://www.snort.org/reg-rules/|oinkcode|/snortrules-snapshot-2983.tar.gz
rule_url=https://rules.emergingthreats.net/open/snort-2.9.0/emerging.rules.tar.gz
Local rules
rule_path=/etc/snort/rules/snort.rules
local_rules=/etc/snort/rules/local.rules
sorules_path=/etc/snort/so_rules/snort.rules
Paths
snort_path=/etc/snort/snort.conf
distro=Ubuntu-18-4
Enable communities rules
etpro=0
emerging_threats=1
sudo apt install oinkmaster -y
sudo nano /etc/oinkmaster.conf
Download Emerging Threats rules
cd /tmp
wget https://rules.emergingthreats.net/open/snort-2.9.0/emerging.rules.tar.gz
tar -xzvf emerging.rules.tar.gz
sudo cp rules/*.rules /etc/snort/rules/
Download Snort community rules
wget https://www.snort.org/downloads/community/community-rules.tar.gz
tar -xzvf community-rules.tar.gz
sudo cp community-rules/community.rules /etc/snort/rules/
sudo nano /etc/snort/snort.conf
sudo nano /usr/local/bin/update-snort-rules.sh
sudo chmod +x /usr/local/bin/update-snort-rules.sh
Update rules manually
sudo /usr/local/bin/update-snort-rules.sh
Or run the manual download method from step 4
Add to crontab for daily updates
sudo crontab -e


manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
Add to crontab for daily updates
sudo crontab -e
sudo snort -T -c /etc/snort/snort.conf
Run Snort with new rules
sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s8
Check line 39 of snort.conf
sudo sed -n '39p' /etc/snort/snort.conf
Backup the current file
sudo cp /etc/snort/snort.conf /etc/snort/snort.conf.backup
Create a clean snort.conf
sudo nano /etc/snort/snort.conf
ipvar HOME_NET 192.168.56.10/24
sudo nano /etc/snort/snort.conf
sudo nano /etc/snort/rules/local.rules
sudo snort -T -c /etc/snort/snort.conf
sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s8
ping 192.168.56.5
ping 192.168.56.10
ping 192.168.56.5
ping 192.168.1.6
ping 192.
sudo snort -Q -c /etc/snort/snort-ips.conf -i enp0s3
sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s3
sudo snort --daq afpacket --daq-var inline -Q -c /etc/snort/snort.conf -A console -i enp0s3:enp0s8
sudo nano /etc/snort/snort.rules
sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s3
sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s8
sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s3
On your HOST machine, check running processes
Look for security software, antivirus, or network tools
netstat -ano | findstr :53
On your HOST machine, check running processes
Look for security software, antivirus, or network tools
netstat -ano | findstr :53
tasklist | findstr "nmap|nessus|scan"
Check what's listening on your VM
sudo netstat -tulpn | grep LISTEN
Check for network scanning processes

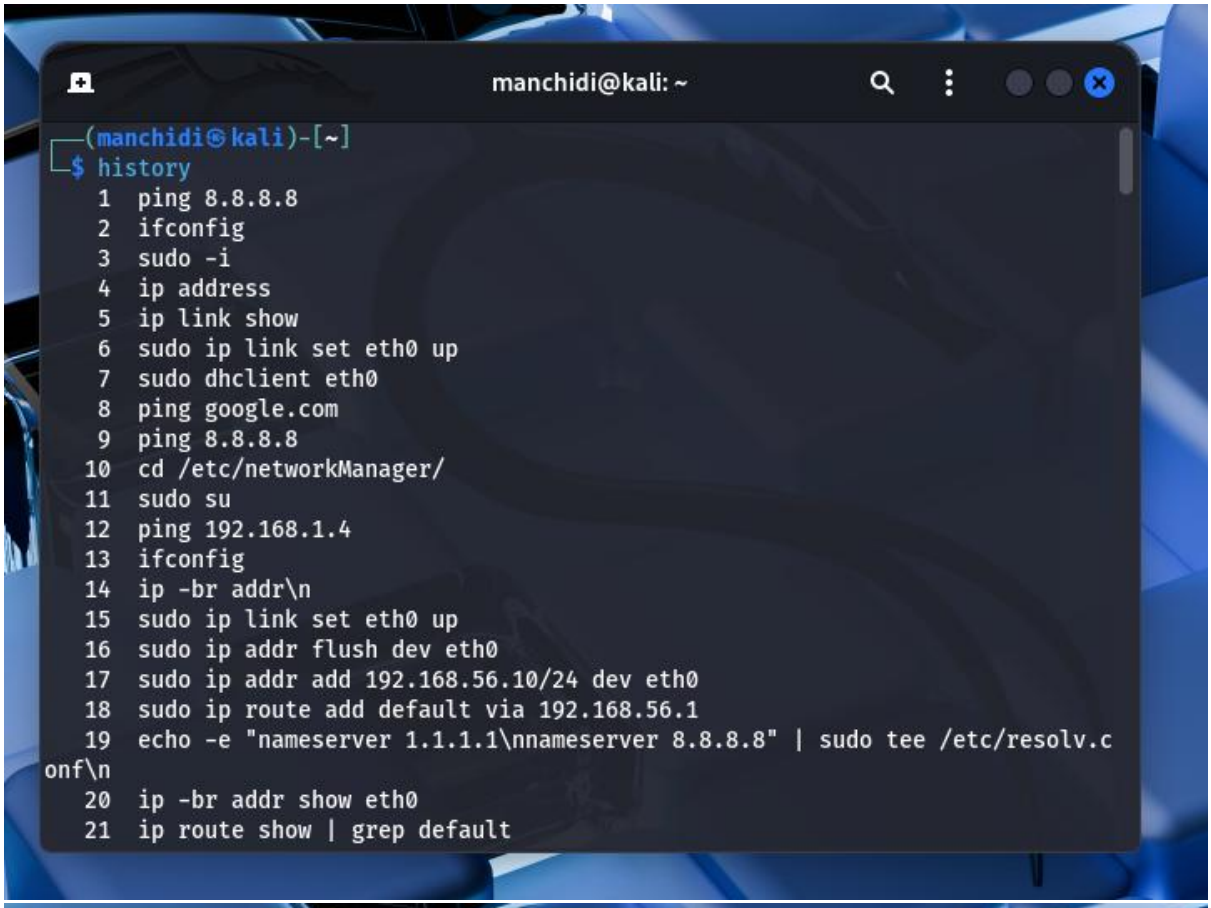

manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
287 sudo netstat -tulpn | grep LISTEN
288 # Check for network scanning processes
289 ps aux | grep -i "nmap\|nessus\|scan"
290 # Check all running processes
291 ps aux
292 # Edit Snort configuration
293 sudo nano /etc/snort/snort.conf
294 # Look for these lines and modify:
295 var HOME_NET [192.168.56.0/24]
296 var EXTERNAL_NET any
297 # Or add whitelist
298 var WHITE_LIST [192.168.56.1]
299 # Configure UFW firewall to be less restrictive with your host
300 sudo ufw allow from 192.168.56.1
301 sudo ufw enable
302 sudo nano /etc/snort/snort.conf
303 sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s8
304 # On your Ubuntu VM (192.168.56.103)
305 sudo systemctl stop snort
306 # Whitelist your attacking machine
307 sudo nano /etc/snort/snort.conf
308 # Add: ipvar WHITE_LIST [192.168.56.1]
309 sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s8
310 sudo snort -A console -q -i enp0s3 -c /etc/snort/snort.conf
311 sudo apt update
312 sudo nano /etc/snort/rules/local.rules
313 sudo snort -A console -q -i enp0s3 -c /etc/snort/snort.conf
314 sudo snort --daq afpacket --daq-var inline -Q -c /etc/snort/snort.conf -A console -i enp0s3:enp0s8
315 sudo snort -A console -q -i enp0s3 -c /etc/snort/snort.conf
316 sudo snort --daq afpacket --daq-var inline -Q -c /etc/snort/snort.conf -A console -i enp0s3:enp0s8
317 sudo snort --daq afpacket --daq-var inline -Q -c /etc/snort/snort.conf -A console -i enp0s3:enp0s8
318 # Check Snort process specifically
319 ps aux | grep snort
320 top -p $(pgrep snort)
321 # Check process limits
322 cat /proc/$(pgrep snort)/limits
323 # Check interface statistics
324 ip -s link show enp0s3

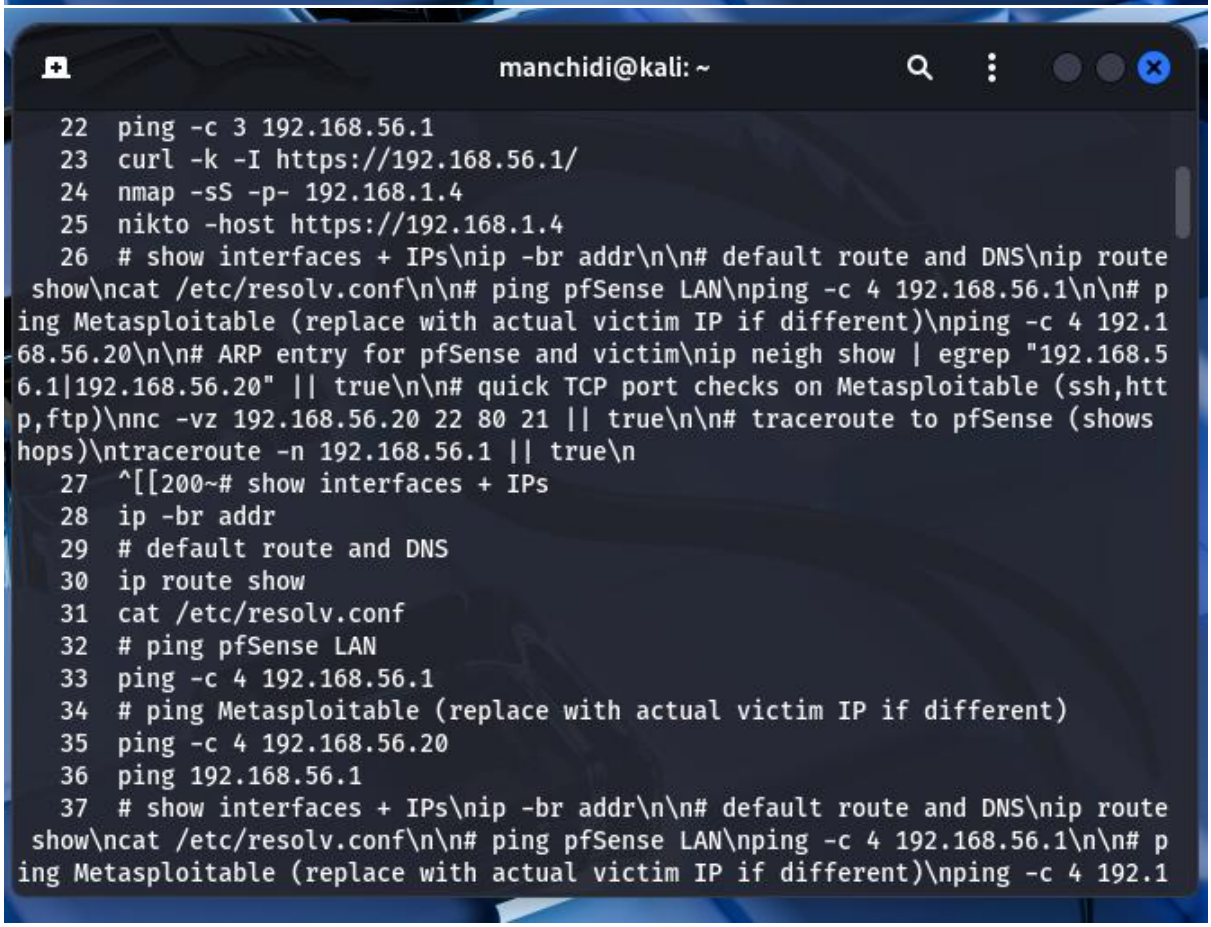

manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
323 # Check interface statistics
324 ip -s link show enp0s3
325 # or if your interface is different:
326 ip -s link show enp0s8
327 # Alternative using ethtool
328 ethtool -S enp0s3 | grep -i drop
329 ethtool -S enp0s8 | grep -i drop
330 # Check all network interfaces
331 netstat -i
332 # Real-time packet capture to verify traffic
333 tcpdump -i enp0s3 -c 10 icmp
334 # Check interface settings
335 ethtool enp0s3
336 ethtool enp0s8
337 # Check ring buffers (increase if needed)
338 ethtool -g enp0s3
339 # Check interrupt coalescing
340 ethtool -c enp0s3
341 # If Snort is running with statistics enabled, check:
342 tail -f /var/log/snort/snort.stats
343 # Check Snort performance metrics
344 grep -i "drop\|performance" /var/log/snort/snort.log
345 # Monitor Snort in real-time with performance stats
346 snort -c /etc/snort/snort.conf -i enp0s3 -A console -q --perfmon-file snort_perf.log
347 sudo snort --daq afpacket --daq-var inline -Q -c /etc/snort/snort.conf -A console -i enp0s3:enp0s8
348 # Real-time system monitoring
349 top
350 sudo nano /etc/snort/rules/local.rules
351 ping 192.168.56.10
352 ping 192.168.56.5
353 sudo snort --daq afpacket --daq-var inline -Q -c /etc/snort/snort.conf -A console -i enp0s3:enp0s8
354 sudo apt update
355 ping 192.168.56.5
356 ping 192.168.56.10
357 sudo apt update
358 ping 8.8.8.8
359 ping 192.168.56.10
360 ping 192.168.56.5

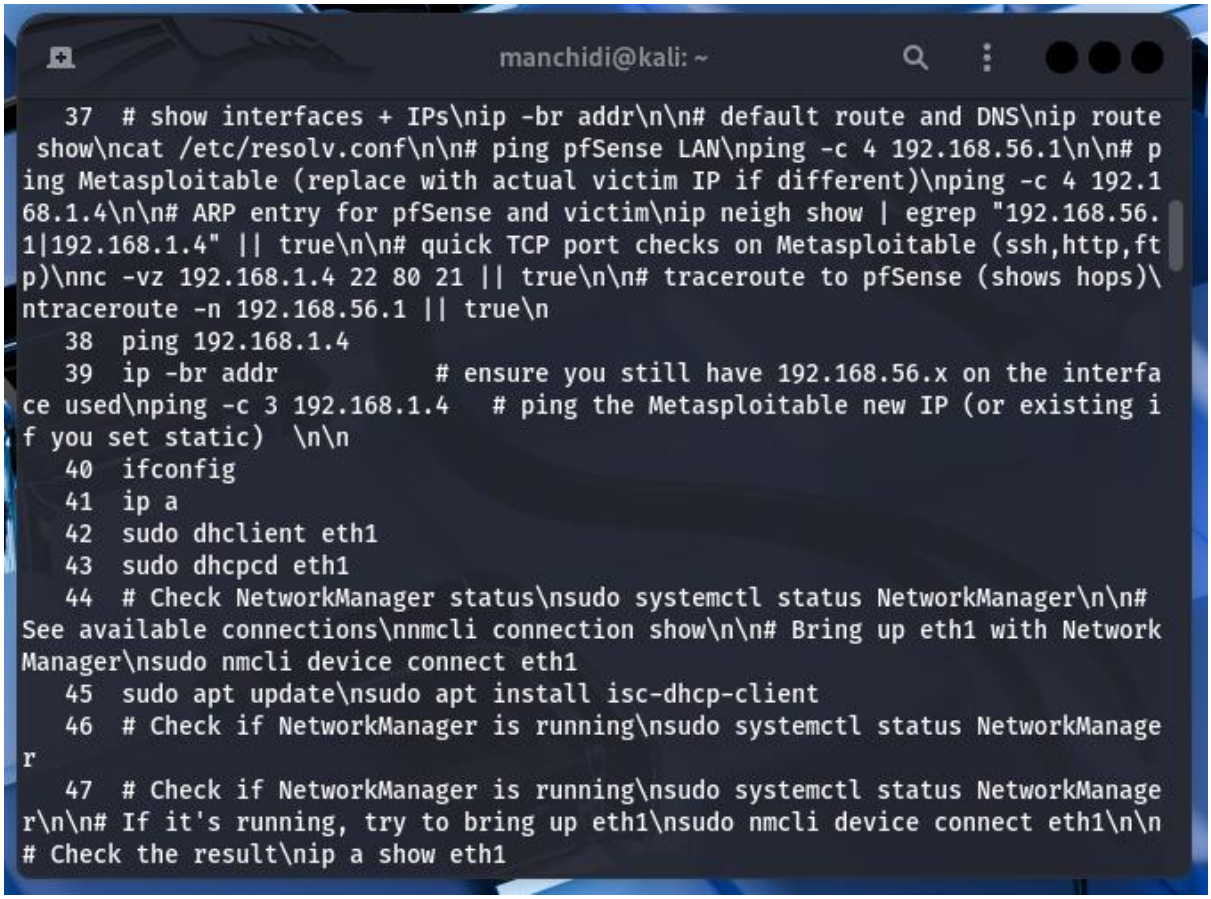

manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
manchidi-john@manchidi-john-VirtualBox: ~
342 tail -f /var/log/snort/snort.stats
343 # Check Snort performance metrics
344 grep -i "drop\|performance" /var/log/snort/snort.log
345 # Monitor Snort in real-time with performance stats
346 snort -c /etc/snort/snort.conf -i enp0s3 -A console -q --perfmon-file snort_perf.log
347 sudo snort --daq afpacket --daq-var inline -Q -c /etc/snort/snort.conf -A console -i enp0s3:enp0s8
348 # Real-time system monitoring
349 top
350 sudo nano /etc/snort/rules/local.rules
351 ping 192.168.56.10
352 ping 192.168.56.5
353 sudo snort --daq afpacket --daq-var inline -Q -c /etc/snort/snort.conf -A console -i enp0s3:enp0s8
354 sudo apt update
355 ping 192.168.56.5
356 ping 192.168.56.10
357 sudo apt update
358 ping 8.8.8.8
359 ping 192.168.56.10
360 ping 192.168.56.5
361 history
362 sudo nano /etc/snort/rules/local.rules
363 history
364 ping 192.168.56.10
365 ping 192.168.56.5
366 ping 192.168.56.10
367 sudo update
368 ip a
369 sudo apt update
370 snort --version
371 sudo nano /etc/snort/rules/local.rules
372 sudo apt update
373 sudo apt install -y snort snort-rules-default snort-community
374 snort --version
375 sudo snort -T -c /etc/snort/snort.conf
376 ip a | grep -E "^([0-9]+):|inet "
377 sudo nano /etc/snort/rules/local.rules
378 history
manchidi-john@manchidi-john-VirtualBox:~$

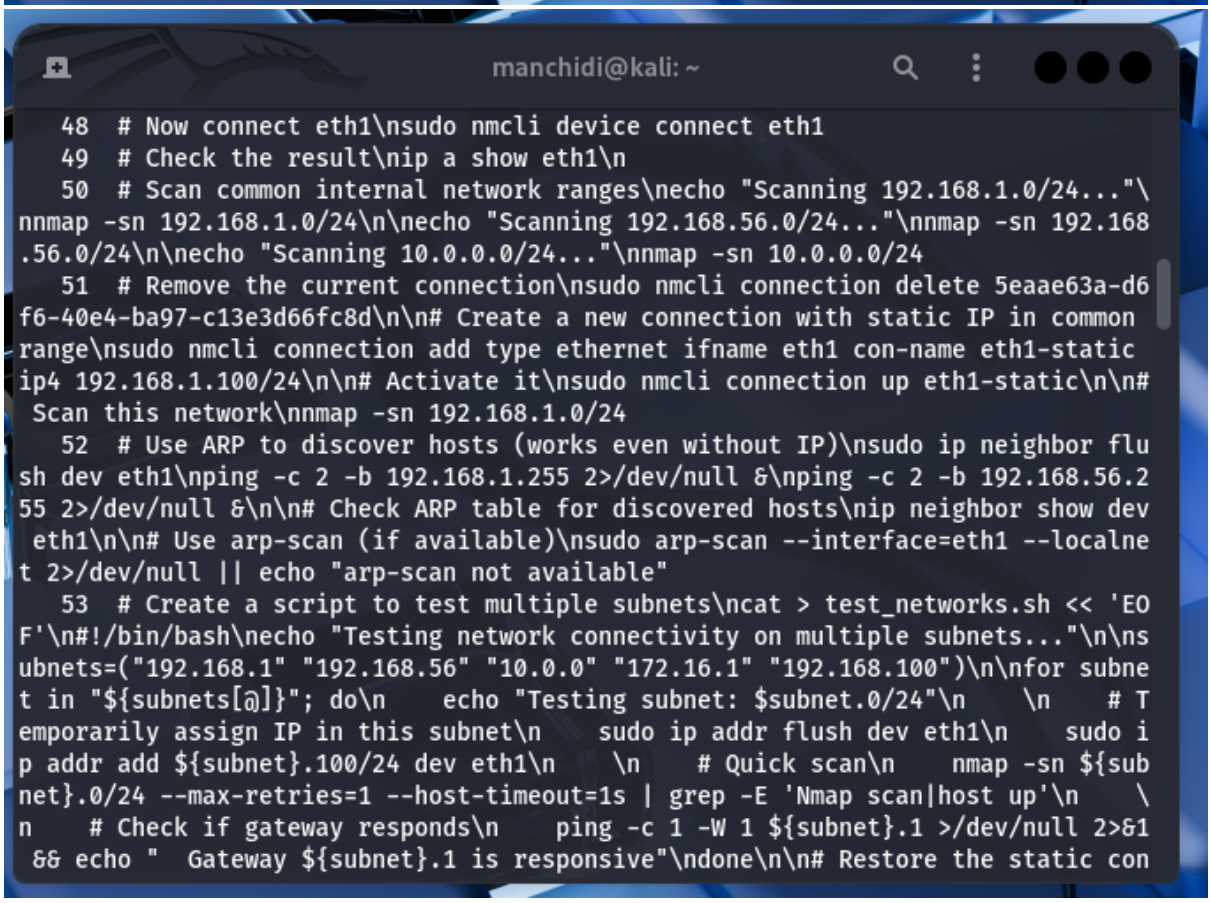
Installing Snort from the default repository and by compiling from source including prerequisites like the DAQ (Data Acquisition) library comes after network setup (checking IP addresses installing network manager and switching the interface to promiscuous mode). Creating the required directories and configuration files (such as /etc/snort/snort.conf and rule files) downloading and extracting Snort rules (both Community Rules and Emerging Threats) and setting up the rule-updating utility Pulledpork take up a lot of effort. Setting important variables like HOME_NET and EXTERNAL_NET in the Snort configuration file is part of the configuration step. Lastly the commands demonstrate testing the configuration with snort -T and executing Snort in console mode (-A console) and inline mode (--daq-var inline) to test its detection capabilities. Additionally network statistics running processes and interface settings are checked using tools like netstat ps aux ethool and tcpdump.

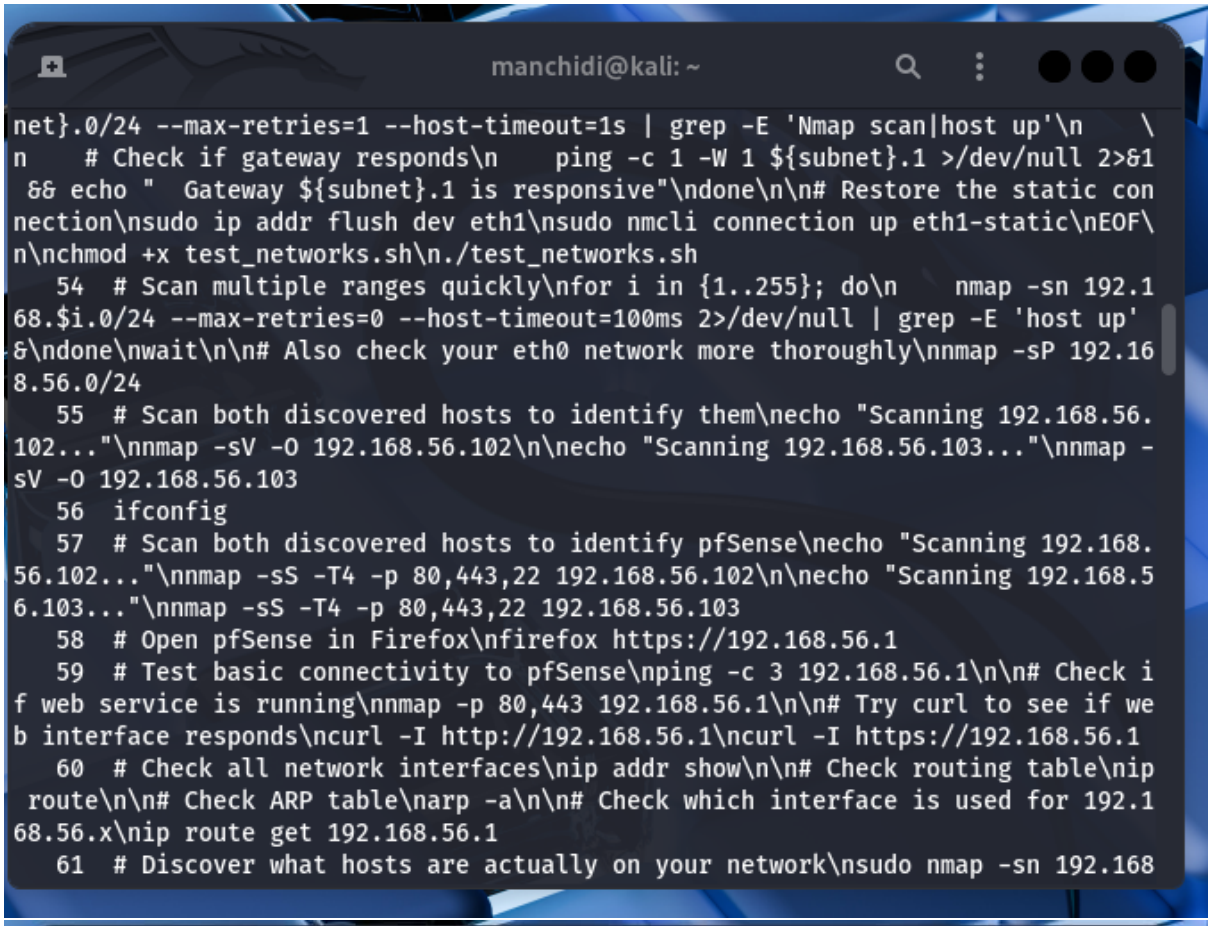
Kali Configuration Summary

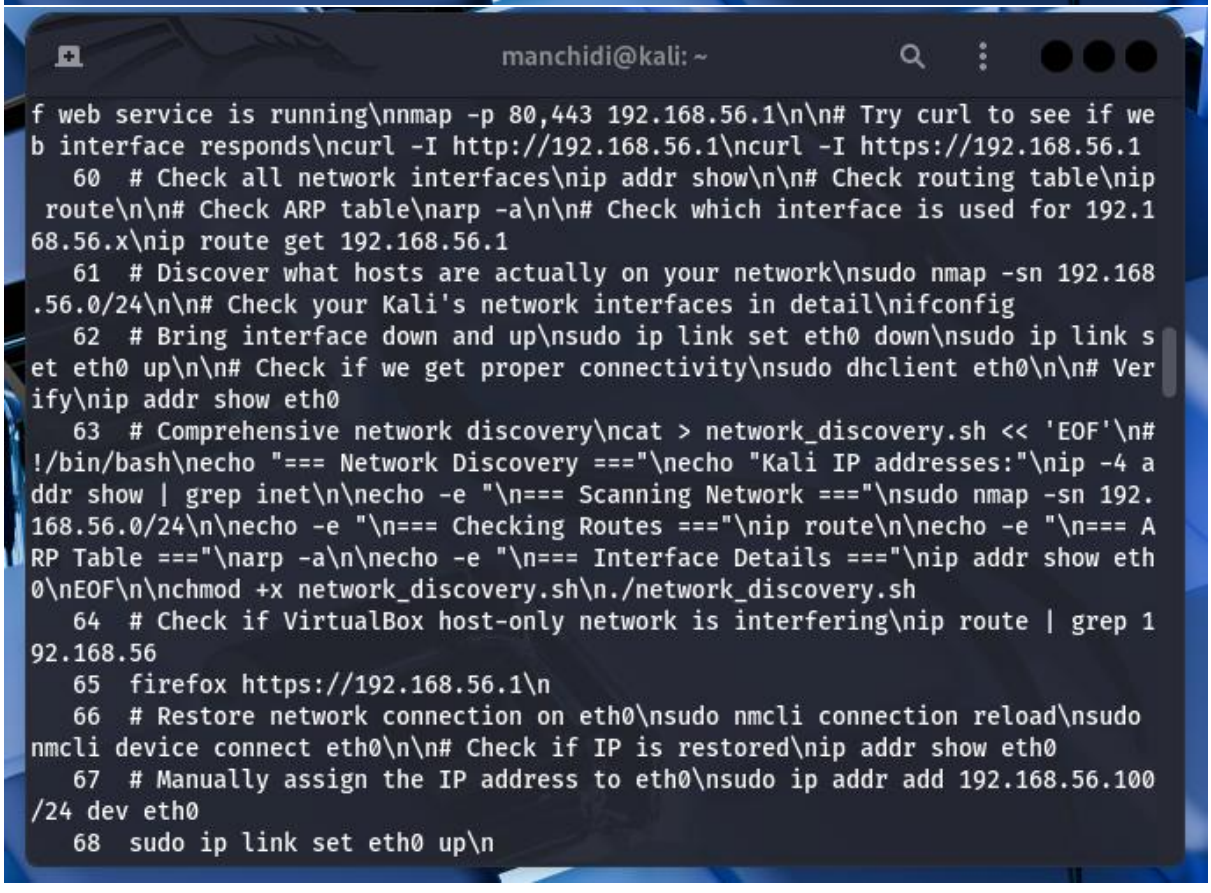

manchidi@kali: ~
(manchidi㉿kali)-[~]
$ history
1 ping 8.8.8.8
2 ifconfig
3 sudo -i
4 ip address
5 ip link show
6 sudo ip link set eth0 up
7 sudo dhclient eth0
8 ping google.com
9 ping 8.8.8.8
10 cd /etc/networkManager/
11 sudo su
12 ping 192.168.1.4
13 ifconfig
14 ip -br addr\n
15 sudo ip link set eth0 up
16 sudo ip addr flush dev eth0
17 sudo ip addr add 192.168.56.10/24 dev eth0
18 sudo ip route add default via 192.168.56.1
19 echo -e "nameserver 1.1.1.1\nnameserver 8.8.8.8" | sudo tee /etc/resolv.c
onf\n
20 ip -br addr show eth0
21 ip route show | grep default

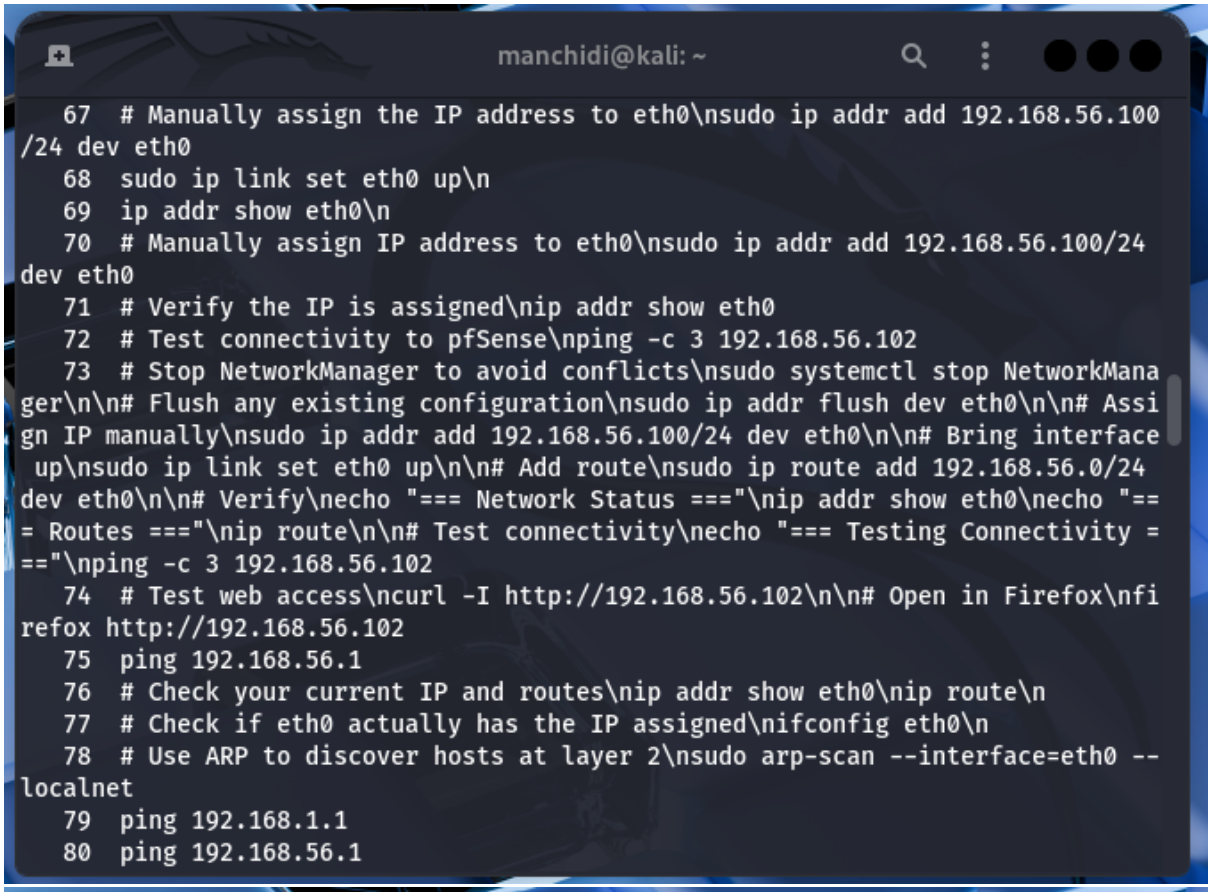

manchidi@kali: ~
22 ping -c 3 192.168.56.1
23 curl -k -I https://192.168.56.1/
24 nmap -sS -p- 192.168.1.4
25 nikto -host https://192.168.1.4
26 # show interfaces + IPs\nip -br addr\n\n# default route and DNS\nip route
show\ncat /etc/resolv.conf\n\n# ping pfSense LAN\nping -c 4 192.168.56.1\n\n# p
ing Metasploitable (replace with actual victim IP if different)\nping -c 4 192.1
68.56.20\n\n# ARP entry for pfSense and victim\nip neigh show | egrep "192.168.5
6.1|192.168.56.20" || true\n\n# quick TCP port checks on Metasploitable (ssh,htt
p,ftp)\nnc -vz 192.168.56.20 22 80 21 || true\n\n# traceroute to pfSense (shows
hops)\ntraceroute -n 192.168.56.1 || true\n
27 ^[[200~# show interfaces + IPs
28 ip -br addr
29 # default route and DNS
30 ip route show
31 cat /etc/resolv.conf
32 # ping pfSense LAN
33 ping -c 4 192.168.56.1
34 # ping Metasploitable (replace with actual victim IP if different)
35 ping -c 4 192.168.56.20
36 ping 192.168.56.1
37 # show interfaces + IPs\nip -br addr\n\n# default route and DNS\nip route
show\ncat /etc/resolv.conf\n\n# ping pfSense LAN\nping -c 4 192.168.56.1\n\n# p
ing Metasploitable (replace with actual victim IP if different)\nping -c 4 192.1

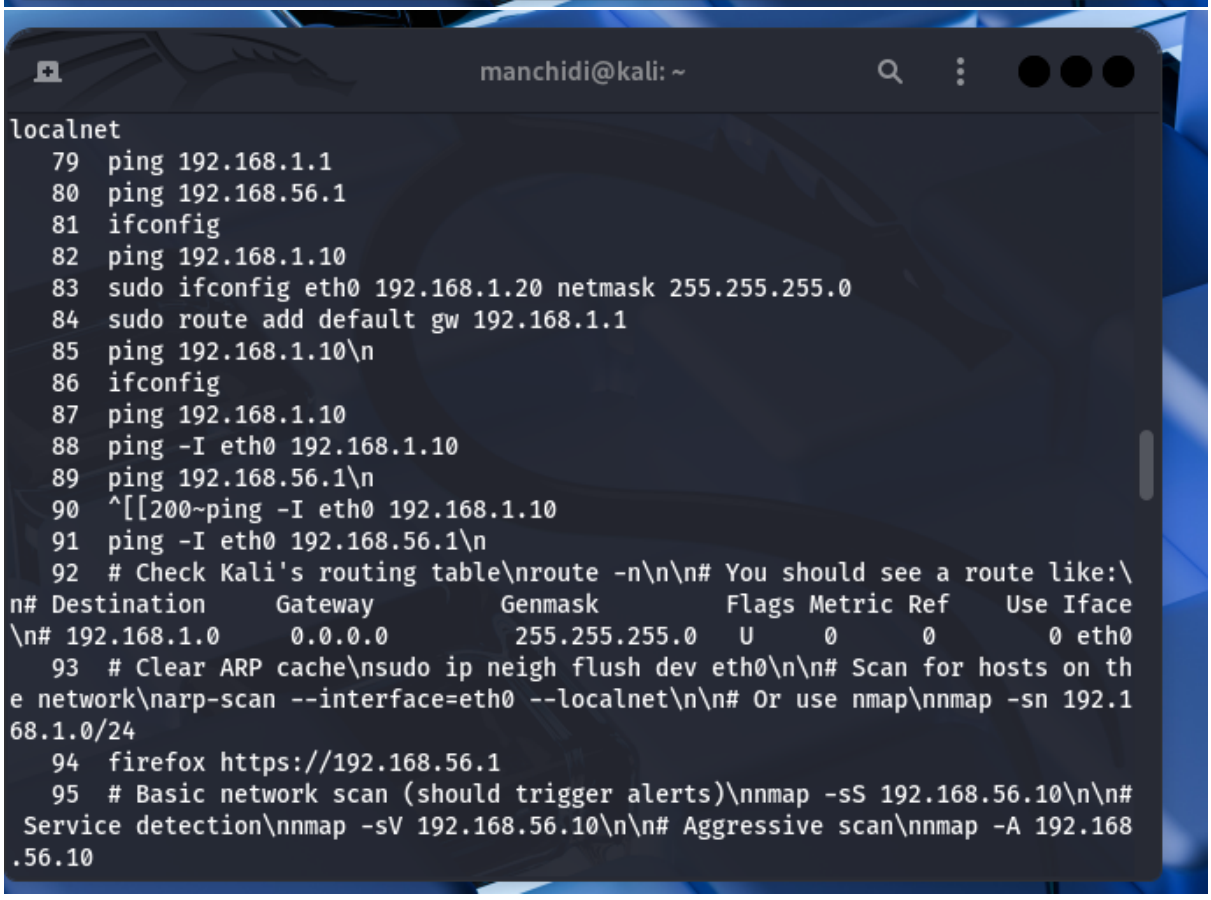

manchidi@kali: ~
37 # show interfaces + IPs\nip -br addr\n\n# default route and DNS\nip route show\ncat /etc/resolv.conf\n\n# ping pfSense LAN\nping -c 4 192.168.56.1\n\n# ping Metasploitable (replace with actual victim IP if different)\nping -c 4 192.168.1.4\n\n# ARP entry for pfSense and victim\nip neigh show | egrep "192.168.56.1|192.168.1.4" || true\n\n# quick TCP port checks on Metasploitable (ssh,http,ftp)\nnc -vz 192.168.1.4 22 80 21 || true\n\n# traceroute to pfSense (shows hops)\ntraceroute -n 192.168.56.1 || true\n
38 ping 192.168.1.4
39 ip -br addr # ensure you still have 192.168.56.x on the interface used\nping -c 3 192.168.1.4 # ping the Metasploitable new IP (or existing if you set static) \n\n
40 ifconfig
41 ip a
42 sudo dhclient eth1
43 sudo dhcpcd eth1
44 # Check NetworkManager status\nsudo systemctl status NetworkManager\n\n# See available connections\nnmcli connection show\n\n# Bring up eth1 with NetworkManager\nsudo nmcli device connect eth1
45 sudo apt update\nsudo apt install isc-dhcp-client
46 # Check if NetworkManager is running\nsudo systemctl status NetworkManager
47 # Check if NetworkManager is running\nsudo systemctl status NetworkManager\n\n# If it's running, try to bring up eth1\nsudo nmcli device connect eth1\n\n# Check the result\nip a show eth1

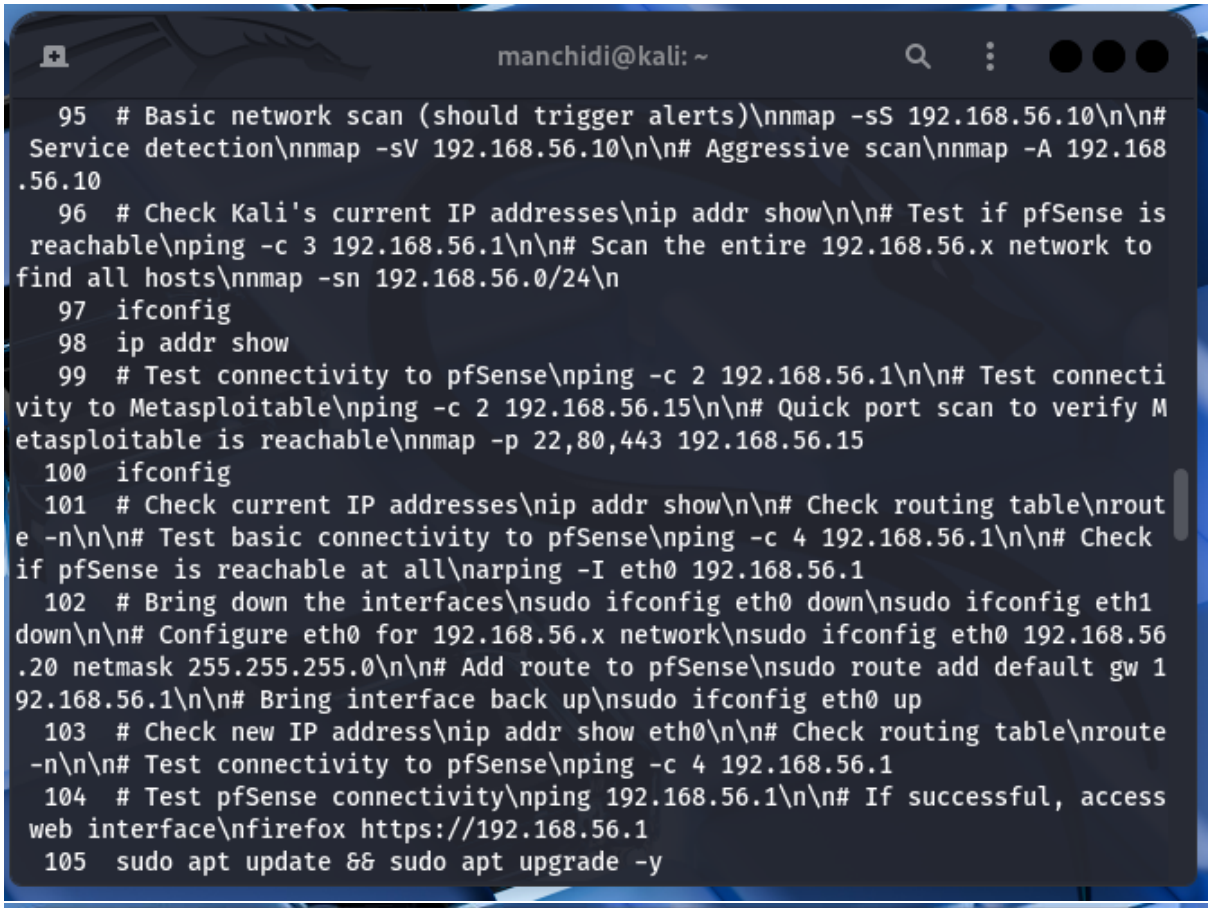

manchidi@kali: ~
48 # Now connect eth1\nsudo nmcli device connect eth1
49 # Check the result\nip a show eth1\n
50 # Scan common internal network ranges\necho "Scanning 192.168.1.0/24..."\nnmap -sn 192.168.1.0/24\n\necho "Scanning 192.168.56.0/24..."\nnmap -sn 192.168.56.0/24\n\necho "Scanning 10.0.0.0/24..."\nnmap -sn 10.0.0.0/24
51 # Remove the current connection\nsudo nmcli connection delete 5eaae63a-d6f6-40e4-ba97-c13e3d66fc8d\n\n# Create a new connection with static IP in common range\nsudo nmcli connection add type ethernet ifname eth1 con-name eth1-static ip4 192.168.1.100/24\n\n# Activate it\nsudo nmcli connection up eth1-static\n\n# Scan this network\nnmap -sn 192.168.1.0/24
52 # Use ARP to discover hosts (works even without IP)\nsudo ip neighbor flush dev eth1\nping -c 2 -b 192.168.1.255 2>/dev/null &\nping -c 2 -b 192.168.56.255 2>/dev/null &\n\n# Check ARP table for discovered hosts\nip neighbor show dev eth1\n\n# Use arp-scan (if available)\nsudo arp-scan --interface=eth1 --localnet 2>/dev/null || echo "arp-scan not available"
53 # Create a script to test multiple subnets\ncat > test_networks.sh << 'EOF'\n#!/bin/bash\necho "Testing network connectivity on multiple subnets..."\n\nsubnets=("192.168.1" "192.168.56" "10.0.0" "172.16.1" "192.168.100")\n\nfor subnet in "${subnets[@]}"; do\n echo "Testing subnet: $subnet.0/24"\n \n # Temporarily assign IP in this subnet\n sudo ip addr flush dev eth1\n sudo ip addr add ${subnet}.100/24 dev eth1\n \n # Quick scan\n nmap -sn ${subnet}.0/24 --max-retries=1 --host-timeout=1s | grep -E 'Nmap scan|host up'\n \n # Check if gateway responds\n ping -c 1 -W 1 ${subnet}.1 >/dev/null 2>&1 && echo " Gateway ${subnet}.1 is responsive"\ndone\n\n# Restore the static con

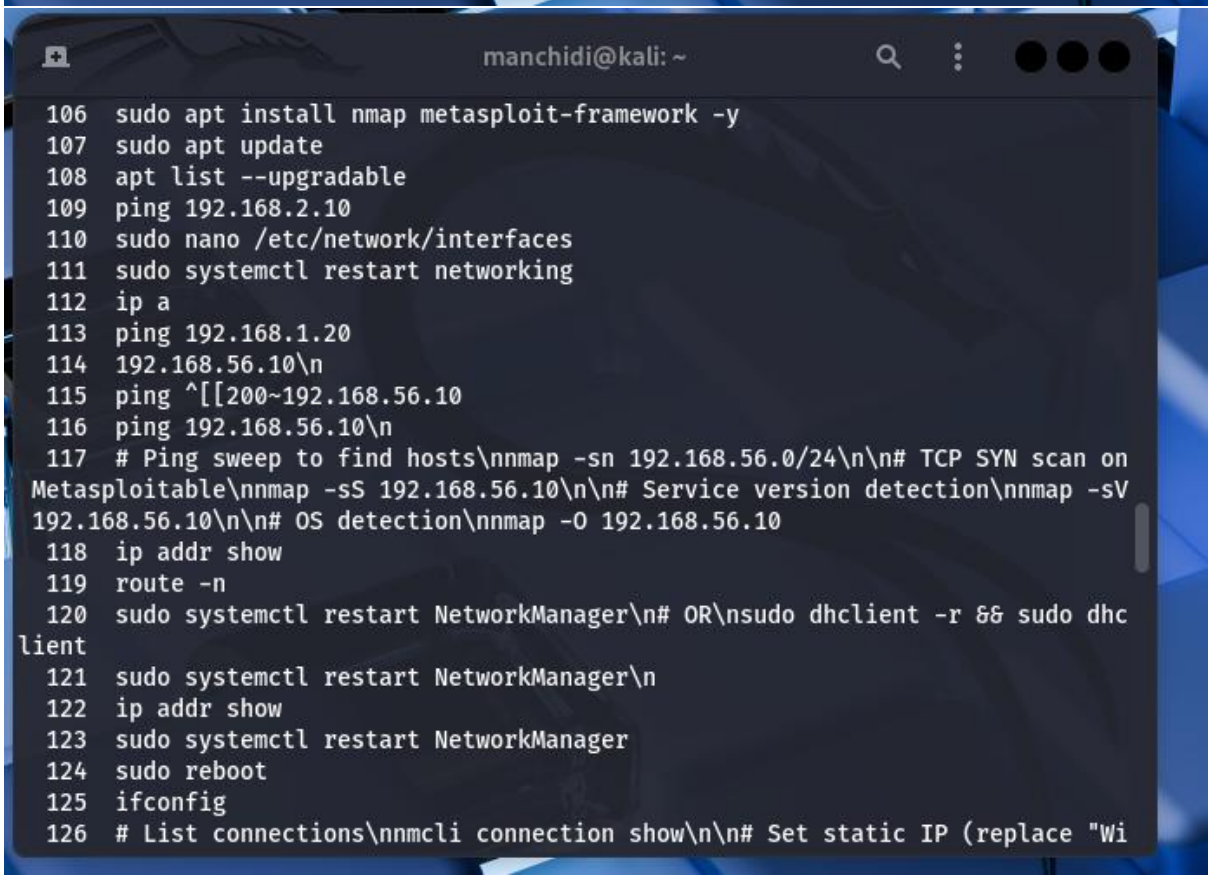

manchidi@kali: ~
net}.0/24 --max-retries=1 --host-timeout=1s | grep -E 'Nmap scan|host up'\n \
n # Check if gateway responds\n ping -c 1 -W 1 ${subnet}.1 >/dev/null 2>&1
&& echo " Gateway ${subnet}.1 is responsive"\ndone\n\n# Restore the static con
nection\nsudo ip addr flush dev eth1\nsudo nmcli connection up eth1-static\nEOF\
n\nchmod +x test_networks.sh\n./test_networks.sh
54 # Scan multiple ranges quickly\nfor i in {1..255}; do\n nmap -sn 192.1
68.$i.0/24 --max-retries=0 --host-timeout=100ms 2>/dev/null | grep -E 'host up'
&\ndone\nwait\n\n# Also check your eth0 network more thoroughly\nnmap -sP 192.16
8.56.0/24
55 # Scan both discovered hosts to identify them\necho "Scanning 192.168.56.
102..."\nnmap -sV -O 192.168.56.102\n\necho "Scanning 192.168.56.103..."\nnmap -
sV -O 192.168.56.103
56 ifconfig
57 # Scan both discovered hosts to identify pfSense\necho "Scanning 192.168.
56.102..."\nnmap -sS -T4 -p 80,443,22 192.168.56.102\n\necho "Scanning 192.168.5
6.103..."\nnmap -sS -T4 -p 80,443,22 192.168.56.103
58 # Open pfSense in Firefox\nfirefox https://192.168.56.1
59 # Test basic connectivity to pfSense\nping -c 3 192.168.56.1\n\n# Check i
f web service is running\nnmap -p 80,443 192.168.56.1\n\n# Try curl to see if we
b interface responds\ncurl -I http://192.168.56.1\ncurl -I https://192.168.56.1
60 # Check all network interfaces\nip addr show\n\n# Check routing table\nip
route\n\n# Check ARP table\narp -a\n\n# Check which interface is used for 192.1
68.56.x\nip route get 192.168.56.1
61 # Discover what hosts are actually on your network\nsudo nmap -sn 192.168

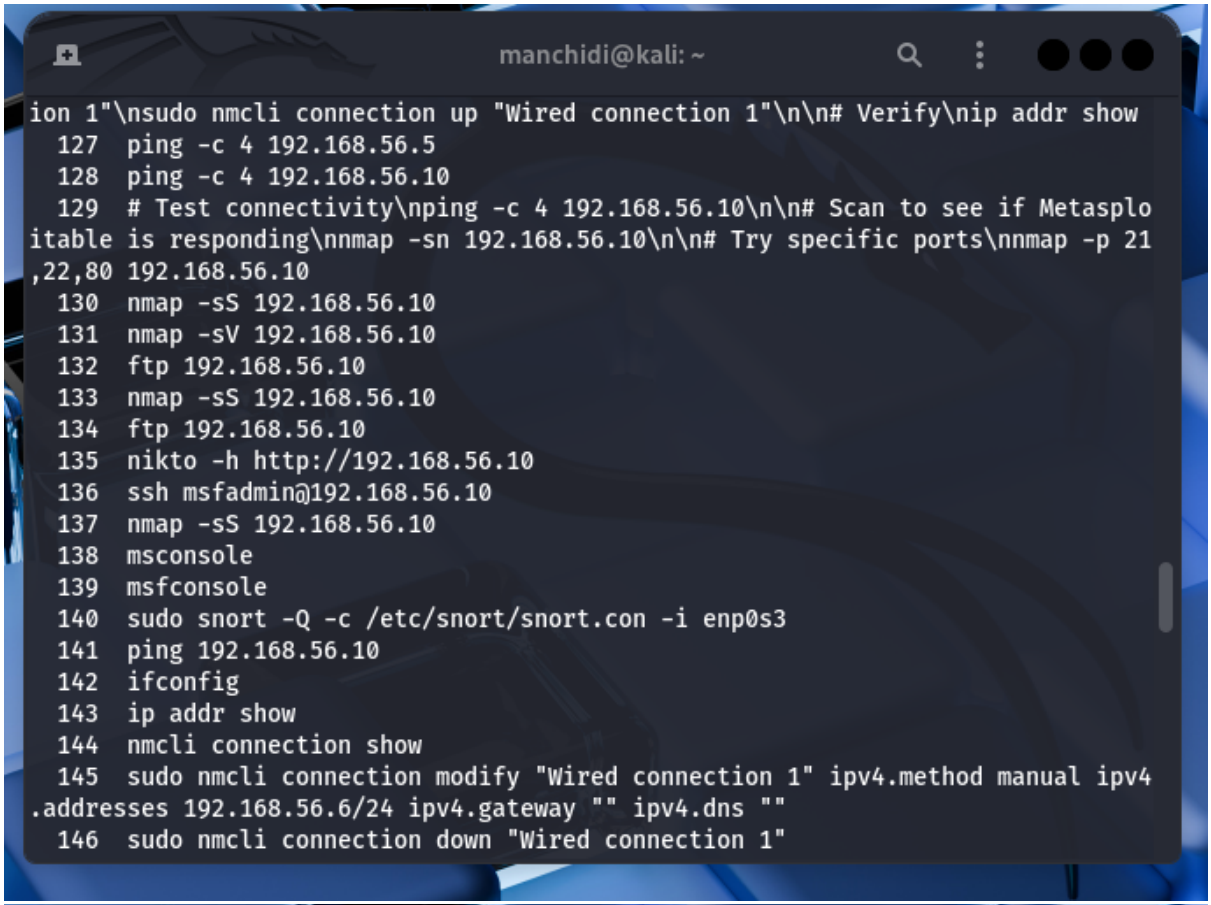

manchidi@kali: ~
f web service is running\nnmap -p 80,443 192.168.56.1\n\n# Try curl to see if we
b interface responds\ncurl -I http://192.168.56.1\ncurl -I https://192.168.56.1
60 # Check all network interfaces\nip addr show\n\n# Check routing table\nip
route\n\n# Check ARP table\narp -a\n\n# Check which interface is used for 192.1
68.56.x\nip route get 192.168.56.1
61 # Discover what hosts are actually on your network\nsudo nmap -sn 192.168
.56.0/24\n\n# Check your Kali's network interfaces in detail\nifconfig
62 # Bring interface down and up\nsudo ip link set eth0 down\nsudo ip link s
et eth0 up\n\n# Check if we get proper connectivity\nsudo dhclient eth0\n\n# Ver
ify\nip addr show eth0
63 # Comprehensive network discovery\ncat > network_discovery.sh << 'EOF'\n#
!/bin/bash\necho "=== Network Discovery ==="\necho "Kali IP addresses:"\nip -4 a
ddr show | grep inet\n\necho -e "\n=== Scanning Network ==="\nsudo nmap -sn 192.
168.56.0/24\n\necho -e "\n=== Checking Routes ==="\nip route\n\necho -e "\n=== A
RP Table ==="\narp -a\n\necho -e "\n=== Interface Details ==="\nip addr show eth
0\nEOF\n\nchmod +x network_discovery.sh\n./network_discovery.sh
64 # Check if VirtualBox host-only network is interfering\nip route | grep 1
92.168.56
65 firefox https://192.168.56.1\n
66 # Restore network connection on eth0\nsudo nmcli connection reload\nsudo
nmcli device connect eth0\n\n# Check if IP is restored\nip addr show eth0
67 # Manually assign the IP address to eth0\nsudo ip addr add 192.168.56.100
/24 dev eth0
68 sudo ip link set eth0 up\n

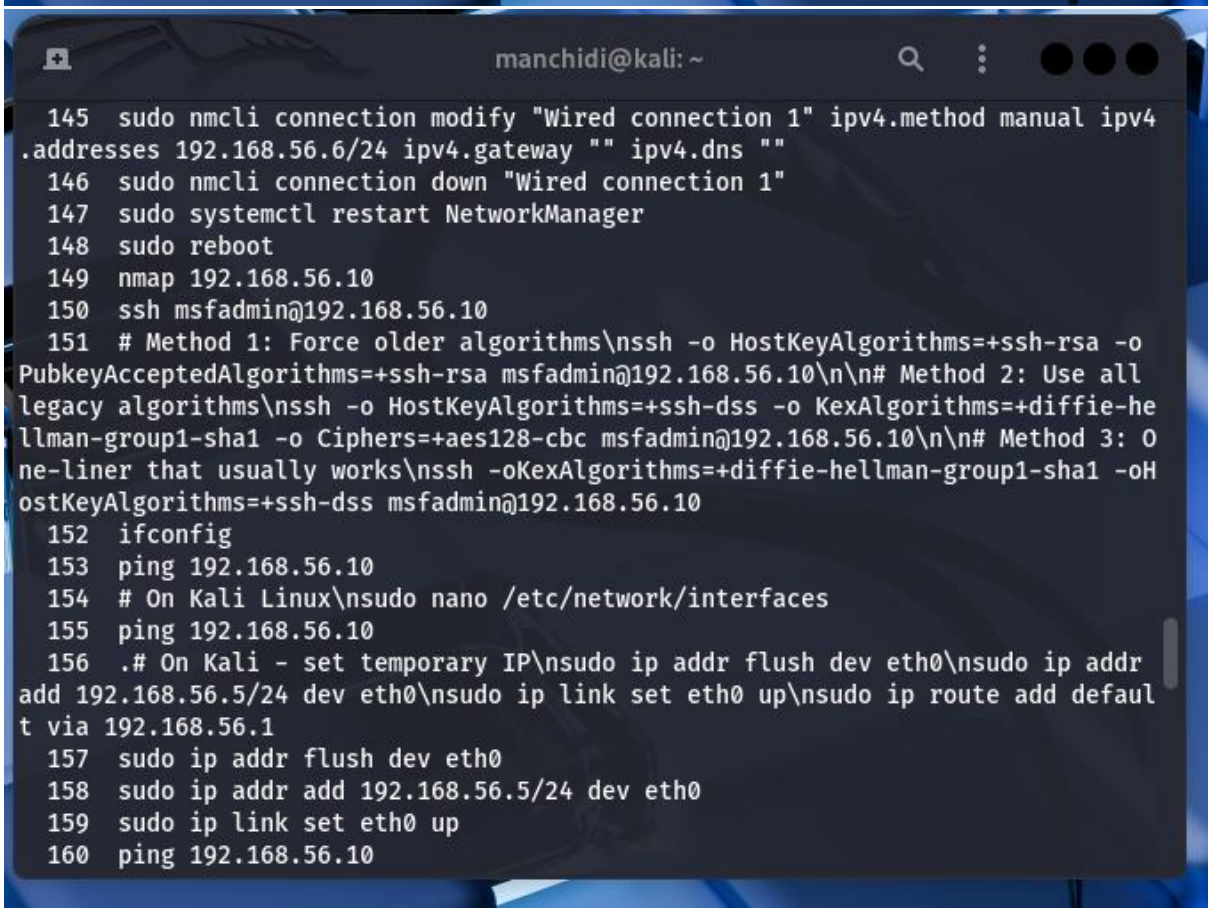

manchidi@kali: ~
67 # Manually assign the IP address to eth0\nsudo ip addr add 192.168.56.100/24 dev eth0
68 sudo ip link set eth0 up\n
69 ip addr show eth0\n
70 # Manually assign IP address to eth0\nsudo ip addr add 192.168.56.100/24 dev eth0
71 # Verify the IP is assigned\nip addr show eth0
72 # Test connectivity to pfSense\nping -c 3 192.168.56.102
73 # Stop NetworkManager to avoid conflicts\nsudo systemctl stop NetworkManager\n\n# Flush any existing configuration\nsudo ip addr flush dev eth0\n\n# Assign IP manually\nsudo ip addr add 192.168.56.100/24 dev eth0\n\n# Bring interface up\nsudo ip link set eth0 up\n\n# Add route\nsudo ip route add 192.168.56.0/24 dev eth0\n\n# Verify\necho "=== Network Status ==="\nip addr show eth0\necho "=== Routes ==="\nip route\n\n# Test connectivity\necho "=== Testing Connectivity ==="\nping -c 3 192.168.56.102
74 # Test web access\ncurl -I http://192.168.56.102\n\n# Open in Firefox\nfirefox http://192.168.56.102
75 ping 192.168.56.1
76 # Check your current IP and routes\nip addr show eth0\nip route\n
77 # Check if eth0 actually has the IP assigned\nifconfig eth0\n
78 # Use ARP to discover hosts at layer 2\nsudo arp-scan --interface=eth0 --localnet
79 ping 192.168.1.1
80 ping 192.168.56.1

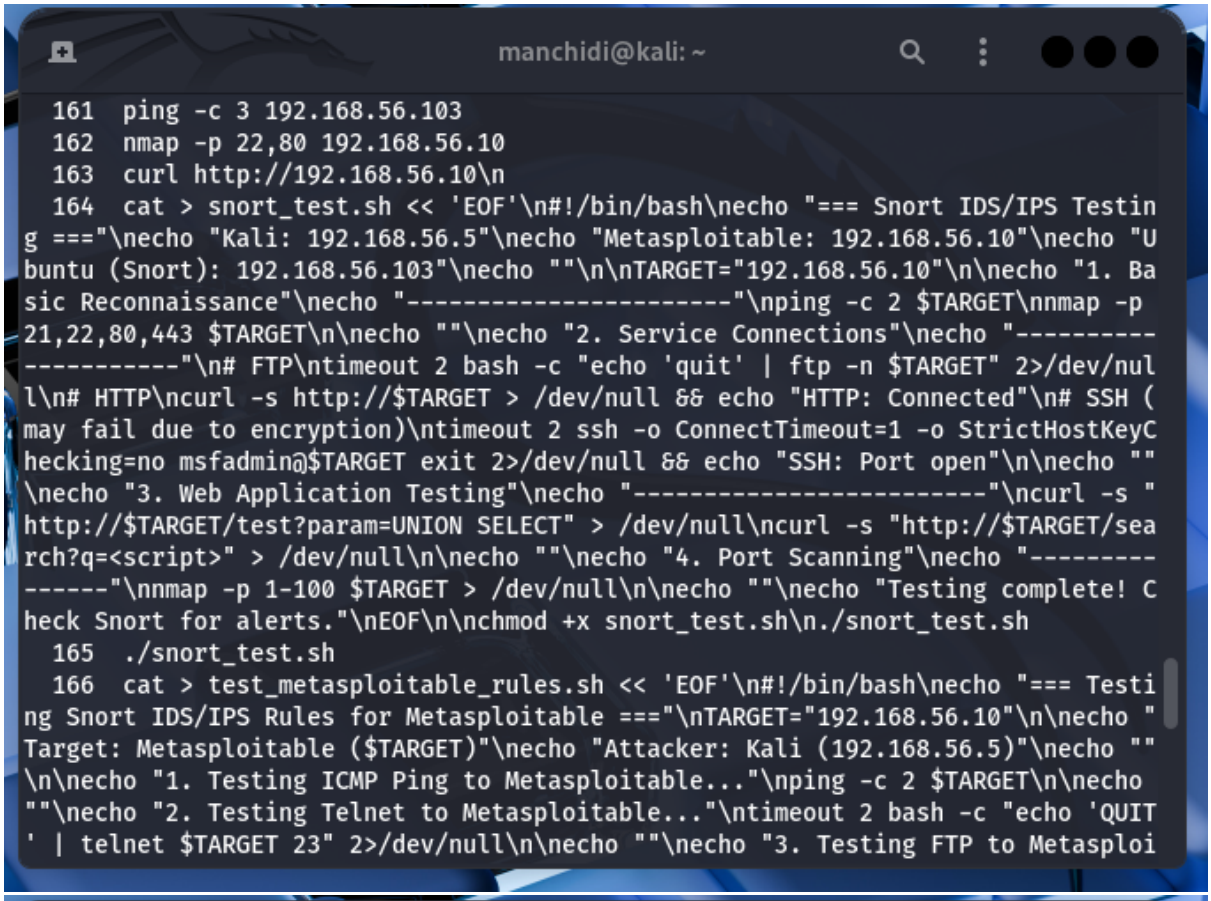

manchidi@kali: ~
localnet
79 ping 192.168.1.1
80 ping 192.168.56.1
81 ifconfig
82 ping 192.168.1.10
83 sudo ifconfig eth0 192.168.1.20 netmask 255.255.255.0
84 sudo route add default gw 192.168.1.1
85 ping 192.168.1.10\n
86 ifconfig
87 ping 192.168.1.10
88 ping -I eth0 192.168.1.10
89 ping 192.168.56.1\n
90 ^[[200~ping -I eth0 192.168.1.10
91 ping -I eth0 192.168.56.1\n
92 # Check Kali's routing table\nroute -n\n\n# You should see a route like:\n# Destination Gateway Genmask Flags Metric Ref Use Iface
\n# 192.168.1.0 0.0.0.0 255.255.255.0 U 0 0 0 eth0
93 # Clear ARP cache\nsudo ip neigh flush dev eth0\n\n# Scan for hosts on the network\narp-scan --interface=eth0 --localnet\n\n# Or use nmap\nnmap -sn 192.168.1.0/24
94 firefox https://192.168.56.1
95 # Basic network scan (should trigger alerts)\nnmap -sS 192.168.56.10\n\n# Service detection\nnmap -sV 192.168.56.10\n\n# Aggressive scan\nnmap -A 192.168.56.10

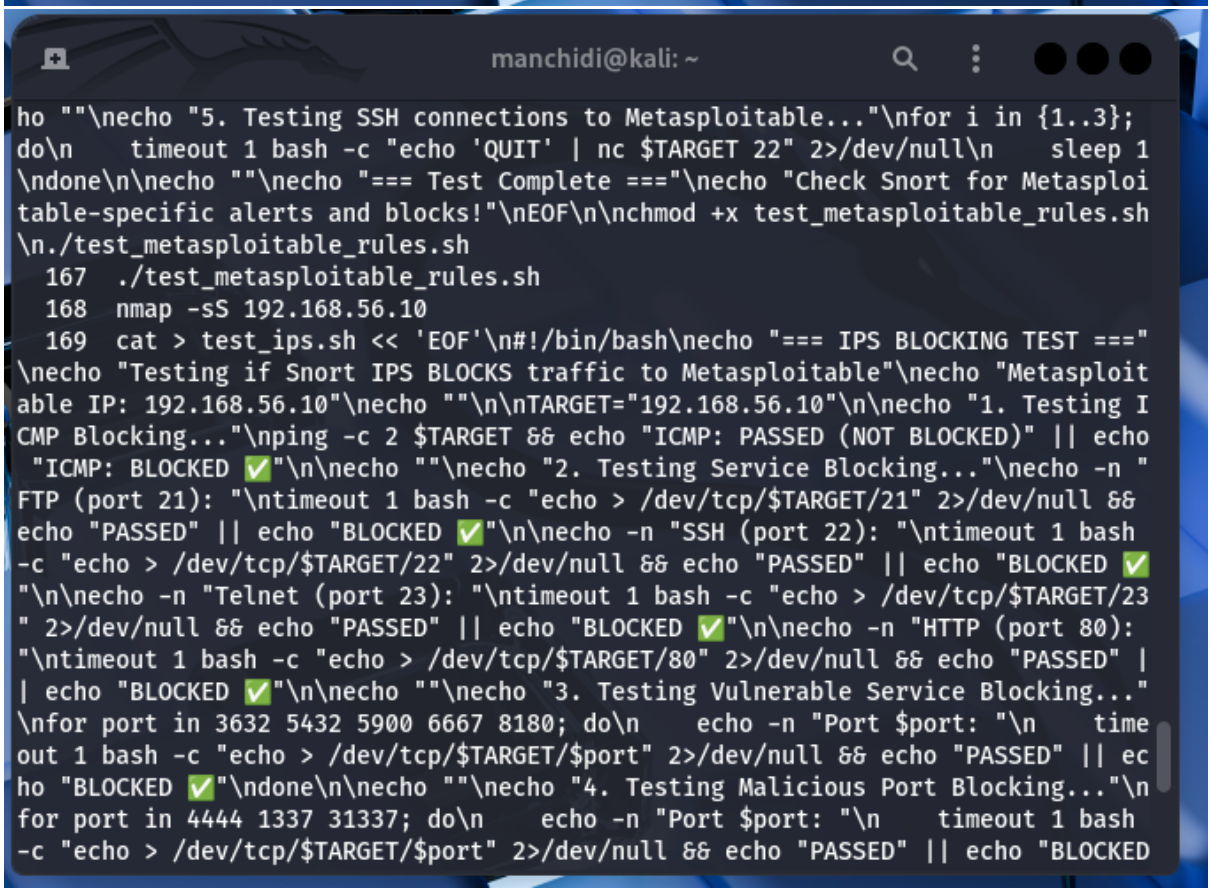

manchidi@kali: ~
95 # Basic network scan (should trigger alerts)\nnmap -sS 192.168.56.10\n\n# Service detection\nnmap -sV 192.168.56.10\n\n# Aggressive scan\nnmap -A 192.168.56.10
96 # Check Kali's current IP addresses\nip addr show\n\n# Test if pfSense is reachable\nping -c 3 192.168.56.1\n\n# Scan the entire 192.168.56.x network to find all hosts\nnmap -sn 192.168.56.0/24\n
97 ifconfig
98 ip addr show
99 # Test connectivity to pfSense\nping -c 2 192.168.56.1\n\n# Test connectivity to Metasploitable\nping -c 2 192.168.56.15\n\n# Quick port scan to verify Metasploitable is reachable\nnmap -p 22,80,443 192.168.56.15
100 ifconfig
101 # Check current IP addresses\nip addr show\n\n# Check routing table\nroute -n\n\n# Test basic connectivity to pfSense\nping -c 4 192.168.56.1\n\n# Check if pfSense is reachable at all\narping -I eth0 192.168.56.1
102 # Bring down the interfaces\nsudo ifconfig eth0 down\nsudo ifconfig eth1 down\n\n# Configure eth0 for 192.168.56.x network\nsudo ifconfig eth0 192.168.56.20 netmask 255.255.255.0\n\n# Add route to pfSense\nsudo route add default gw 192.168.56.1\n\n# Bring interface back up\nsudo ifconfig eth0 up
103 # Check new IP address\nip addr show eth0\n\n# Check routing table\nroute -n\n\n# Test connectivity to pfSense\nping -c 4 192.168.56.1
104 # Test pfSense connectivity\nping 192.168.56.1\n\n# If successful, access web interface\nfirefox https://192.168.56.1
105 sudo apt update && sudo apt upgrade -y

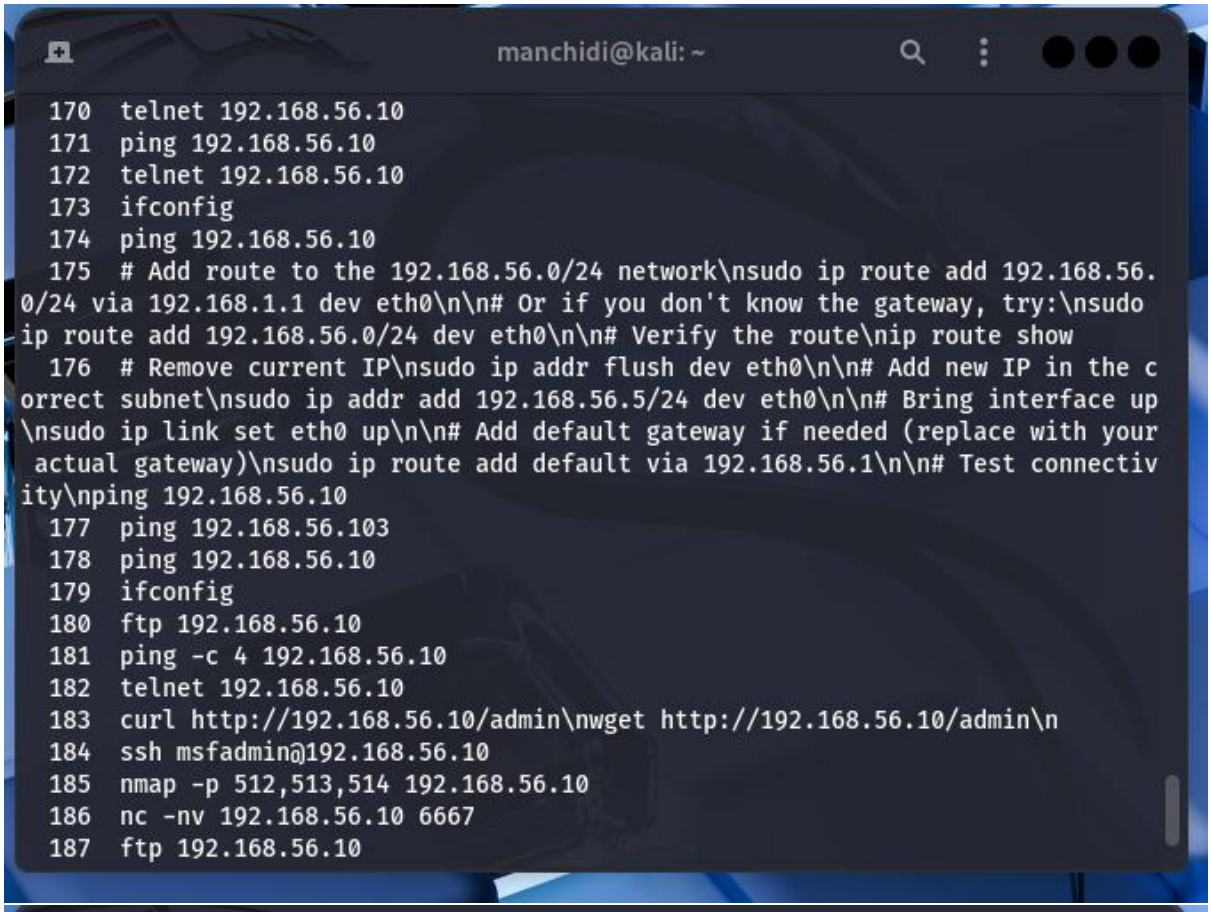

manchidi@kali: ~
106 sudo apt install nmap metasploit-framework -y
107 sudo apt update
108 apt list --upgradable
109 ping 192.168.2.10
110 sudo nano /etc/network/interfaces
111 sudo systemctl restart networking
112 ip a
113 ping 192.168.1.20
114 192.168.56.10\n
115 ping ^[[200~192.168.56.10
116 ping 192.168.56.10\n
117 # Ping sweep to find hosts\nnmap -sn 192.168.56.0/24\n\n# TCP SYN scan on Metasploitable\nnmap -sS 192.168.56.10\n\n# Service version detection\nnmap -sV 192.168.56.10\n\n# OS detection\nnmap -O 192.168.56.10
118 ip addr show
119 route -n
120 sudo systemctl restart NetworkManager\n# OR\nsudo dhclient -r && sudo dhclient
121 sudo systemctl restart NetworkManager\n
122 ip addr show
123 sudo systemctl restart NetworkManager
124 sudo reboot
125 ifconfig
126 # List connections\nnmcli connection show\n\n# Set static IP (replace "Wi

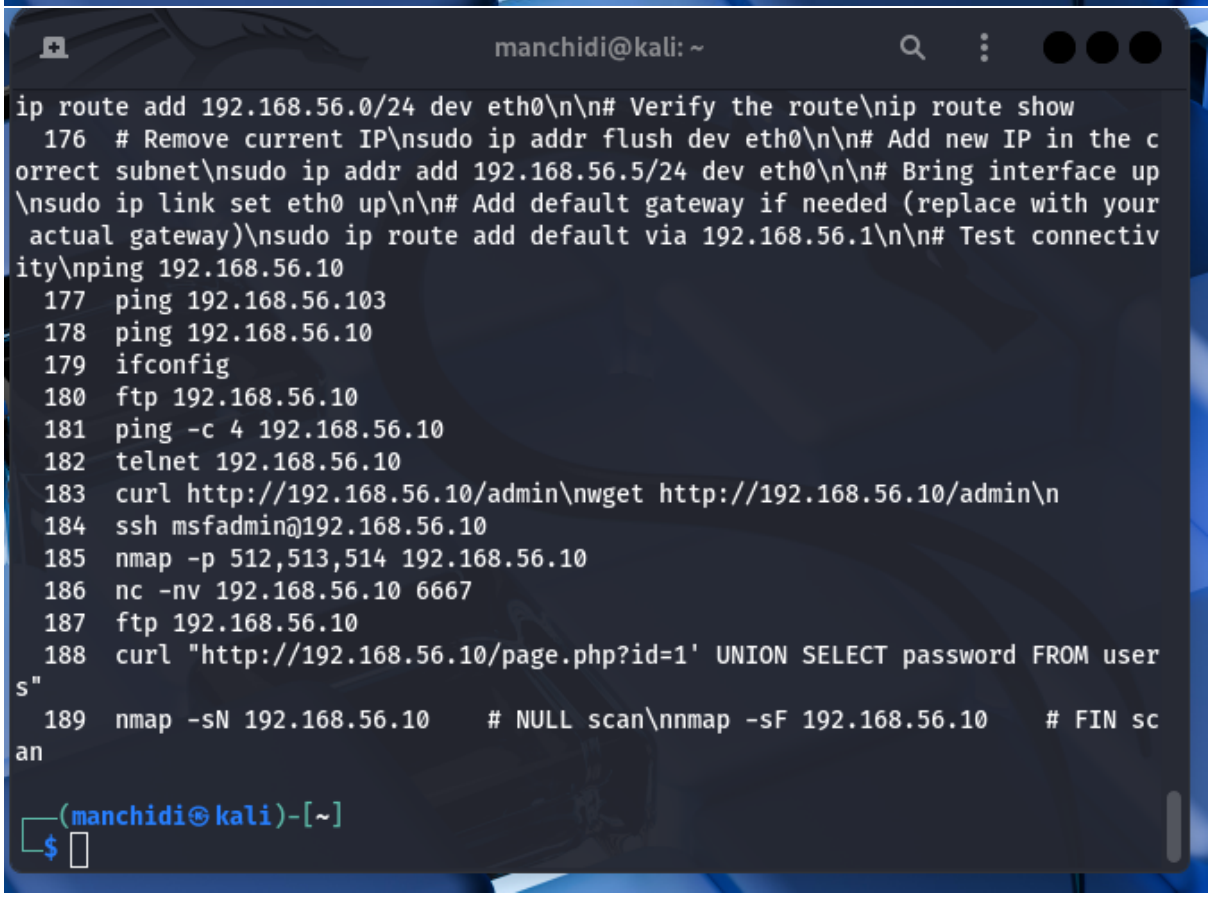

manchidi@kali: ~
ion 1"\nsudo nmcli connection up "Wired connection 1"\n\n# Verify\nip addr show
127 ping -c 4 192.168.56.5
128 ping -c 4 192.168.56.10
129 # Test connectivity\nping -c 4 192.168.56.10\n\n# Scan to see if Metasplo
itable is responding\nnmap -sn 192.168.56.10\n\n# Try specific ports\nnmap -p 21
,22,80 192.168.56.10
130 nmap -sS 192.168.56.10
131 nmap -sV 192.168.56.10
132 ftp 192.168.56.10
133 nmap -sS 192.168.56.10
134 ftp 192.168.56.10
135 nikto -h http://192.168.56.10
136 ssh msfadmin@192.168.56.10
137 nmap -sS 192.168.56.10
138 msconsole
139 msfconsole
140 sudo snort -Q -c /etc/snort/snort.con -i enp0s3
141 ping 192.168.56.10
142 ifconfig
143 ip addr show
144 nmcli connection show
145 sudo nmcli connection modify "Wired connection 1" ipv4.method manual ipv4
.addresses 192.168.56.6/24 ipv4.gateway "" ipv4.dns ""
146 sudo nmcli connection down "Wired connection 1"


manchidi@kali: ~
145 sudo nmcli connection modify "Wired connection 1" ipv4.method manual ipv4
.addresses 192.168.56.6/24 ipv4.gateway "" ipv4.dns ""
146 sudo nmcli connection down "Wired connection 1"
147 sudo systemctl restart NetworkManager
148 sudo reboot
149 nmap 192.168.56.10
150 ssh msfadmin@192.168.56.10
151 # Method 1: Force older algorithms\nssh -o HostKeyAlgorithms=+ssh-rsa -o
PubkeyAcceptedAlgorithms=+ssh-rsa msfadmin@192.168.56.10\n\n# Method 2: Use all
legacy algorithms\nssh -o HostKeyAlgorithms=+ssh-dss -o KexAlgorithms=+diffie-he
llman-group1-sha1 -o Ciphers=+aes128-cbc msfadmin@192.168.56.10\n\n# Method 3: O
ne-liner that usually works\nssh -oKexAlgorithms=+diffie-hellman-group1-sha1 -oH
ostKeyAlgorithms=+ssh-dss msfadmin@192.168.56.10
152 ifconfig
153 ping 192.168.56.10
154 # On Kali Linux\nsudo nano /etc/network/interfaces
155 ping 192.168.56.10
156 .# On Kali - set temporary IP\nsudo ip addr flush dev eth0\nsudo ip addr
add 192.168.56.5/24 dev eth0\nsudo ip link set eth0 up\nsudo ip route add defaul
t via 192.168.56.1
157 sudo ip addr flush dev eth0
158 sudo ip addr add 192.168.56.5/24 dev eth0
159 sudo ip link set eth0 up
160 ping 192.168.56.10


manchidi@kali: ~
161 ping -c 3 192.168.56.103
162 nmap -p 22,80 192.168.56.10
163 curl http://192.168.56.10\n
164 cat > snort_test.sh << 'EOF'\n#!/bin/bash\necho "=== Snort IDS/IPS Testing ==="\necho "Kali: 192.168.56.5"\necho "Metasploitable: 192.168.56.10"\necho "Ubuntu (Snort): 192.168.56.103"\necho ""\n\nTARGET="192.168.56.10"\n\necho "1. Basic Reconnaissance"\necho "----------"\nping -c 2 $TARGET\nnmap -p 21,22,80,443 $TARGET\n\necho ""\necho "2. Service Connections"\necho "----------"\n# FTP\ntimeout 2 bash -c "echo 'quit' | ftp -n $TARGET" 2>/dev/null\n# HTTP\ncurl -s http://$TARGET > /dev/null && echo "HTTP: Connected"\n# SSH (may fail due to encryption)\ntimeout 2 ssh -o ConnectTimeout=1 -o StrictHostKeyChecking=no msfadmin@$TARGET exit 2>/dev/null && echo "SSH: Port open"\n\necho ""\necho "3. Web Application Testing"\necho "----------"\ncurl -s "http://$TARGET/test?param=UNION SELECT" > /dev/null\ncurl -s "http://$TARGET/search?q=<script>" > /dev/null\n\necho ""\necho "4. Port Scanning"\necho "----------"\nnmap -p 1-100 $TARGET > /dev/null\n\necho ""\necho "Testing complete! Check Snort for alerts."\nEOF\n\nchmod +x snort_test.sh\n./snort_test.sh
165 ./snort_test.sh
166 cat > test_metasploitable_rules.sh << 'EOF'\n#!/bin/bash\necho "=== Testing Snort IDS/IPS Rules for Metasploitable ==="\nTARGET="192.168.56.10"\n\necho "Target: Metasploitable ($TARGET)"\necho "Attacker: Kali (192.168.56.5)"\necho ""\n\necho "1. Testing ICMP Ping to Metasploitable..."\nping -c 2 $TARGET\n\necho ""\necho "2. Testing Telnet to Metasploitable..."\ntimeout 2 bash -c "echo 'QUIT' | telnet $TARGET 23" 2>/dev/null\n\necho ""\necho "3. Testing FTP to Metasploi

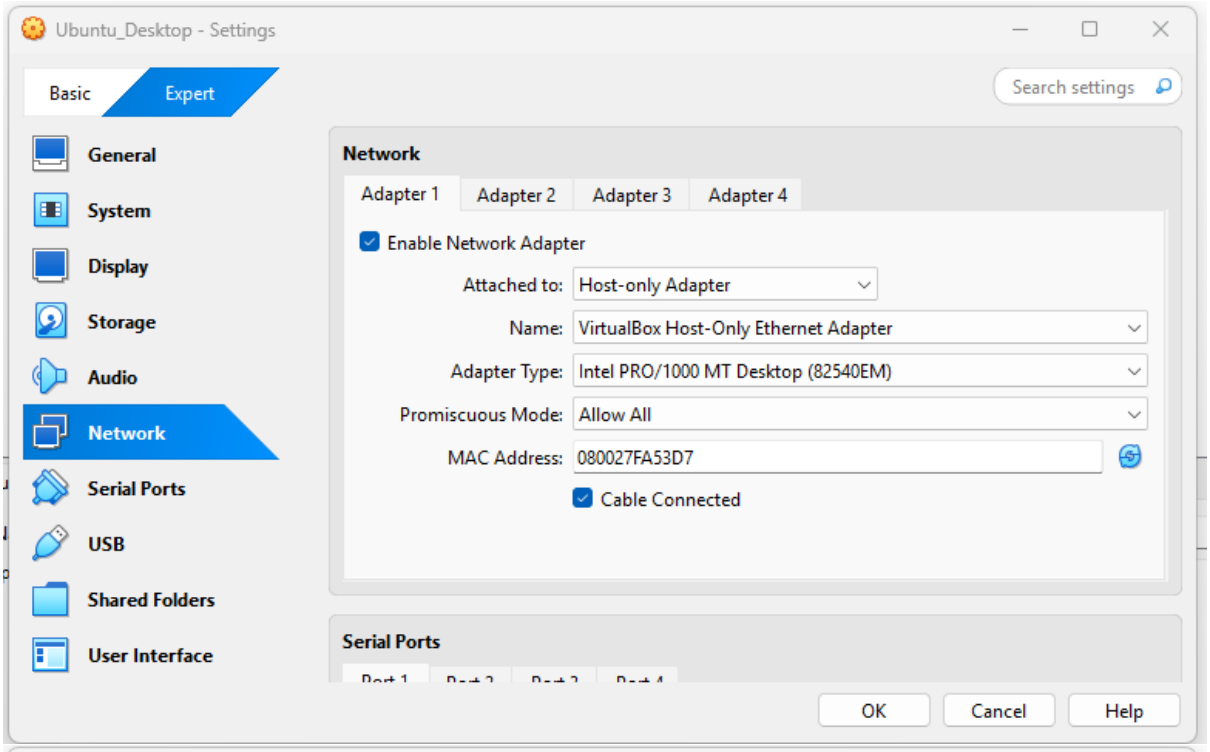

manchidi@kali: ~
ho ""\necho "5. Testing SSH connections to Metasploitable..."\nfor i in {1..3}; do\n timeout 1 bash -c "echo 'QUIT' | nc $TARGET 22" 2>/dev/null\n sleep 1\ndone\n\necho ""\necho "=== Test Complete ==="\necho "Check Snort for Metasploitable-specific alerts and blocks!"\nEOF\n\nchmod +x test_metasploitable_rules.sh\n./test_metasploitable_rules.sh
167 ./test_metasploitable_rules.sh
168 nmap -sS 192.168.56.10
169 cat > test_ips.sh << 'EOF'\n#!/bin/bash\necho "=== IPS BLOCKING TEST ==="\necho "Testing if Snort IPS BLOCKS traffic to Metasploitable"\necho "Metasploitable IP: 192.168.56.10"\necho ""\n\nTARGET="192.168.56.10"\n\necho "1. Testing ICMP Blocking..."\nping -c 2 $TARGET && echo "ICMP: PASSED (NOT BLOCKED)" || echo "ICMP: BLOCKED ✅"\n\necho ""\necho "2. Testing Service Blocking..."\necho -n "FTP (port 21): "\ntimeout 1 bash -c "echo > /dev/tcp/$TARGET/21" 2>/dev/null && echo "PASSED" || echo "BLOCKED ✅"\n\necho -n "SSH (port 22): "\ntimeout 1 bash -c "echo > /dev/tcp/$TARGET/22" 2>/dev/null && echo "PASSED" || echo "BLOCKED ✅"\n\necho -n "Telnet (port 23): "\ntimeout 1 bash -c "echo > /dev/tcp/$TARGET/23" 2>/dev/null && echo "PASSED" || echo "BLOCKED ✅"\n\necho -n "HTTP (port 80): "\ntimeout 1 bash -c "echo > /dev/tcp/$TARGET/80" 2>/dev/null && echo "PASSED" || echo "BLOCKED ✅"\n\necho ""\necho "3. Testing Vulnerable Service Blocking..."\nfor port in 3632 5432 5900 6667 8180; do\n echo -n "Port $port: "\n timeout 1 bash -c "echo > /dev/tcp/$TARGET/$port" 2>/dev/null && echo "PASSED" || echo "BLOCKED ✅"\ndone\n\necho ""\necho "4. Testing Malicious Port Blocking..."\nfor port in 4444 1337 31337; do\n echo -n "Port $port: "\n timeout 1 bash -c "echo > /dev/tcp/$TARGET/$port" 2>/dev/null && echo "PASSED" || echo "BLOCKED

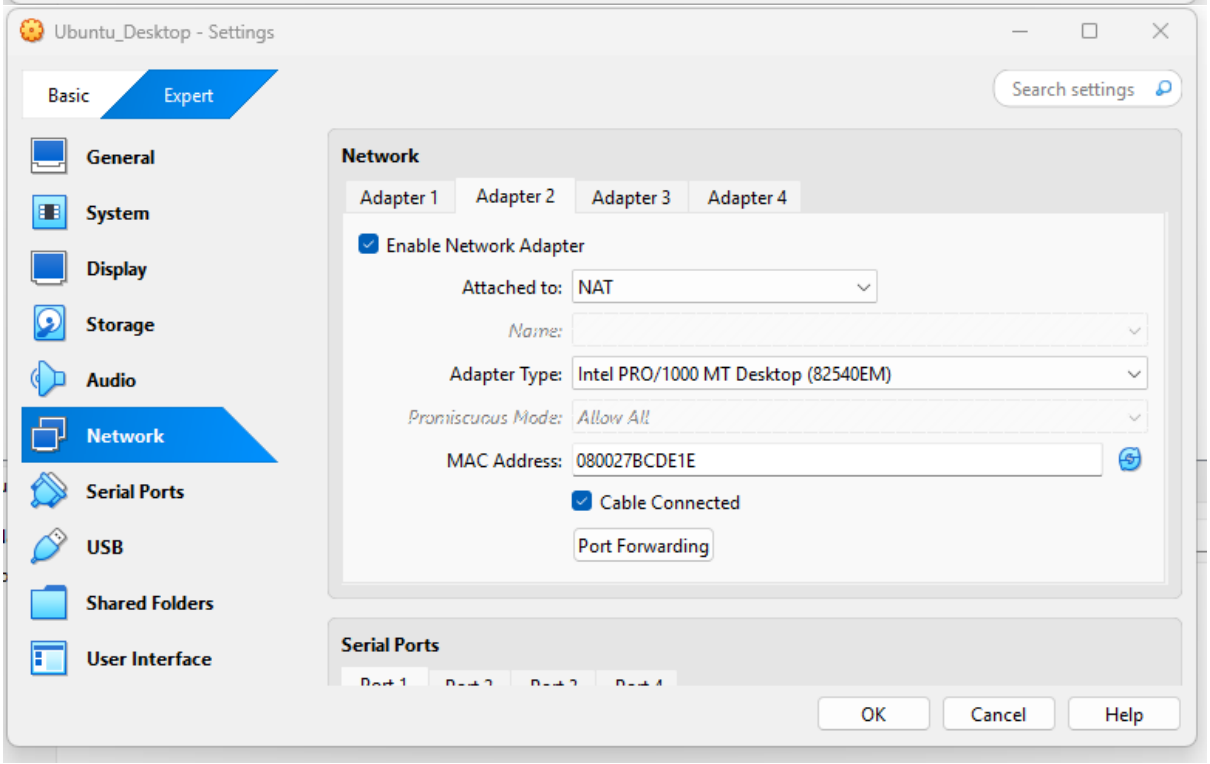

manchidi@kali: ~
170 telnet 192.168.56.10
171 ping 192.168.56.10
172 telnet 192.168.56.10
173 ifconfig
174 ping 192.168.56.10
175 # Add route to the 192.168.56.0/24 network\nsudo ip route add 192.168.56.0/24 via 192.168.1.1 dev eth0\n\n# Or if you don't know the gateway, try:\nsudo ip route add 192.168.56.0/24 dev eth0\n\n# Verify the route\nip route show
176 # Remove current IP\nsudo ip addr flush dev eth0\n\n# Add new IP in the correct subnet\nsudo ip addr add 192.168.56.5/24 dev eth0\n\n# Bring interface up\nsudo ip link set eth0 up\n\n# Add default gateway if needed (replace with your actual gateway)\nsudo ip route add default via 192.168.56.1\n\n# Test connectivity\nping 192.168.56.10
177 ping 192.168.56.103
178 ping 192.168.56.10
179 ifconfig
180 ftp 192.168.56.10
181 ping -c 4 192.168.56.10
182 telnet 192.168.56.10
183 curl http://192.168.56.10/admin\nwget http://192.168.56.10/admin\n
184 ssh msfadmin@192.168.56.10
185 nmap -p 512,513,514 192.168.56.10
186 nc -nv 192.168.56.10 6667
187 ftp 192.168.56.10

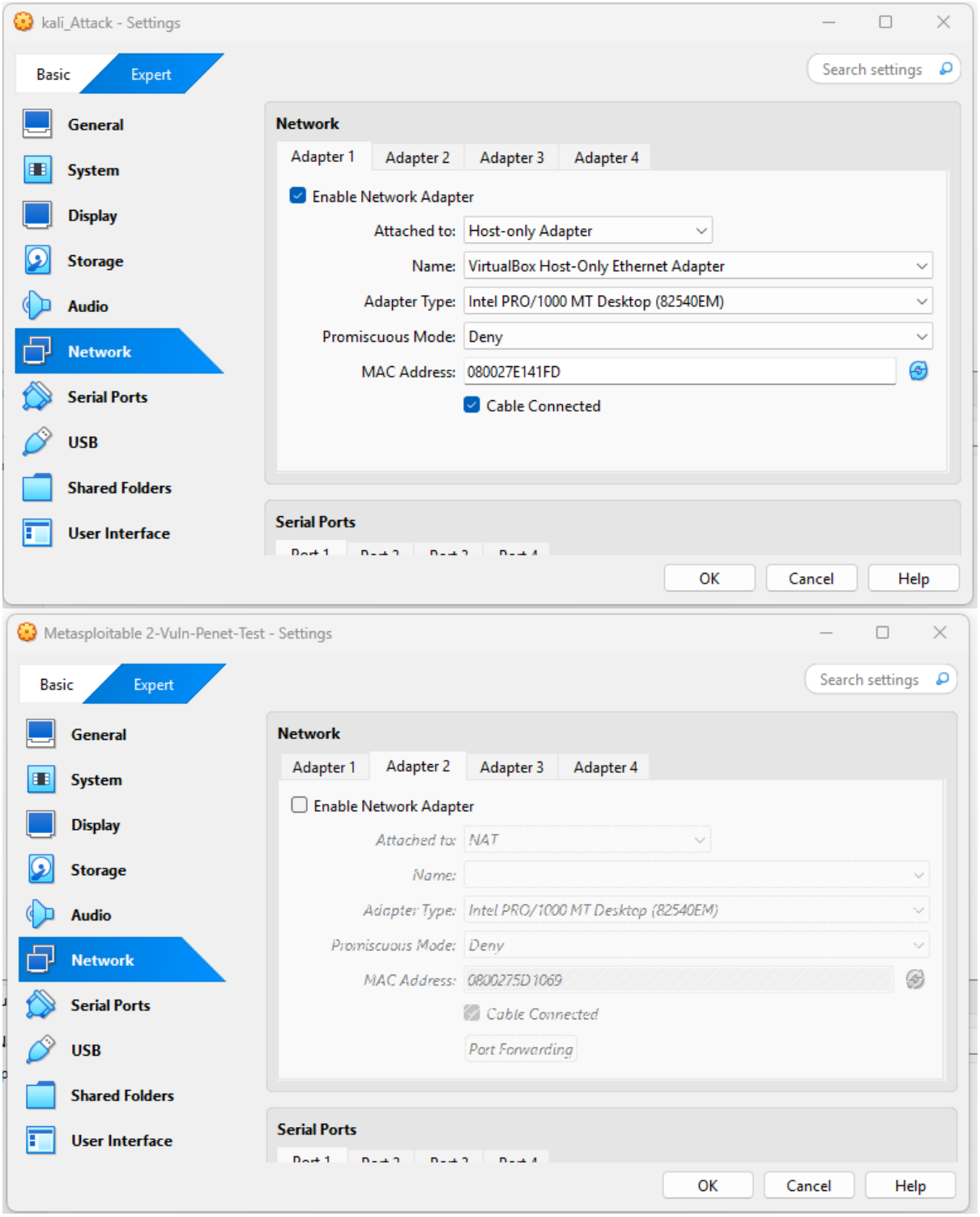

manchidi@kali: ~
ip route add 192.168.56.0/24 dev eth0\n\n# Verify the route\nip route show
176 # Remove current IP\nsudo ip addr flush dev eth0\n\n# Add new IP in the correct subnet\nsudo ip addr add 192.168.56.5/24 dev eth0\n\n# Bring interface up\nsudo ip link set eth0 up\n\n# Add default gateway if needed (replace with your actual gateway)\nsudo ip route add default via 192.168.56.1\n\n# Test connectivity\nping 192.168.56.10
177 ping 192.168.56.103
178 ping 192.168.56.10
179 ifconfig
180 ftp 192.168.56.10
181 ping -c 4 192.168.56.10
182 telnet 192.168.56.10
183 curl http://192.168.56.10/admin\nwget http://192.168.56.10/admin\n
184 ssh msfadmin@192.168.56.10
185 nmap -p 512,513,514 192.168.56.10
186 nc -nv 192.168.56.10 6667
187 ftp 192.168.56.10
188 curl "http://192.168.56.10/page.php?id=1' UNION SELECT password FROM users"
189 nmap -sN 192.168.56.10 # NULL scan\nnmap -sF 192.168.56.10 # FIN scan
(manchidi㉿kali)-[~]
$

An extensive set of network and security tests targeting a Metasploitable virtual machine at 192.168.56.103 were carried out on October 06 2025 according to the terminal logs from a Kali Linux system (user "manchidi"). The tasks involve setting up a static IP address (192.168.56.1/24) on the eth0 interface using ping and curl to confirm connectivity and using nmap to do thorough network scans to find open ports and services like SSH HTTP and FTP. While Snort IDS/IPS tests assess network security by simulating attacks and looking for blocked or passed connections scripts such as test_metasploitable_rules.sh and metasploitable_rules.sh test SSH connections evaluating encryption techniques and rule enforcement.

VIRTUALBOX MACHINES SETUP NETWORK SUMMARY







The pictures display the network configurations for several virtual computers (VMs) in VirtualBox such as Kali Attack Ubuntu Desktop and Metasploitable 2-Vuln-Penet-Test. Based on the given settings the following is a summary of each virtual machine's configurations and suggested actions:

Ubuntu Desktop: Promiscuous mode is set to "Deny All" and the network adapter is configured to "NAT" with an Intel PRO/1000 MT Desktop (82540EM) adapter type and MAC address 080027CDE1E. Although port forwarding is not set up the cable is connected. Make

sure the virtual machine is powered on and use ping to an external IP address (such as 8.8.8.8) to verify connectivity before enabling network access. Set the adapter to "Host-Only Adapter" if necessary and for local network testing set a static IP in the guest operating system.

Kali Attack: The promiscuous mode is set to "Allow All" and the network adapter is setup as a "Host-Only Adapter" with the same Intel PRO/1000 MT Desktop (82540EM) adapter type and MAC address 08027A53D7. The cable is attached. For isolated penetration testing this configuration works well. Use the guest OS's ip addr and ping 192.168.56.1 to confirm the network. Make that their IP addresses are in the same subnet (e.g. 192.168.56.x/24) when testing against other virtual machines (e.g. Metasploitable).

The network adapter is also configured to "NAT" with the Intel PRO/1000 MT Desktop (82540EM) adapter type MAC address 08027D51069 and promiscuous mode set to "Deny All" in Metasploitable 2-Vuln-Penet-Test. The cable is attached. Consider switching the adapter to a "Host-Only Adapter" for penetration testing in order to match the network of the Kali virtual machine and assign a static IP address (such as 192.168.56.103). After configuration use Kali's ping to check connectivity.